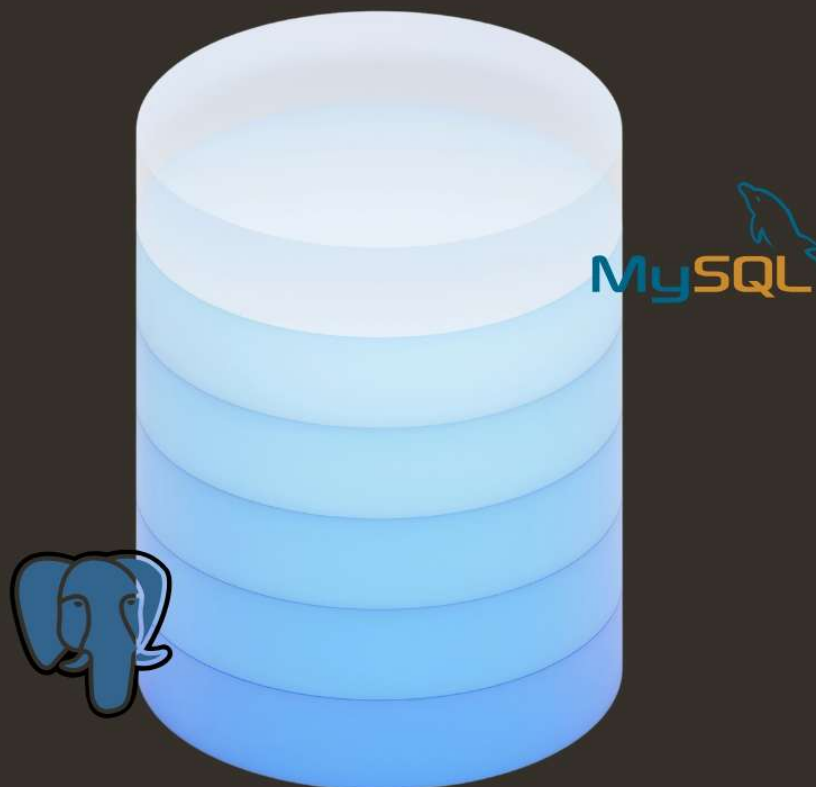


# DATA QUEST

SQL for Real-World Thinking

- Aditya Varma



# Preface

## Welcome to Data Quest SQL

This book is written to help you learn SQL as a skill, not as a subject.

SQL is not about writing long programs. It is about understanding data and asking the right questions. It focuses on clarity over complexity. Because of this, learning SQL builds a strong foundation for working with real-world data across different domains.

The chapters in this book are arranged carefully. Each concept is introduced only when it is needed, and every idea is supported by simple queries that explain *why* something works, not just *how* to write it.

You will not just learn how to write queries, but how to think in terms of data. You will learn how information is structured, how relationships are built, and how meaningful insights are extracted.

No prior database knowledge is assumed. You are expected only to read, practice, and stay curious. This book is meant to be read slowly, like a guide. Run the queries. Modify them. Observe what changes.

Take your time. Ask questions. Make mistakes. That is how real understanding is built.

*-Aditya Varma*

## LICENSE & CREDITS

### License:

This work is licensed under the **Creative Commons Attribution–NonCommercial–ShareAlike 4.0 (CC BY-NC-SA 4.0)** license. You may share and adapt this material for non-commercial purposes, provided proper credit is given and derivative works use the same license.

### Credits:

Certain examples, datasets, and reference ideas in this book are inspired by publicly available educational resources and open datasets from platforms such as Kaggle and other learning communities.

All original content, explanations, and structure in this material are created for educational purposes. Proper credit is given wherever applicable. All rights remain with their respective creators.

# Index

- 📖 Understanding Data
- 📖 Relational Database
- 📖 SQL
- 📖 SQL CRUD OPERATIONS
- 📖 CREATE Operations
- 📖 SQL Sublanguages (Command Categories)
- 📖 Data Definition Language (DDL)
- 📖 Data Query Language (DQL)
- 📖 SQL Practice
- 📖 THE END!

# Understanding DATA!

## What is Data?

Imagine waking up in the morning and checking your phone. The time you see on the screen, the number of notifications, the weather update, even the number of steps you walked yesterday all have one thing in common. They are small pieces of information describing something about the world or about you. These small pieces are what we call **data**.

Data is simply a collection of facts. It does not need to be complicated. Your name is data. Your age is data. The marks you scored in your last exam are data. Even something as simple as whether it is raining or not is also data. These are all observations, captured and recorded in some form.

On their own, these facts do not say much. The number “85” means nothing unless you know it is your exam score. The number “30” could be temperature, age, or price. Data by itself is quiet. It just exists. It waits to be understood.

When we begin to organize and interpret data, it starts becoming useful. It starts telling us something. That transformation is what makes data powerful, and that is exactly what you will learn throughout this course.

## Data and Information: The Subtle Difference

Let's take a small situation. You are given a list of numbers: 85, 90, 78, 92. At first glance, these are just numbers. There is no meaning attached. This is data.

Now someone tells you, “These are the marks of a student in four subjects.” Suddenly, your understanding changes. You might think the student is doing well. You might even calculate the average. The same numbers now carry meaning.

That meaning is called **information**.

Data is raw and unprocessed. Information is what you get after understanding and interpreting that data. This distinction may seem small right now, but it is at the heart of everything you will do with SQL. You will take raw data and turn it into meaningful information.

## **Why Do We Store Data?**

Think about how much you try to remember in a day. Names, tasks, passwords, conversations. Now imagine trying to remember the details of hundreds or thousands of things over weeks or months. It quickly becomes impossible.

This is why we store data. Consider a small grocery shop. Every day, customers come in and buy products. If the shop owner does not record anything, they will have no idea which products sell the most, which ones are rarely bought, or how much profit they are making. Everything becomes guesswork.

Now scale this up to a large company like an online shopping platform. Millions of users, millions of products, and millions of transactions happen every day. Without storing data, such a system would collapse almost instantly.

Data is stored so that it can be remembered, analyzed, and used later. It allows us to look back at what has happened and make better decisions for the future. A company can improve its services. A teacher can track student performance. A doctor can monitor patient history.

In a way, stored data acts like memory, but far more reliable and far more detailed than what a human mind can handle.

## **Data in the Real World**

Once you start noticing it, you will realize that data is everywhere, quietly working behind the scenes. When you use a shopping app, every action is recorded. The products you search for, the items you add to your cart, the things you actually buy, all of this becomes data. This is why the app seems to “know” what you might like next.

When you watch movies on a streaming platform, it remembers what you watched, how long you watched it, and what you skipped. That data is then used to recommend new movies or shows that match your taste.

Even something as simple as booking a ride involves data. Your pickup location, destination, travel time, and fare are all stored. Over time, this data helps improve routes, estimate prices, and manage drivers more efficiently.

Social media platforms take this even further. Every like, comment, share, and scroll is recorded. Even the time you spend looking at a post becomes data. It may feel invisible, but it is constantly being generated.

Without realizing it, you are producing data every single day.

## **A Brief Journey Through the History of Data**

Data has always been part of human life, even long before computers existed. In ancient times, people recorded information by carving symbols into stone or writing on walls. Kings and rulers kept records of taxes, land, and population using simple tools. These were early forms of data storage.

As societies grew, paper became the main method of recording information. Large registers and books were used to store records of businesses, schools, and governments. While this was an improvement, it had its own limitations. Finding specific information was slow, updating records was difficult, and storing large amounts of data required a lot of physical space.

The real transformation began with computers. Suddenly, data could be stored in digital form. It could be searched quickly, updated easily, and managed efficiently. This led to the creation of databases, which are designed specifically to organize and handle large amounts of data.

Today, we live in a world where data is generated at an enormous scale. Every click, every interaction, every transaction adds to the growing ocean of data. This modern era is often referred to as the age of data.

Some people describe data as the “new oil.” Not because it looks similar, but because of its value. Just like oil, raw data is not very useful until it is processed. But once it is refined and analyzed, it becomes incredibly powerful.

## **Different Forms of Data**

Not all data looks neat and organized. In fact, most of it is quite messy. Some data is structured, like a table with rows and columns. This is the kind of data you might see in a spreadsheet, where everything is clearly organized. This is also the type of data you will work with most when using SQL.

Other types of data are less organized. For example, messages, images, videos, and audio files do not fit neatly into rows and columns. These are still data, but they require different ways of handling and processing. For now, it is enough to understand that data comes in many forms, but structured data is where your journey with SQL will begin.

## **From Data to Databases**

Imagine trying to manage student records by writing them on random pieces of paper. Some names are here, some marks are there, and some details are missing altogether. It would be confusing and inefficient. Now imagine organizing all that information into a table where each student has a row, and each detail has its own column. Suddenly, everything becomes clear and easy to manage.

This organized collection of data is called a **database**.

A database allows us to store data in a structured way so that it can be easily accessed, updated, and analysed. It brings order to what would otherwise be chaos.

## **Why This Chapter Matters**

At this point, you might feel that everything here is simple, almost obvious. And that is exactly the point. Before learning how to write SQL queries, you need to understand what you are working with. SQL is not just about writing commands. It is about asking questions to data and getting meaningful answers.

If you understand what data is, why it is stored, and how it is used, the rest of the journey becomes much easier.

## **Closing Thought**

Right now, data may seem like nothing more than numbers, text, and records. Quiet, passive, and uninteresting.

But as you move forward, you will learn how to interact with it. You will learn how to filter it, combine it, and analyse it. And when that happens, something changes. Data stops being silent. It starts telling stories.

## // Why Data Matters

If data were just stored and never used, it would be like writing a diary and never reading it again. The real value of data comes from what we *do* with it.

To understand why data matters, imagine running a small café.

Every day, customers walk in and order coffee, tea, snacks. You notice that sometimes the café is crowded, and sometimes it is empty. You might have a feeling that weekends are busier, or that people prefer cold coffee in the afternoon. But these are just guesses.

Now imagine you start recording everything. What customers ordered, at what time, on which day. After a few weeks, you look at the data and notice clear patterns. Cold coffee sells more between 2 PM and 5 PM. Weekends bring in twice as many customers. One particular snack is rarely ordered.

Suddenly, you are no longer guessing. You are making decisions based on facts. This is why data matters. It replaces assumptions with clarity.

### **Data and Decision Making**

Every important system today runs on decisions, and most of those decisions are powered by data.

Think about a navigation app. When you enter a destination, it suggests the fastest route. It is not guessing. It is using data collected from thousands of users, traffic patterns, road conditions, and time of day.

Banks use data to decide whether to approve a loan. They look at income, past transactions, credit history, and other factors. The decision is based on patterns found in data, not just human judgment.

Even in something as simple as a classroom, a teacher can use data to track student performance. Instead of relying on memory, they can see exactly which topics students struggle with and adjust their teaching.

Data helps answer questions like:

- What is happening?
- Why is it happening?
- What should we do next?

Without data, decisions become guesses. With data, decisions become informed.



## **Data and Analytics**

If storing data is like collecting puzzle pieces, analytics is the process of putting those pieces together to see the full picture.

Analytics is about exploring data to discover patterns, trends, and insights.

Consider a movie streaming platform. Millions of users watch content every day. The platform collects data about what people watch, how long they watch, what they skip, and what they search for.

By analysing this data, the platform can answer questions like:

- Which genres are popular right now?
- Which shows are being watched completely?
- What type of content keeps users engaged longer?

This is how recommendations work. When a platform suggests a movie “you might like,” it is not random. It is based on data analysis.

Businesses use analytics to understand customers, improve products, and increase profits. Governments use it to study population trends. Sports teams use it to analyze player performance.

Analytics turns raw data into useful knowledge.

## **Data and Automation**

Now imagine taking it one step further. Not only are we analyzing data, but we are also using it to make systems act automatically.

This is where automation comes in.

Think about an online shopping app. When a product goes out of stock, the system can automatically detect it and notify the seller. When a user places an order, the system processes payment, updates inventory, and sends a confirmation message without any human involvement.

All of this happens because data is being continuously monitored and used to trigger actions.

Another example is email filtering. Your inbox automatically separates important emails from spam. It learns from data, patterns, and behaviour to make these decisions.

Automation saves time, reduces errors, and allows systems to operate at a scale that humans alone cannot manage.

In simple terms:

- Data tells us what is happening
- Analytics helps us understand it
- Automation helps us act on it

## **Types of Data**

So far, we have been talking about data as if it is all the same. But in reality, data comes in different forms, and not all of it is neatly organized.

Understanding these types will help you see why SQL is designed the way it is.

- **Structured Data:**

Structured data is the most organized form of data. It is arranged in a clear format, usually in rows and columns, like a table.

Imagine a bank storing customer details. Each customer has:

- a name
- an account number
- a balance

This information fits perfectly into a table. Each row represents a customer, and each column represents a specific type of information. This kind of data is easy to store, search, and analyse. It follows a fixed structure, which makes it ideal for databases and SQL.

Whenever you see data in spreadsheets or tables, you are looking at structured data.

- **Semi-Structured Data:**

Not all data fits perfectly into tables. Sometimes, data has some organization, but not a rigid structure. This is called semi-structured data.

For example, consider a user profile stored in a flexible format. One user might have a phone number and address, while another might only have an email. The structure is not fixed, but there is still some pattern. Formats like JSON and XML are used to represent this kind of data.

It is like having notes written in a consistent style, but not in strict rows and columns. You can still understand it, but it is not as neatly organized as structured data.

- **Unstructured Data**

Now imagine data that has no predefined structure at all. This includes:

- images
- videos
- audio files
- text messages

A photo on your phone is data. A voice recording is data. A long paragraph of text is also data. But you cannot easily put these into a table with rows and columns. They require different methods to store and process.

Interestingly, most of the data in the world today is unstructured. Think about how many photos, videos, and messages are created every second.

## **Real-World Examples of Data Types**

To make this clearer, let's connect these types to real-world systems.

In a bank, most of the data is structured. Account numbers, balances, and transaction records are all neatly organized in tables. This is why SQL is heavily used in banking systems.

On a platform like Instagram, things are more mixed. Usernames and follower counts are structured data. But photos, videos, and captions are unstructured. Comments and messages may fall somewhere in between.

Amazon uses all three types. Product listings and prices are structured. Product descriptions and reviews can be semi-structured. Images and videos of products are unstructured.

These systems combine different types of data to create a complete experience.

## // What is a Database?

Imagine a school office. There are hundreds of students, each with records like name, class, marks, attendance, and contact details. Now imagine all of this information written on random sheets of paper and thrown into a drawer.

Finding one student's record would be a nightmare.

So instead, the school organizes everything neatly. Each student has a proper record. Everything is stored in a structured way so it can be easily found, updated, or reviewed.

That organized collection of data is what we call a **database**. A database is simply a system designed to **store, manage, and organize data efficiently**.

It is not just about storing data. It is about storing it in a way that makes it:

- easy to access
- easy to update
- easy to analyse

Think of a database as a highly disciplined librarian. Nothing is misplaced, everything has a position, and when you ask a question, it knows exactly where to look.

### Why Not Just Use Excel?

At this point, a very natural question comes up. "If databases store tables, why not just use Excel?"

It's a fair question, because Excel *does* look similar to a database at first glance. Rows, columns, and data all sitting neatly in cells. For small tasks, Excel works perfectly fine. You can store marks of a class, track expenses, or manage a small list of items. But problems begin when things grow.

Imagine trying to manage a large online store using Excel. Thousands of products, customers, and orders. Multiple people trying to access and update the file at the same time. The file becomes slow, errors start appearing, and maintaining consistency becomes difficult.

Now imagine a bank trying to store account details in Excel. One mistake, one accidental deletion, and critical data could be lost or corrupted. There is no strong control over who can access or modify what. Databases are built to handle these problems.

They can manage huge amounts of data without slowing down. Multiple users can access the system at the same time without breaking anything. They provide security, consistency, and reliability.

Excel is like a notebook. A database is like a full library system. Both are useful, but they serve very different scales.

## **Database vs File System**

Before databases became common, data was often stored using simple file systems.

A file system is what you use when you store files on your computer. You might have folders like “Documents,” “Photos,” or “Projects,” and inside them, files containing information. This works well for basic storage. But imagine trying to manage complex data using only files.

Suppose you are building a system for a college. You create:

- one file for students
- one file for courses
- one file for results

Now, what happens when you want to find all students enrolled in a specific course? You would have to open multiple files, search manually, and somehow connect the information. This quickly becomes messy and inefficient.

Databases solve this problem by allowing data to be connected and queried easily. Instead of jumping between files, you can ask one clear question and get a precise answer. For example, instead of manually checking multiple files, you can simply ask: “Show me all students enrolled in this course.” And the database will give you the answer instantly.

Another issue with file systems is duplication. The same data might be stored in multiple files, leading to inconsistencies. If one copy is updated and another is not, errors start creeping in. Databases reduce duplication and maintain consistency by organizing data properly and linking related pieces together.

In simple terms:

- File systems store data
- Databases manage data intelligently

## Types of Databases

As data grew in size and complexity, different types of databases are created to handle different needs.

- **Relational Databases (SQL):**

Relational databases are the most common and the most structured type. They store data in tables. Each table has rows and columns. Different tables can be connected using relationships. For example, in an online store: one table stores customers, other stores orders, other stores products.

These tables are connected. A customer places orders. Orders contain products. Everything is linked. This is why they are called **relational** databases.

To work with these databases, we use SQL, which allows us to:

- retrieve data
- insert new data
- update existing data
- delete unwanted data

Relational databases are used in:

- banking systems
- e-commerce platforms
- school management systems

They are reliable, structured, and perfect for handling organized data.

- **NoSQL Databases (Basic Idea):**

As applications grew more complex, especially with social media and large-scale systems, a new type of database became popular: **NoSQL**. Unlike relational databases, NoSQL databases do not rely strictly on tables. They are flexible & can store data in different formats.

For example, instead of forcing all users to have the same structure, a NoSQL database can store different types of data for different users.

This is useful for applications where:

- data structure changes frequently
- large amounts of unstructured data are involved
- flexibility is more important than strict organization

NoSQL databases are commonly used in:

- social media platforms
- real-time applications
- big data systems

For now, you do not need to go deep into this. Just remember: Relational databases are structured and table-based. NoSQL databases are flexible and less structured.

## Examples of Databases

Let's look at some real tools that are used in the industry.

- **MySQL:**

MySQL is one of the most widely used relational databases. It is known for being simple, fast, and reliable. Many websites and applications use MySQL to store their data.

If you have ever used a website that requires login, there is a good chance MySQL is working in the background.

- **PostgreSQL:**

PostgreSQL is another powerful relational database. It is known for being more advanced and feature-rich. It supports complex queries and is often used in systems that require high performance and reliability.

Many modern applications prefer PostgreSQL because of its flexibility and strong capabilities.

- **MongoDB:**

MongoDB is a popular NoSQL database. Instead of storing data in tables, it stores data in a flexible format similar to JSON.

This makes it easier to handle data that does not fit neatly into rows and columns, such as user profiles with varying details.

It is widely used in modern web applications, especially those dealing with large and dynamic data.

## // What is a DBMS?

The DBMS quietly handles many important responsibilities behind the scenes. It allows users to store new data, update existing data, and retrieve information whenever needed. It also ensures that multiple users can work on the same database without interfering with each other.

For example, in a banking system, thousands of people might be checking balances, transferring money, or updating details at the same time. The DBMS makes sure all of this happens smoothly without conflicts or data loss.

It also provides security. Not everyone should have access to everything. A customer can view their account, but only authorized employees can modify certain details. The DBMS controls who can see and change data.

Another important role is maintaining consistency. If data is updated in one place, the DBMS ensures that the change is reflected correctly everywhere it is needed.

In short, the DBMS makes working with data reliable, secure, and efficient.

### Examples of DBMS:

Some common DBMS software used in the real world include:

- MySQL, which is widely used in web applications
- PostgreSQL, known for its advanced features and reliability
- MongoDB, used for flexible and large-scale data

These tools are what developers and companies use to actually work with databases.

## Basic Terminology

Before going further, it is important to become comfortable with a few basic terms. These are the building blocks of everything you will do in SQL.

- **Table:** A table is where data is stored inside a database. It is organized in a structured way, similar to a spreadsheet.  
For example, a “students” table might contain details like names, ages, and marks. Each type of data has its own column, and all related information is grouped together.
- **Row:** A row represents a single entry in a table. If you imagine a table of students, each row would represent one student. It contains all the information related to that specific record.
- **Column:** A column represents a specific type of data within a table. For example, in a student table, columns could be “name,” “age,” and “marks.” Each column stores one kind of information for all rows.
- **Record:** A record is simply another word for a row. It refers to a complete set of information about one item. So, one student’s full details in a table would be called a record.



- **Schema:** A schema defines the structure of a database. It describes how data is organized, including what tables exist, what columns they have, and how they are related to each other.

You can think of a schema as a blueprint. Just like a building plan shows how rooms are arranged, a schema shows how data is structured inside a database.

## **Closing Thought**

At first, these terms may feel simple, almost too basic. But they are the language of databases. As you move forward and start writing SQL queries, you will use these concepts again and again. Tables, rows, columns, and schemas will become second nature.

Right now, you are learning the names of the pieces.

# Relational Database

## Understanding Relationships in Data

Imagine you are managing a college. You have information about students. You also have information about courses. And somewhere, there is information about which student is enrolled in which course. At first, you might think of putting everything into one big table:

- student name
- course name
- marks
- teacher

But very quickly, problems begin to appear.

If one student is enrolled in three courses, their name will be repeated three times. If a course has fifty students, the course name will be repeated fifty times. The data starts becoming repetitive, messy, and harder to maintain.

Now imagine a better approach. You create one table for students, one table for courses, and another table that connects them. Instead of repeating data again and again, you link related information together. This idea of organizing data into separate tables and connecting them using relationships is the foundation of a **relational database**.

## What Does “Relational” Actually Mean?

The word *relational* comes from the idea of **relationships between data**. Instead of storing everything in one place, data is divided into multiple tables, and these tables are connected using common fields.

For example:

- A student has an **ID**
- A course has an **ID**
- An enrolment table connects student IDs with course IDs

Now, instead of repeating full information, the system simply uses these IDs to relate data.

This makes the database:

- more organized
- less repetitive
- easier to update

If a student's name changes, you update it in one place. Everywhere else, the relationship remains intact.

## Why Relational Databases Matter

Relational databases are not just a design choice. They solve real problems that appear when data grows.

Imagine a company managing customer orders. If customer details are stored again and again with every order, even a small mistake can create inconsistencies. One order might have a slightly different spelling of the customer's name. Another might have outdated contact details.

By separating customers and orders into different tables and linking them, the system avoids duplication and keeps data clean. This approach improves:

- accuracy
- consistency
- efficiency

It also makes querying data much easier. Instead of searching through large, repetitive datasets, the system can quickly connect the right pieces of information.

## A Simple Real-World Example

Let's take a familiar system: an online shopping platform. Instead of storing everything in one place, the system is divided into multiple tables:

- Customers (who is buying)
- Products (what is being sold)
- Orders (what was purchased)

Now imagine you want to answer a question: "Which customers bought a specific product?" Without relationships, this would require scanning through large, repeated data. With a relational structure, the system simply connects: customers → orders → products; and gives you the answer efficiently. This is the power of relational thinking.

## The Idea of Breaking Things Down

One of the most important ideas in relational databases is this: **Do not store everything together. Break it into meaningful parts.**

This might feel counterintuitive at first. Many beginners try to put everything into one table because it seems easier. But in reality, breaking data into smaller, connected tables makes systems:

- easier to manage
- easier to scale
- easier to understand

It is similar to organizing your room. Instead of putting everything in one big box, you use different sections for clothes, books, and other items. Finding things becomes faster and more logical.

## **Where You See This in Real Life**

Relational databases are used in almost every system that handles structured data.

- In a bank, customer details, accounts, and transactions are stored in separate tables but connected through relationships.
- In a school, students, teachers, classes, and results are all linked together.
- In a social media platform, users, posts, and comments are stored separately but connected through relationships.

These systems rely on relational concepts to keep data organized and meaningful.

## // Tables and Structure

Imagine you are managing a small school. On your first day, you decide to store student information. You open a notebook and start writing:

“John, 20, Maths, 85”

“Sarah, 19, Science, 90”

“John, 20, English, 78”

At first, it seems fine. But look closer. The same student appears multiple times. Information is repeated. If John’s age needs updating, you now have to change it everywhere. This is where tables come in.

Instead of writing everything randomly, you organize data into a structured format:

| Student ID | Name  | Age |
|------------|-------|-----|
| 1          | John  | 20  |
| 2          | Sarah | 19  |

And then another table:

| Student ID | Subject | Marks |
|------------|---------|-------|
| 1          | Maths   | 85    |
| 1          | English | 78    |
| 2          | Science | 90    |

Now things start making sense. Data is no longer repeated unnecessarily. Each table has a clear purpose. Information is connected, not duplicated. A **table** is simply a structured way of storing data in rows and columns. But more importantly, it is a way of **thinking clearly about data**.

### Table Design

Designing a table is not just about creating columns. It is about deciding *what belongs together* and *what should be separated*. A common mistake beginners make is trying to put everything into one table because it feels easier. But this often leads to messy, repetitive, and confusing data.

Let’s take a real-world example. Imagine an online store. You might be tempted to create one big table like this:

| Customer Name | Product | Price | Order Date |
|---------------|---------|-------|------------|
| Alice         | Phone   | 500   | Jan 1      |
| Alice         | Laptop  | 1000  | Jan 5      |

At first, it works. But soon problems appear:

- Customer data is repeated again and again
- If Alice changes her name or details, you must update multiple rows
- Data becomes harder to maintain

A better design would be:

- **Customers Table**

| ID | Name  |
|----|-------|
| 1  | Alice |

- **Orders Table**

| Order ID | Customer ID | Date  |
|----------|-------------|-------|
| 101      | 1           | Jan 1 |
| 102      | 1           | Jan 5 |

- **Products Table**

| Product ID | Name   | Price |
|------------|--------|-------|
| 1          | Phone  | 500   |
| 2          | Laptop | 1000  |

Now everything is cleaner. Each table has one responsibility. Data is connected instead of repeated.

Good table design is about:

- reducing repetition
- keeping data consistent
- making future changes easier

It may take a little more thinking at the start, but it saves a lot of trouble later.

## // Keys: Giving Identity to Data

Imagine a classroom again. There are many students, and some of them may even share the same name. If you call out “John,” more than one student might respond.

So how do schools solve this? They assign a **roll number** to every student. That number is unique. No two students share it. It becomes their identity inside the system.

This is exactly what keys do in a database. Keys help us uniquely identify data and connect it across tables.

### Primary Key:

A **primary key** is like that roll number. It is a column (or a set of columns) that uniquely identifies each row in a table. No two rows can have the same primary key value, and it cannot be empty.

Imagine a **students** table:

| student_id | name  | age |
|------------|-------|-----|
| 1          | John  | 20  |
| 2          | Sarah | 19  |

Here, **student\_id** is the primary key. Even if there are two students named John, their IDs will always be different. Without a primary key, it becomes very difficult to uniquely identify and manage records. Updates, deletions, and relationships would all become unreliable.

### Candidate Key:

Sometimes, more than one column can uniquely identify a record. For example, in a student table:

- **student\_id** is unique
- email might also be unique

Both of these can act as identifiers. These are called **candidate keys**.

A candidate key is any column that *could* be used as a primary key. Out of all candidate keys, one is chosen as the primary key.

Think of it like this: A person can be identified by passport number, national ID, or email. All are valid, but the system chooses one as the main identifier.

**Foreign Key:**

Now comes the part where tables start talking to each other. A **foreign key** is a column in one table that refers to the primary key in another table.

Let's go back to the school example. You have a **students** table:

| student_id | name  |
|------------|-------|
| 1          | John  |
| 2          | Sarah |

And a marks table:

| student_id | subject | marks |
|------------|---------|-------|
| 1          | Maths   | 85    |
| 1          | English | 78    |
| 2          | Science | 90    |

Here, **student\_id** in the marks table is a **foreign key**. It connects each row in the marks table to a student in the **students** table.

This is how databases avoid repetition. Instead of copying full student details into the marks table, they just use the ID to link the data. Foreign keys create connections. They turn separate tables into a unified system.



## // Relationships: Connecting the Data

Keys help identify data. Relationships define how data is connected. Let's explore the different types of relationships using simple real-world scenarios.

### One-to-One Relationship:

In a one-to-one relationship, one record in a table is linked to exactly one record in another table. Imagine a system where each person has one passport.

| Person | Passport |
|--------|----------|
| John   | P123     |
| Sarah  | P456     |

Each person has only one passport, and each passport belongs to only one person. This type of relationship is not very common but is used when data needs to be separated for clarity or security.

### One-to-Many Relationship:

This is the most common type of relationship. In a one-to-many relationship, one record in a table can be linked to multiple records in another table.

Think about a teacher and students. One teacher teaches many students, but each student is assigned to one teacher. Or consider a customer placing orders. One customer can place multiple orders, but each order belongs to one customer.

This is how it looks conceptually:

- One customer → many orders
- One student → many marks

This type of relationship is everywhere in real-world systems.

### Many-to-Many Relationship:

Now imagine a situation where both sides can have multiple connections. Students and courses are a perfect example. A student can enroll in multiple courses. A course can have multiple students. This creates a many-to-many relationship.

But relational databases cannot directly handle this type of relationship in a single step. So we introduce a third table, often called a "junction table."

For example:

**Students Table**  
| id | name |

**Courses Table**  
| id | course |

**Enrollments Table**  
| student\_id | course\_id |

This third table connects students and courses, allowing multiple connections on both sides.

### **A Small Story to Tie It Together:**

Imagine building a social media platform. You have users, posts, and comments.

Each user has a unique ID (primary key).

Each post is linked to a user (foreign key).

Each comment is linked to both a user and a post.

Now relationships start forming:

- One user → many posts
- One post → many comments
- Many users ↔ many posts (through likes, shares, etc.)

Without keys and relationships, all this data would be scattered and repetitive. With them, everything becomes structured and connected.

## // Constraints: Setting Rules for Data

Imagine you are designing a form for a school admission system. You would never want:

- a student without a name
- two students with the exact same roll number
- missing important information

So you add rules. The form won't accept invalid or incomplete entries.

In databases, these rules are called **constraints**. Constraints ensure that the data being stored follows certain conditions. They protect the database from incorrect or messy data.

### **NOT NULL:**

Some information is too important to be left empty. Imagine a student record without a name. Or a bank account without an account number. That would not make sense. The **NOT NULL** constraint ensures that a column cannot have an empty value.

For example, in a students table, the name field should always have a value. By applying NOT NULL, the database will reject any attempt to insert a record without a name. It is like saying, "This field is required. You cannot skip it."

### **UNIQUE:**

Now imagine assigning roll numbers to students. If two students accidentally get the same roll number, confusion is guaranteed. Records may overlap, updates may affect the wrong person, and the system loses reliability. The **UNIQUE** constraint ensures that all values in a column are different.

For example, an email address in a user system is usually unique. No two users should have the same email. This constraint protects identity. It makes sure that certain fields remain one-of-a-kind.

### **DEFAULT:**

Sometimes, when data is not provided, you still want a value to be filled automatically. Imagine a new user signing up on a website. If they do not specify their account status, the system might automatically set it to "active." This is where the **DEFAULT** constraint is useful.

It assigns a predefined value when no value is given. It is like saying, "If nothing is specified, assume this." This helps keep data consistent and avoids unnecessary empty values.

## **Why Constraints Matter?**

Without constraints, a database becomes unreliable very quickly. You might end up with:

- missing critical information
- duplicate entries
- inconsistent data

Constraints act like guards at the gate. They ensure that only valid and meaningful data enters the system.

## // Normalization: Keeping Data Clean

Now let's talk about something slightly bigger than constraints. Imagine you are storing customer and order data in one table:

| Customer Name | Product | Price |
|---------------|---------|-------|
| Alice         | Phone   | 500   |
| Alice         | Laptop  | 1000  |

At first, it looks fine. But notice something:

- “Alice” is repeated multiple times
- If her name changes, you must update multiple rows
- If you miss one update, data becomes inconsistent

This repetition is called **data duplication**, and it creates problems. Normalization is the process of organizing data to **reduce duplication and improve consistency**. It is like cleaning and organizing a messy room so that everything has its own place.

### Why Avoid Duplicate Data?

Duplicate data might seem harmless at first, but it leads to serious issues over time. If the same information exists in multiple places:

- updating becomes difficult
- mistakes become more likely
- storage is wasted
- data becomes unreliable

By reducing duplication, we make the system cleaner and easier to manage.

### First Normal Form (1NF):

The first step in normalization is called **First Normal Form (1NF)**. The idea is simple: Each column should contain **atomic values**, meaning no multiple values in a single field.

Imagine a table like this:

| Name | Subjects       |
|------|----------------|
| John | Maths, English |

Here, the “Subjects” column contains multiple values. This makes it harder to query and manage.

In 1NF, we break this into:

| Name | Subject |
|------|---------|
| John | Maths   |
| John | English |

Now each cell contains a single value. The data becomes easier to work with. 1NF is about making sure data is stored in a clean, simple structure.

### Second Normal Form (2NF):

Once data is in 1NF, the next step is **Second Normal Form (2NF)**. This focuses on removing **partial dependency**, which simply means that data in a table should depend on the entire key, not just part of it.

Let's simplify this with an example. Imagine a table:

| Student ID | Course | Student Name |
|------------|--------|--------------|
|------------|--------|--------------|

Here:

- Student ID + Course together identify each row
- But "Student Name" depends only on Student ID, not on the course

This creates unnecessary repetition of the student's name.

To fix this, we split the table:

**Students Table**  
| Student ID | Name |

**Enrollments Table**  
| Student ID | Course |

Now, each piece of data lives where it belongs. 2NF is about separating data so that each table has a clear purpose and no unnecessary repetition.

### Closing Thought:

Constraints and normalization may seem like rules and restrictions, but they actually make your life easier. They prevent errors before they happen. They keep data clean and meaningful. They make systems easier to manage as they grow.

At this stage, you are learning how to design data properly. Very soon, you will start using SQL to interact with that data. And when the structure is clean, everything you do with SQL becomes smoother, faster, and far more powerful.

## // Relational Database Systems in the Real World

So far, we've been talking about databases in a general sense. But in reality, you never interact with an "abstract database." You always use a **database system**, which is actual software installed on a computer.

Think of it like this: A database is the idea of storing organized data. A database system is the actual tool that makes it possible. These systems are used everywhere, from small apps to massive global platforms.

### Database vs SQL (Important Distinction):

Before going further, it is very important to clear one common confusion. A **database** is where data is stored. **SQL** is the language used to interact with that data.

It's similar to this:

- A library stores books
- Language is what you use to ask the librarian for a book

You don't "store data in SQL." You use SQL to talk to a database system.

For example:

- PostgreSQL stores the data
- SQL is used to query that data

Once students understand this difference, everything becomes much clearer.

### Popular Relational Database Systems

Let's explore some of the most widely used relational databases in the world. These are not just academic tools. These are the engines running real applications you use every day.



- **MySQL:**

MySQL is one of the most popular relational databases, especially in web development. It is known for being simple to use, fast, and reliable. Many websites and applications use MySQL as their backend database.

Imagine a basic e-commerce website. When a user logs in, views products, or places an order, all that data is stored and managed using a system like MySQL. It became extremely popular because it is open-source and works well with many programming languages.

Real-world usage includes:

- small to medium websites
- blogging platforms
- basic web applications

If you have ever used a login system on a website, there is a high chance MySQL was involved somewhere behind the scenes.

- **PostgreSQL:**

PostgreSQL, often called “Postgres,” is a more advanced and powerful relational database. It is designed for systems that require complex operations, large-scale data handling, and strong reliability. While MySQL focuses on simplicity, PostgreSQL focuses on **capability and precision**.

For example, a financial system that tracks transactions must be extremely accurate. Even a small error can cause serious problems. PostgreSQL is often chosen in such cases because of its strong support for data integrity and advanced features.

Real-world usage includes:

- banking systems
- large-scale applications
- data-heavy platforms

Many modern companies prefer PostgreSQL because it can handle both simple and complex tasks very well.

- **SQLite:**

SQLite is quite different from the others. It is a lightweight, file-based database. This means it does not run as a separate server. Instead, it exists as a simple file on your system. Because of this, it is extremely easy to use and requires almost no setup.

Imagine building a mobile app. You need to store user data locally on the device. Running a full database server would be unnecessary. Instead, SQLite can handle everything inside a small file. Many apps on your phone use SQLite without you even realizing it.

Real-world usage includes:

- mobile applications
- small desktop applications
- embedded systems

- **Oracle Database:**

Oracle is a powerful, enterprise-level database system. It is used by large organizations that deal with massive amounts of data and require high performance, security, and reliability.

Unlike MySQL or PostgreSQL, Oracle is not typically used for small projects. It is designed for large-scale systems like corporate applications, government databases, and major financial institutions. Imagine a multinational company managing millions of transactions every day across different countries. Oracle is built for that level of complexity.

Real-world usage includes:

- large enterprises
- government systems
- high-security environments

- **Microsoft SQL Server:**

SQL Server is widely used in companies that rely on Microsoft technologies. It integrates well with tools like .NET and Azure, making it popular in enterprise environments.

You will often find it in:

- corporate systems
- internal business applications
- enterprise dashboards

- **Amazon Aurora:**

Aurora is a cloud-based database built by Amazon. It is compatible with MySQL and PostgreSQL but designed to handle large-scale cloud workloads efficiently.

Used in:

- cloud applications
- scalable web systems

- **Google Cloud SQL:**

This is not a new database engine, but a managed service that runs MySQL, PostgreSQL, or SQL Server in the cloud. It removes the need to manage infrastructure manually.

### **Putting It All Together:**

At this point, the landscape might feel large, but there is a simple way to think about it.

- Small apps → SQLite
- Web apps → MySQL / PostgreSQL
- Growing systems → PostgreSQL / MariaDB
- Enterprises → Oracle / SQL Server / Db2
- Cloud systems → Aurora / Cloud SQL
- Distributed systems → CockroachDB

Different tools for different needs.



# SQL

At this point in your journey, you already understand that data lives inside databases, neatly organized into tables with rows and columns. But having data stored somewhere is only half the story. The real question is: *how do we actually interact with it?*

This is where SQL comes in.

SQL, which stands for **Structured Query Language**, is the language we use to communicate with a database. It acts as a bridge between you and the stored data. Without SQL, the database would just sit there silently, holding information but offering no easy way to access or modify it.

Imagine walking into a massive library with millions of books but no catalog system, no librarian, and no way to search. You would be surrounded by information, yet unable to use it effectively. SQL solves this exact problem for databases. It allows you to ask precise questions and get exact answers.

SQL is a language used to **communicate with a database**. It allows you to:

- ask questions
- retrieve data
- add new data
- update existing data
- delete unwanted data

If a database is where data lives, then SQL is how you talk to it.

When you write SQL, you are not writing long programs. You are forming requests. These requests can be as simple as “show me all records” or as specific as “show me customers who bought a product in the last 7 days and spent more than a certain amount.”

## Understanding “Structured Query Language”

The name itself tells you a lot about what SQL does.

“Structured” refers to the fact that SQL works with structured data, meaning data that is organized in tables. Unlike random text or images, this kind of data follows a clear format, which makes it easier to search and analyse.

“Query” means asking a question. In SQL, a query is simply a request for data. Every time you use SQL, you are essentially asking the database something.

“Language” means it has its own syntax and rules, just like any other programming language. But unlike most programming languages, SQL is designed to be closer to human thinking. Many SQL statements read almost like plain English, which makes them easier to understand and remember.

When you write a query like: `SELECT name FROM students WHERE age > 18;`

You are simply saying: “Give me the names of students whose age is greater than 18.”

You are not telling the database how to search through rows or how to compare values internally. The database system figures that out for you.

This brings us to one of the most important ideas in SQL.

## **The Declarative Nature of SQL (WHAT vs HOW)**

One of the most important ideas in SQL, and one that often feels different at first, is that SQL is a **declarative language**.

In most programming languages like Java or Python, you are required to tell the computer exactly how to perform a task. You break down the process step by step. For example, if you want to find all students older than 18, you would typically loop through a list, check each value, and store the results.

In SQL, you don’t do that. Instead, you simply state what you want, and the database system figures out how to get it.

For example, when you write a query to find students above a certain age, you are not telling the database how to scan rows, how to compare values, or how to optimize the search. You are only describing the result you want. The database engine handles the entire process internally. This is a powerful idea because it allows you to focus on the problem rather than the steps. It also allows the database system to optimize performance in ways that would be difficult to do manually.

It is **declarative**, which means you only describe *what* you want, not *how* to get it. You say: “Give me this data.”

The database decides:

- how to search
- how to filter
- how to optimize the process

This makes SQL powerful and efficient, especially when dealing with large amounts of data.

A good way to think about this is ordering food in a restaurant. You don’t go into the kitchen and explain how to cook the dish. You simply place your order. The kitchen handles the rest. SQL works in the same way.

## Examples of SQL in Action

To understand SQL better, it helps to see how naturally it expresses ideas. If you want to see all the data in a table, you write a query that essentially says, “show me everything from this table.” The structure of the language reflects this intention very clearly. If you want only certain columns, you specify exactly what you need. If you want to filter data based on a condition, you describe that condition directly.

For example, if you are working with student data and want to see only students above a certain age, your query expresses that requirement directly. There is no need to manually search through each record. The database does that for you.

Let’s look at how natural SQL feels once you understand it.

If you want to see all data in a table: `SELECT * FROM students;`

This means: “Show me everything from the students table.”

If you want specific columns: `SELECT name, age FROM students;`

This means: “Show me only the name and age.”

If you want to filter data: `SELECT name FROM students WHERE age > 18;`

This means: “Show me names of students older than 18.”

Notice how close this is to normal English. That is one of the reasons SQL is considered easy to read and write compared to many programming languages. What makes SQL interesting is how closely it mirrors human thinking. You are not writing instructions for a machine step by step. You are expressing a requirement in a structured form.

## Types of Operations in SQL

Although SQL feels simple on the surface, it is capable of handling many different types of operations. Sometimes you will use SQL to retrieve data. This is the most common use case, where you ask questions and get results. Other times, you will use it to modify data. You might add new records, update existing ones, or remove data that is no longer needed. You can also use SQL to define the structure of your database itself. This includes creating tables, modifying them, and setting rules for how data should be stored.

So, while SQL may look like a simple query language, it actually covers a wide range of actions, all centered around managing and working with data.

You don’t need to memorize everything right now, but it’s helpful to see the big picture. Some commands are used to **retrieve data**, like `SELECT`. Some are used to **modify data**, like:

- `INSERT` (add data)
- `UPDATE` (change data)
- `DELETE` (remove data)

Others are used to **define structure**, like:

- **CREATE TABLE**
- **ALTER TABLE**

As you move forward, you will explore each of these in detail.

## **SQL in the Real World**

SQL is not just something used in classrooms or small projects. It is deeply embedded in real-world systems.

When you log into a website, SQL is often used to check your credentials. When you browse products on an online store, SQL queries are used to fetch product details. When a company analyzes customer behaviour, SQL is used to retrieve and process the necessary data.

Even applications that seem very modern and complex still rely on SQL at their core when it comes to handling structured data. This is why SQL has remained relevant for decades. It solves a very fundamental problem: how to work with organized data efficiently.

Imagine you are working in a company that has thousands of customers. Your manager asks: “Can you give me a list of customers who placed orders in the last 7 days?”

Without SQL, this would be extremely difficult. You would have to manually search through records, match data, and compile results. With SQL, it becomes a single query. You ask the database the question, and it gives you the answer.

This is why SQL is used everywhere:

- in banking systems
- in e-commerce platforms
- in social media
- in analytics and reporting

Wherever there is structured data, SQL is usually there.

## **A Few Interesting Facts About SQL**

SQL has been around since the 1970s, yet it continues to be one of the most widely used technologies in the world. Few technologies last that long without becoming outdated.

Another interesting fact is that while there are many database systems, the core ideas of SQL remain largely the same across all of them. This means that once you learn SQL, you are not tied to a single tool. You can apply your knowledge in different environments with only small adjustments.

It is also worth noting that SQL is often used by people who are not traditional programmers. Analysts, data scientists, and even business professionals use SQL because of its clarity and practical value.

## // SQL vs Programming Languages

By now, you might be wondering something natural: “If SQL is a language... is it like Java or Python?”. The short answer is: **not really**.

SQL is a language, yes. But it behaves very differently from traditional programming languages. If you try to treat SQL like Java or Python, it will feel confusing. But once you understand its nature, it becomes surprisingly simple.

### **Different Purpose, Different Thinking**

Programming languages like Java or Python are designed to build complete applications. You use them to define logic, control flow, create functions, and manage how a program behaves step by step.

SQL, on the other hand, is not designed to build applications. It is designed for one main purpose:

### **Working with data**

You don't use SQL to create full software systems. You use it to interact with the data inside those systems.

Think of it like this:

- Java/Python → building the entire machine
- SQL → accessing and managing the machine's data

Both are important, but they play very different roles.

### **SQL is Not Step-by-Step Programming**

In programming languages, you often write instructions in sequence. For example, if you want to find students above 18 in Python, you would:

- loop through each student
- check their age
- store matching results

You are controlling the entire process step by step.

In SQL, you skip all of that. You simply say: “Give me students whose age is greater than 18.”. And the database handles everything internally.

This is why SQL feels simpler. You are not managing the process, you are defining the result.

## **No Loops (Mostly)**

One of the biggest differences students notice is this: SQL does not use loops in the way programming languages do.

There is no need to say:

- “for each row, do this”
- “repeat this process”

Why? Because SQL is built to work with entire datasets at once. When you write a query, you are not thinking about one row at a time. You are thinking about *all rows together*.

For example, when you ask for students above 18, SQL checks all rows internally. You don’t need to write a loop for it.

It’s like asking: “Show me all red cars in the parking lot.” You don’t go car by car. You let the system filter everything at once.

## **Set-Based Thinking (The Real Shift)**

This brings us to an important concept. Programming languages often use **step-by-step thinking**. SQL uses **set-based thinking**.

A “set” simply means a collection of data. Instead of thinking: “Take one row, process it, then move to the next”. You think: “Work with the entire group of rows together”

This is why SQL is so powerful when dealing with large amounts of data. It can handle thousands or even millions of rows in a single query without you writing complex logic.

## **Focus on Data, Not Logic**

In programming, a lot of your effort goes into:

- writing logic
- controlling flow
- handling conditions

In SQL, your focus shifts completely.

You think about:

- what data you need
- where it is stored
- how it is related

You are not building logic-heavy programs. You are asking meaningful questions. This is why SQL is widely used not only by developers, but also by analysts and business professionals.

## **A Simple Comparison**

To make it clearer, let's put it side by side in a simple way:

- In Python, you might write a loop to filter data
- In SQL, you write one query and get the result
- In Java, you define classes, methods, and control flow
- In SQL, you define what data you want
- Programming languages tell the computer *how to do something*
- SQL tells the database *what you want*

## **A Small Story**

Imagine you are working in a company and your manager asks: "Can you find all customers who made purchases this month?"

If you were using a programming language, you might:

- load all data
- loop through records
- apply conditions
- collect results

With SQL, you simply write one query. You ask the question directly, and the database gives you the answer. Less effort, more clarity.

## **Closing Thought**

SQL is not here to replace programming languages. It works alongside them. Programming languages build systems. SQL powers those systems with data.

Once you understand this difference, SQL stops feeling strange. Instead, it starts feeling... efficient. You are no longer telling the system what to do step by step. You are simply asking the right question and letting the system do the heavy lifting.

## // SQL Tools Setup: Getting Your Environment Ready

### SQL Databases in Practice

Before installing anything, it's important to understand what kind of systems we are dealing with. When we say "SQL database," we are not talking about a single tool. We are talking about a **family of database systems** that all follow the relational model and support SQL as their language.

These systems are used everywhere, but different ones are chosen based on needs like scale, complexity, performance, and environment.

Some of the most commonly used SQL databases in the industry today include:

- MySQL
- PostgreSQL
- SQLite
- Microsoft SQL Server
- Oracle Database
- MariaDB

Each of these follows SQL, but they differ in how they are built and where they are used. For example, a mobile app might use SQLite because it is lightweight and runs inside the app itself. A startup building a web app might use MySQL or PostgreSQL. A large bank might use Oracle or SQL Server because of their enterprise-level features.

So, while SQL remains the same conceptually, the **database system behind it can vary**. For this course, we will focus on **MySQL**, because it is simple, widely used, and perfect for beginners.

### MySQL

MySQL is a relational database management system (DBMS) that uses SQL to manage data. In simple terms, MySQL is the software that:

- stores your data
- organizes it into tables
- allows you to query it using SQL

If SQL is the language, MySQL is the system that understands and executes that language.

### When is MySQL Used?

MySQL is used whenever structured data needs to be stored and accessed efficiently. It is commonly used in:

- websites with login systems
- e-commerce platforms
- content management systems
- small to medium-scale applications



For example, when you log into a website, MySQL checks:

- your username
- your password
- your stored profile

All of this happens in milliseconds.

### **Where is MySQL Used in Real Life?**

MySQL powers a large part of the internet. It has been used in platforms like:

- social media systems
- blogging platforms
- online stores

Even if the front-end looks modern and flashy, behind the scenes, MySQL is often quietly managing the data.

### **How Does MySQL Work?**

Think of MySQL as a server running on your computer.

- It stores data in databases
- Each database contains tables
- Each table contains rows and columns

When you write a SQL query, it is sent to MySQL. MySQL processes it and returns the result.

So, the flow is: You write SQL → MySQL processes it → You get results

### **Installing MySQL (Step-by-Step)**

Step 1: Download MySQL

1. Go to the official website: <https://dev.mysql.com/downloads/installer/>
2. Download: MySQL Installer for Windows

Choose either:

- Web Installer (small, downloads components later)
- Full Installer (large, includes everything)

Step 2: Run the Installer

1. Open the downloaded file
2. You will see setup options

Choose: Developer Default

This installs:

- MySQL Server
- MySQL Workbench
- Required tools

Click Next

### Step 3: Install Dependencies

- If prompted, click Execute to install required components
- Wait for installation to complete

### Step 4: Configure MySQL Server

1. Choose: Standalone MySQL Server
2. Configuration Type: Select Development Machine
3. Keep default: Port: 3306

Click Next

### Step 5: Set Root Password

You will create a password for MySQL. This is very important. Save it somewhere.

Example: root123

### Step 6: Complete Installation

- Click Execute
- Wait for completion
- Click Finish

Now MySQL is installed and running.

## SQL Editors (Where You Write Queries)

### What is a SQL Editor?

After installing MySQL, you still need a place to *write SQL queries*. You don't interact with MySQL directly through raw commands. Instead, you use a **SQL editor**, which provides a clean interface.

A SQL editor allows you to:

- write queries
- run them
- view results in table format

### Why Do We Need SQL Editors?

Imagine writing SQL in a plain text file with no execution button, no output, no highlighting. It would be extremely inconvenient. SQL editors make things easier by:

- providing syntax highlighting
- showing results instantly
- managing connections
- organizing databases visually

They are your daily workspace.

## Common SQL Editors

Some widely used SQL editors include:

- DBeaver
- MySQL Workbench
- pgAdmin
- DataGrip

Each has its own style, but all serve the same purpose.

For this course, we will use **DBeaver**, because it is: simple, powerful, supports multiple databases

## Installing DBeaver (Step-by-Step):

Step 1: Download DBeaver

1. Go to: <https://dbeaver.io/download/>
2. Download: **Community Edition**

Step 2: Install

1. Open the installer
2. Click: Next → Next → Install → Finish

Step 3: Open DBeaver

- Launch DBeaver
- You will see a clean dashboard

Step 4: Connect to MySQL

1. Click **New Database Connection**
2. Select **MySQL**
3. Enter:
  - Host: localhost
  - Port: 3306
  - Username: root
  - Password: (your password)
4. Click **Test Connection** → then **Finish**

Step 5: Start Writing SQL

- Open SQL Editor
- Write your first query
- Run and see results instantly

## Final Mental Model:

At this point, everything is connected:

- MySQL → stores and manages data
- DBeaver → interface to interact
- SQL → language you use

## Online SQL Editors

Until now, we installed tools like MySQL and DBeaver to create a complete working environment. That is ideal for long-term learning. But sometimes, you just want to:

- try a query quickly
- practice a concept
- experiment without installing anything

This is where **online SQL editors** come in. Online SQL editors are web-based tools that allow you to write and run SQL queries directly in your browser. There is no installation, no setup, and no configuration required.

You open a website, type your query, and see the result instantly. It's like using a temporary lab where everything is already set up for you.

### Why Use Online SQL Editors?

Online editors are especially useful in the early stages of learning. They allow students to focus purely on SQL without worrying about:

- installation issues
- system configuration
- connection problems

They are also helpful when:

- you are using a different computer
- you want to quickly test a query
- you are sharing queries with others

For teaching, they are great for demonstrations because everything works immediately.

### How They Work

Most online SQL tools provide:

- a text area to write queries
- a sample or custom database
- an execution button
- a results panel

Some even allow you to:

- create tables
- insert data
- share your queries using links

However, keep in mind that these are **temporary environments**. Data is usually not saved permanently unless you export it.

## **Common Online SQL Editors**

### **DB Fiddle:**

DB Fiddle is one of the most useful online SQL tools. One of its best features is that you can generate a shareable link. This is very useful for teaching or discussing queries.

It allows you to:

- choose a database system (MySQL, PostgreSQL, etc.)
- write SQL queries
- create tables and insert data
- run everything in one place

For example, a teacher can create a problem and share the link with students, and everyone sees the same setup.

### **SQLFiddle:**

SQLFiddle is another popular online tool. It is slightly more structured compared to DB Fiddle, which can help beginners understand how databases are built step by step.

It allows you to:

- define your database structure
- insert data
- run queries

You typically:

1. Create schema (tables)
2. Insert data
3. Run queries

This makes it good for learning the flow of SQL.

## **Online vs Installed Tools**

At this point, it helps to understand when to use what.

Online editors are best for:

- quick practice
- learning concepts
- trying examples

Installed tools like MySQL + DBeaver are better for:

- long-term projects
- real-world practice
- managing larger datasets

Think of it like this:

- Online editor → practice notebook
- Installed setup → full workspace

Both are useful, just for different purposes.

### **A Small Learning Strategy**

A smart way to learn SQL is to combine both:

- Start with online tools to understand concepts
- Move to installed tools to build real systems

This way, you avoid being stuck in setup issues at the beginning, while still gaining practical experience later.

## // Connecting to Database & Your First Interaction with MySQL

You have installed MySQL. You have installed a SQL editor like DBeaver. But right now, they are just two separate tools sitting quietly. To actually work with data, you need to **connect them**.

Think of it like this:

- MySQL → a machine running in the background
- DBeaver → your control panel
- Connection → the bridge between them

Once connected, you can finally start talking to your database using SQL.

### Connecting to MySQL (Using DBeaver)

When you open DBeaver for the first time, it may look like an empty workspace. That's because it is waiting for you to connect to a database.

#### Step 1: Open Connection Setup

- Click on "New Database Connection"
- A list of database types will appear

Select: MySQL

This tells DBeaver which type of database you want to connect to.

#### Step 2: Enter Connection Details

Now you will see a form asking for connection details.

Fill in:

- Host → localhost  
(This means the database is running on your own computer)
- Port → 3306  
(Default MySQL port)
- Username → root
- Password → the one you set during installation

These details allow DBeaver to locate and access your MySQL server.

#### Step 3: Test the Connection

Click "Test Connection"

If everything is correct, you will see a success message.

If not, it usually means:

- wrong password
- MySQL service not running

Once successful, click Finish.

## What Just Happened?

You just connected your editor to your database.

Now DBeaver can:

- send SQL queries to MySQL
- receive results
- display them in a readable format

You are officially inside the system.

## Creating Your First Database

Now let's create something of your own. Until now, MySQL is empty. It has no data, no tables, nothing personal. A **database** is like a container that will hold your tables.

### Step 1: Open SQL Editor

- Right-click your MySQL connection
- Click SQL Editor → New SQL Script

You will now see a space where you can write SQL.

### Step 2: Create Database

Write: `CREATE DATABASE my_first_db;`

Run the query (▶ button).

What Does This Mean? You just told MySQL: "Create a new database named my\_first\_db", And MySQL created it for you.

### Step 3: Use the Database

Now you need to tell MySQL that you want to work inside this database.

`USE my_first_db;`

Run it.

Why This Step Matters? MySQL can contain multiple databases.

The USE command tells it: "From now on, work inside this database."

## Running Your First Query

Let's now run something very simple.

### Step 1: Basic Query

`SELECT 1;`

Run it.



## **What Happens Here?**

This query does not depend on any table. It simply tells MySQL: “Return the number 1”

And MySQL responds with: 1

It may look trivial, but this confirms something very important:

- ✓ Your connection works
- ✓ Your query is executed
- ✓ You are interacting with the database

## **Your First Real Feeling of SQL**

At this moment, something subtle but important has happened.

You: wrote a query → sent it to a database → received a result

This is the core loop of SQL.

Everything you learn next will build on this:

- more complex queries
- real tables
- meaningful data

## **Common Beginner Confusions**

It's normal to wonder:

- “Where is my database stored?”  
→ Inside MySQL server
- “Why do I need USE?”  
→ To select which database you are working on
- “Why did we run SELECT 1?”  
→ To test that everything works

## // Working with Data: Creating & Importing Tables

When learning SQL, one of the first practical questions students face is: “Do I always have to create everything from scratch?” The answer is: not always.

There are two main ways to start working with data:

- You create your own tables and data manually
- You import existing datasets (like CSV files)

In real-world scenarios, both approaches are used. If you are building a system, you design tables yourself. If you are analysing data, you often import existing datasets. For learning, it is best to understand both.

### Method 1: Creating Tables Manually

#### Understanding Manual Creation

Creating tables manually is like designing your own system.

You decide:

- what data to store
- how it should be structured
- what each column represents

This is important because it teaches you how databases are built.

#### Step 1: Create a Table

Make sure you are inside your database:

```
USE my_first_db;
```

Now create a table:

```
CREATE TABLE students (  
  id INT PRIMARY KEY,  
  name VARCHAR(50),  
  age INT,  
  marks INT  
);
```

What This Means? You just told MySQL:

- Create a table named students
- Add columns:
  - id → unique identifier
  - name → text
  - age → number
  - marks → number

This is your structure.

## Step 2: Insert Data

Now let's add data:

```
INSERT INTO students (id, name, age, marks) VALUES  
(1, 'John', 20, 85),  
(2, 'Sarah', 19, 90),  
(3, 'Mike', 21, 78);
```

## Step 3: View Data

```
SELECT * FROM students;
```

Now you will see your table filled with data.

## What You Just Did

You:

- created structure
- added data
- retrieved it

This is the complete basic cycle of working with databases.

## Method 2: Importing Data (CSV Files)

### Why Import Data?

In real life, you rarely type thousands of rows manually. Data usually comes from:

- Excel files
- CSV files
- exported datasets

Importing allows you to bring that data directly into your database.

### What is a CSV File?

CSV stands for **Comma-Separated Values**. It looks like this:

```
id,name,age,marks  
1,John,20,85  
2,Sarah,19,90  
3,Mike,21,78
```

It is basically a simple table stored as text.

## Step-by-Step: Import CSV in DBeaver

### Step 1: Create Empty Table

Before importing, you need a table with matching structure:

```
CREATE TABLE students_import (  
    id INT,  
    name VARCHAR(50),  
    age INT,  
    marks INT  
);
```

## Step 2: Import File

In DBeaver:

1. Right-click the table → Import Data
2. Choose:
  - CSV file
3. Select your file

## Step 3: Map Columns

DBeaver will show:

- CSV columns
- Table columns

Make sure they match correctly.

## Step 4: Execute Import

Click Next → Finish

Now your data is inside the table.

## Step 5: Verify

```
SELECT * FROM students_import;
```

## What Just Happened?

Instead of typing data manually, you imported it in seconds. This is how most real-world data enters databases

## Where to Get Practice Datasets

If you don't want to create data from scratch, there are many sources.

Common Sources:

- Kaggle → Huge collection of real-world datasets
- Google datasets (CSV downloads)
- Sample datasets (movies, sales, users, etc.)

Examples of Good Practice Data:

- E-commerce (customers, orders, products)
- Movies (films, ratings, actors)
- Students (marks, subjects)
- Sales data (monthly reports)

These datasets make SQL practice more interesting because they feel real.

## Manual vs Import: When to Use What

Manual Creation

Best for:

- learning concepts
- understanding table design

- small examples

### Importing Data

Best for:

- real-world practice
- large datasets
- analytics

### A Smart Learning Approach

Start like this:

1. Create small tables manually
2. Practice queries
3. Move to CSV datasets
4. Solve real-world problems

This builds both:

- understanding
- practical skill

### Closing Thought

At this stage, your database is no longer empty.

You now know how to:

- create tables
- insert data
- import datasets

This is where SQL starts becoming powerful. Because now, when you write queries... You are not working with imaginary data. You are working with something real.

## // Introduction to SQL Language Fundamentals (READ Operations)

Until now, you have learned:

- what data is
- how databases store it
- how tables are structured
- how MySQL works
- how to create databases and tables

But so far, everything has been preparation.

Now we begin the real heart of SQL: **working with data**

And the first thing humans naturally do with data is: **read it**

Before changing, deleting, or updating information, we first want to *see* and *understand* it. That is why almost every SQL journey begins with **READ operations**.

### What Are READ Operations?

READ operations are SQL commands used to:

- retrieve data
- view information
- filter records
- search inside tables

In SQL, these operations are mainly performed using one keyword: **SELECT**. This single keyword is one of the most important words in SQL. Whenever you want information from a database, you use **SELECT**.

### Why READ Operations Matter So Much

Imagine working in:

- a bank
- an online shopping company
- a hospital
- a school system

What do people mostly do all day? They search for information.

- “Show customer details”
- “Find students with highest marks”
- “List products below ₹500”
- “Show today’s transactions”

This is what SQL was built for. In fact, most real-world SQL usage is not writing huge complex systems. It is: asking questions to data That is why READ operations are the foundation of SQL.

## Understanding SQL Queries as Questions

One of the best ways to think about SQL is this: Every query is a question. The clearer your question, the better your answer.

For example: `SELECT * FROM students;`

This simply means: “Show me everything from the students table.”

Or: `SELECT name FROM students;`

Means: “Show me only the names.”

Or: `SELECT name FROM students WHERE age > 18;`

Means: “Show names of students older than 18.”

Notice something interesting? SQL feels surprisingly close to English. That is one reason it became so widely used.

## The READ Mindset

When beginners start SQL, they often think they need to “program.” But READ operations are less about programming and more about:

- observing
- searching
- filtering
- asking meaningful questions

You are not building algorithms here. You are exploring information. It’s almost like being a detective inside a data world.

## Using Existing Databases for Practice

One beautiful thing about SQL is that you do not need to create massive systems from scratch to practice. Most database systems, including MySQL, provide:

- sample databases
- practice datasets
- existing tables

These are extremely useful because they simulate real-world systems. Instead of imaginary student tables forever, you can practice on:

- employees
- products
- orders
- movies
- customers

This makes learning much more realistic and interesting.

## The Famous MySQL Sample Database

One of the most commonly used practice databases in MySQL is the: **Employees Database**. It contains realistic company data like:

- employees
- departments
- salaries
- managers

This is widely used for:

- SQL learning
- interview practice
- analytics exercises

### What Real Databases Feel Like

When students first see a real database, they often expect something magical. But in reality, it is just:

- multiple related tables
- filled with meaningful data

For example:

Employees Table:

|        |      |            |
|--------|------|------------|
| emp_id | name | department |
|--------|------|------------|

Salaries Table:

|        |        |
|--------|--------|
| emp_id | salary |
|--------|--------|

Departments Table:

|         |           |
|---------|-----------|
| dept_id | dept_name |
|---------|-----------|

And suddenly SQL becomes exciting because now you can ask real questions like:

- Which department has highest average salary?
- Which employee earns the most?
- How many employees work in each department?

This is where SQL starts feeling powerful.



## Why We Start with READ First

Before modifying data, you must first understand how to access it safely. Imagine giving a beginner delete access before they even know how to view tables properly. Disaster would arrive very quickly. READ operations are safe and exploratory.

They help students:

- understand tables
- understand structure
- understand relationships
- build confidence

That is why SQL learning almost always starts with: SELECT

## Closing Thought

SQL READ operations are not just about “getting data.”. They are about:

- discovering patterns
- understanding systems
- turning raw records into useful insight

You are no longer just storing information. You are learning how to *explore it*.

## // **SELECT** statement

Now that you understand what READ operations are, it is time to learn the most important command in SQL: **SELECT**. If SQL were a city, **SELECT** would probably be its main road. Almost everything in SQL begins here. Whenever you want to:

- see data
- search records
- analyse information
- explore tables

you use **SELECT**.

It is the command that allows you to retrieve information from a database.

### **Understanding SELECT in a Simple Way**

Imagine a huge school office with thousands of student records stored inside cabinets. You walk in and ask: “Show me all students.” That request is essentially a SQL **SELECT** query.

Or imagine an online shopping company with millions of products. Instead of manually checking records one by one, SQL allows you to instantly ask:

- “Show me all products”
- “Show me product names only”
- “Show me products cheaper than ₹500”

This ability to quickly retrieve information is why SQL became so powerful.

### **How SELECT Works**

The basic idea of **SELECT** is very simple: You tell SQL:

- what information you want
- and where to get it from

The general syntax looks like this:

```
SELECT column_name  
FROM table_name;
```

This can almost be read like English: “Select this column from this table.”

### **SELECT \*: Viewing Everything**

One of the first queries beginners learn is: **SELECT \* FROM students**;

Here:

- **SELECT** → tells SQL you want data
- **\*** → means “everything”
- **FROM students** → tells SQL which table to look at

So, this query means: “Show me all columns and all rows from the students table.”

## What Does \* Actually Mean?

The \* symbol acts like a shortcut for: “Give me everything available.”

Imagine a table like this:

| id | name  | age | marks |
|----|-------|-----|-------|
| 1  | John  | 20  | 85    |
| 2  | Sarah | 19  | 90    |

When you write: `SELECT * FROM students;`

SQL returns the full table.

This is very useful while:

- exploring a new table
- checking data quickly
- debugging queries

## A Small Real-World Fact

Beginners love using `SELECT *`, and that’s completely fine while learning. But in large real-world systems, professionals avoid using it unnecessarily. Why? Because some tables may contain:

- dozens of columns
- thousands or millions of rows

Fetching everything wastes:

- memory
- processing time
- network resources

So, in professional environments, developers usually request only the columns they actually need.

## Selecting Specific Columns

Very often, you do not need the entire table. Imagine a teacher who only wants student names and marks, not age or IDs. Instead of asking for everything, SQL allows you to select specific columns.

Example: `SELECT name, marks FROM students;`

This means: “Show me only the name and marks columns.”

Result:

| name  | marks |
|-------|-------|
| John  | 85    |
| Sarah | 90    |

## Why This Matters

This may seem like a small thing, but it is actually very important.

Good SQL practice is about:

- retrieving only necessary data
- keeping queries efficient
- making results cleaner and easier to read

Imagine opening a restaurant menu and only seeing vegetarian dishes because that's what you asked for. Much cleaner than receiving the entire menu every time. Selecting specific columns works the same way.

## Combining Multiple Columns

You can request multiple columns together by separating them with commas.

Example: `SELECT id, name, age FROM students;`

This returns only:

- id
- name
- age

while ignoring all other columns. SQL allows you to shape the output according to your needs.

## Real-World Example

Imagine an e-commerce company. The products table may contain:

- product\_id
- product\_name
- price
- stock
- supplier\_details
- warehouse\_location
- manufacturing\_info

Now imagine a customer browsing products online. Do they need warehouse details? No.

The website may only request: `SELECT product_name, price FROM products;`

This keeps things:

- fast
- clean
- efficient

This is how real applications use SQL every second behind the scenes.

## **SELECT is About Asking Better Questions**

One interesting thing about SQL is that beginners often focus too much on syntax. But experienced SQL users focus on: asking better questions

For example:

- “What information do I actually need?”
- “Which table contains it?”
- “Do I need everything or just specific columns?”

The clearer the question, the cleaner the query.

## **Common Beginner Mistakes**

Some common things beginners do:

- Forgetting commas  
`SELECT name age FROM students;`  
SQL gets confused because it cannot separate columns properly.  
Correct version: `SELECT name, age FROM students;`
- Forgetting table name  
`SELECT *;`  
SQL does not know where to get data from.  
You must specify: `FROM students`

## **What You Have Learned So Far**

At this point, you now know how to:

- retrieve all data using `SELECT *`
- retrieve specific columns
- control what information appears in results

This may feel basic, but these ideas form the foundation of almost every SQL query you will ever write.

## **Closing Thought**

The SELECT statement is simple, but incredibly powerful. With just this one command, you can explore massive amounts of data, answer important questions, and uncover patterns hidden inside tables.

Right now, you are only selecting columns. Soon, you’ll start filtering, sorting, grouping, and combining data. And that’s when SQL starts feeling less like a language and more like a tool for investigation.

## // The WHERE Clause

So far, you have learned how to retrieve data using the SELECT statement. But there is a problem. Imagine a table containing 50,000 students. If you write: `SELECT * FROM students;`

SQL will return everything. Every student. Every row. Every column. That might sound useful at first, but in real life, we rarely want *a//*the data. Most of the time, we are searching for something specific.

- Students older than 18
- Customers from Mumbai
- Products cheaper than ₹500
- Employees with salary above ₹50,000

This is where the WHERE clause becomes powerful.

### What is the WHERE Clause?

The WHERE clause is used to **filter data**. It allows you to tell SQL: “Don’t show me everything. Show me only the rows that match a condition.”

Think of it like searching inside a large crowd. Imagine a school assembly with thousands of students. If a teacher says: “Only students from Class A should stay.” that is basically a real-world WHERE condition. Everyone else is filtered out.

### Basic Syntax

The general structure looks like this:

```
SELECT column_name
FROM table_name
WHERE condition;
```

This can almost be read like English: “Select this data from this table where this condition is true.”

### Using Simple Conditions

Let’s imagine this table:

| id | name  | age | marks |
|----|-------|-----|-------|
| 1  | John  | 20  | 85    |
| 2  | Sarah | 19  | 90    |
| 3  | Mike  | 17  | 70    |

### **Equal To (=):**

Suppose you want only the student named John.

```
SELECT * FROM students  
WHERE name = 'John';
```

This means: “Show rows where the name is John.” SQL checks every row and returns only matching records.

### **Greater Than (>)**

Now suppose you want students older than 18.

```
SELECT * FROM students  
WHERE age > 18;
```

Result:

- John
- Sarah

Mike is excluded because his age is 17.

### **Less Than (<)**

To find students younger than 18:

```
SELECT * FROM students  
WHERE age < 18;
```

Result:

- Mike

### **Greater Than or Equal (>=)**

Suppose the teacher says: “Students with marks 85 or above are eligible.”

```
SELECT * FROM students  
WHERE marks >= 85;
```

This includes:

- 85
- 90
- anything above

### **Less Than or Equal (<=)**

```
SELECT * FROM students  
WHERE age <= 19;
```

This returns student aged:

- 19
- below 19

## Real-World Importance of Filtering

The WHERE clause is one of the most heavily used parts of SQL in the real world. Think about apps you use daily.

When you search:

- products below a price
- movies above a rating
- flights under a budget

you are basically using filtering conditions.

Even login systems use conditions internally: `WHERE username = 'user1'`

The database checks whether matching data exists.

## Combining Conditions

Sometimes one condition is not enough. Imagine a company wants: “Employees older than 25 AND working in Sales.” Or: “Students with marks above 80 OR attendance above 90.”

This is where logical operators come in.

### AND Operator

AND means: both conditions must be true.

Example:

```
SELECT * FROM students
WHERE age > 18 AND marks > 80;
```

This returns students who satisfy:

- age above 18
- marks above 80

Both conditions are checked together.

Real-Life Analogy: Imagine entering a club where - age must be above 21 & membership card is required. Both conditions must be true. That’s exactly how AND works.

### OR Operator

OR means: at least one condition must be true.

Example:

```
SELECT * FROM students
WHERE age < 18 OR marks > 85;
```

This returns:

- students younger than 18
- students with high marks
- or both

Real-Life Analogy: Imagine a scholarship rule: “Students with excellent marks OR sports achievements can apply.” A student only needs one qualifying condition.



## NOT Operator

NOT reverses a condition.

Example:

```
SELECT * FROM students
WHERE NOT age > 18;
```

This means: “Show students who are NOT older than 18.”

Result:

- students aged 18 or below

## How SQL Thinks During WHERE

One important thing to understand is this: SQL checks the condition for every row.

If the condition is:

✓ true → row is included

✗ false → row is ignored

It's almost like a security guard checking entries one by one.

## Common Beginner Mistakes

- Using == Instead of =  
In programming languages: == is common.  
But in SQL: = is used for equality.
- Forgetting Quotes Around Text  
Wrong: `WHERE name = John`  
Correct: `WHERE name = 'John'`  
Text values need quotes.
- Confusing AND and OR  
This is extremely common.
  - AND → both conditions required
  - OR → any one condition required
 Small difference, huge impact.

## Why WHERE is So Powerful\*\*

Without `WHERE`, SQL would just dump entire tables at you. With `WHERE`, SQL becomes intelligent. You stop looking at *all data* and start finding:

- relevant data
- useful data
- meaningful data

This is the beginning of real querying.

## What You Learned:

At this point, you now know how to:

- filter rows using **WHERE**
- use comparison operators:
  - **=**
  - **>**
  - **<**
  - **>=**
  - **<=**
- combine conditions using:
  - **AND**
  - **OR**
  - **NOT**

These may look small individually, but together they form the foundation of almost every real SQL query.

## Closing Thought

The **SELECT** statement taught you how to ask for data.

The **WHERE** clause teaches you how to ask *better questions*.

And in SQL, better questions lead to better answers.

## // Sorting Data with ORDER BY & LIMIT

So far, you have learned how to:

- retrieve data using SELECT
- filter data using WHERE

But imagine searching through a huge table and getting results in a completely random order.

Suppose an online shopping app shows:

- expensive products mixed with cheap ones
- newest products hidden somewhere in the middle
- top-rated items scattered randomly

It would feel messy and difficult to use. This is why databases allow us to **sort data**. And sometimes, we do not even want all results. We may only want:

- top 5 students
- cheapest 10 products
- latest 3 orders

This is where ORDER BY and LIMIT become extremely useful.

### Sorting Data with ORDER BY

The ORDER BY clause is used to arrange data in a specific order. Without it, SQL usually returns rows in whatever order the database finds convenient. That order may not always make sense to humans. ORDER BY allows us to organize results in a cleaner and more meaningful way.

#### Basic Idea

Imagine this students table:

| id | name  | marks |
|----|-------|-------|
| 1  | John  | 85    |
| 2  | Sarah | 95    |
| 3  | Mike  | 70    |

Now suppose a teacher wants to see students arranged by marks. Instead of manually checking each row, SQL can sort them automatically.

#### Basic Syntax

```
SELECT * FROM students
ORDER BY marks;
```

This tells SQL: “Show all students sorted by marks.” By default, SQL sorts in **ascending order**.

## Ascending Order (ASC)

Ascending means:

- smaller → larger
- A → Z
- low → high

For numbers: 70 → 85 → 95

For text: A → B → C

### Using ASC Explicitly

```
SELECT * FROM students
ORDER BY marks ASC;
```

This means: “Sort marks from lowest to highest.”

Result:

| name  | marks |
|-------|-------|
| Mike  | 70    |
| John  | 85    |
| Sarah | 95    |

## Descending Order (DESC)

Descending means:

- larger → smaller
- Z → A
- high → low

### Using DESC

```
SELECT * FROM students
ORDER BY marks DESC;
```

This means: “Sort marks from highest to lowest.”

Result:

| name  | marks |
|-------|-------|
| Sarah | 95    |
| John  | 85    |
| Mike  | 70    |

## Real-World Examples of Sorting

Sorting is everywhere in modern applications.

When you:

- sort products by price
- sort movies by rating
- sort videos by views
- sort flights by cost

you are indirectly using ORDER BY.

Even social media feeds use sorting internally:

- latest posts first
- most popular content first

Databases constantly sort information behind the scenes.

## Sorting Text Data

ORDER BY also works with text columns.

Example:

```
SELECT * FROM students  
ORDER BY name ASC;
```

This sorts names alphabetically.

Result:

```
John  
Mike  
Sarah
```

## Sorting Multiple Columns

Sometimes one level of sorting is not enough. Imagine a class where multiple students have the same marks.

You may want:

- first sort by marks
- then sort by names alphabetically

Example:

```
SELECT * FROM students  
ORDER BY marks DESC, name ASC;
```

SQL first sorts by marks. If marks are equal, it sorts names. This is common in ranking systems and reports.

## Limiting Results with LIMIT

Now let's imagine something practical. Suppose a products table contains: 1 million rows And you run: `SELECT * FROM products;`

Your screen would explode into a wall of data. Most of the time, we only want a small portion. This is where LIMIT helps.

### What LIMIT Does

LIMIT restricts the number of rows returned. It tells SQL: "Only show this many results."

### Basic Syntax

```
SELECT * FROM students  
LIMIT 3;
```

This means: "Show only the first 3 rows."

### Real-World Importance of LIMIT

LIMIT is extremely common in modern applications.

Imagine:

- top 10 products
- latest 5 notifications
- first 20 search results

Websites almost never load everything at once. Why? Because:

- huge datasets are slow
- users only need small portions initially
- loading everything wastes resources

This is why search engines, social media apps, and shopping websites constantly use limits.

## Combining ORDER BY + LIMIT

This is where things become powerful. Imagine a teacher wants: "Top 3 students by marks."

SQL can do this beautifully:

```
SELECT * FROM students  
ORDER BY marks DESC  
LIMIT 3;
```

What happens here?

1. SQL sorts students by highest marks
2. Then it returns only first 3 rows

This combination is heavily used in analytics and rankings.

## A Small Mental Model

Think of SQL operations like this:

- SELECT → choose data
- WHERE → filter data
- ORDER BY → organize data
- LIMIT → reduce data

Together, they form the core workflow of querying.

## Common Beginner Mistakes

- Writing LIMIT Before ORDER BY

Wrong:

```
LIMIT 5
ORDER BY marks DESC
```

Correct order:

```
ORDER BY marks DESC
LIMIT 5
```

SQL follows a specific structure.

- Confusing ASC and DESC
  - ASC → ascending
  - DESC → descending

A simple memory trick: DESC = “descending down”

## What You Learned?

At this point, you now know how to:

- sort data using ORDER BY
- use:
  - ASC
  - DESC
- limit rows using LIMIT
- combine sorting and limiting together

This may look simple, but these features are used constantly in real applications.

## Closing Thought:

Without sorting, data feels chaotic. Without limiting, data feels overwhelming. ORDER BY brings structure. LIMIT brings focus. And together, they help transform raw tables into useful, readable information.

## // Aliases & DISTINCT

As you continue learning SQL, you'll notice something interesting. Sometimes the problem is not retrieving data.

The problem is making the result:

- cleaner
- easier to read
- more meaningful

Imagine showing query results to:

- a manager
- a teacher
- a customer

Raw database column names are often messy or too technical. And sometimes tables contain repeated values that make the output unnecessarily cluttered.

This is where two very useful SQL features help:

- **Aliases (AS)** → for renaming output temporarily
- **DISTINCT** → for removing duplicate values

These may seem small, but they make SQL results far more readable and professional.

### Aliases: Giving Temporary Names

Imagine a table with a column named: `customer_total_purchase_amount`. Technically correct? Yes. Pleasant to read? Absolutely not.

Now imagine showing this to someone in a report. This is where aliases become useful. An alias allows you to give a temporary name to:

- columns
- tables
- calculated results

without actually changing the original database structure.

### Understanding the AS Keyword

The AS keyword is used to rename something temporarily inside a query.

Basic syntax:

```
SELECT column_name AS alias_name  
FROM table_name;
```

This means: "Show this column using another name."



## Simple Example

Suppose we have this table:

| id | name  | marks |
|----|-------|-------|
| 1  | John  | 85    |
| 2  | Sarah | 90    |

Now imagine you want the output to show:

- “Student Name” instead of name
- “Score” instead of marks

Query:

```
SELECT name AS 'Student Name',
       marks AS Score
FROM students;
```

Result:

| Student Name | Score |
|--------------|-------|
| John         | 85    |
| Sarah        | 90    |

Notice something important: The original table is unchanged. The alias only affects the query output.

## Why Aliases Matter in Real Life

Aliases are extremely common in:

- reports
- dashboards
- analytics
- business queries

Real databases often contain technical column names like:

- cust\_id
- prod\_qty
- txn\_amt

But managers or clients may prefer:

- Customer ID
- Product Quantity
- Transaction Amount

Aliases help make results human-friendly.

## Aliases Without AS

Interestingly, SQL also allows aliases without writing AS.

Example:

```
SELECT name Student_Name
FROM students;
```

This still works. But using AS is considered cleaner and easier to understand, especially for beginners.

## Aliases are Like Nicknames

Think of aliases like nicknames. Your real name remains the same, but people may call you something shorter or easier in conversation.

Similarly:

- database column remains unchanged
- query displays a temporary name

## Using Aliases with Calculations

Aliases become even more useful when working with calculated values.

Imagine this query:

```
SELECT marks + 5 AS updated_marks
FROM students;
```

Without an alias, SQL might display a strange automatic column name. Aliases help keep outputs neat and understandable.

## DISTINCT: Removing Duplicates

Now let's talk about another common problem. Imagine a students table like this:

| name  | city   |
|-------|--------|
| John  | Mumbai |
| Sarah | Delhi  |
| Mike  | Mumbai |
| Anna  | Pune   |

Now suppose you ask: `SELECT city FROM students;`

Result:

```
Mumbai
Delhi
Mumbai
Pune
```

Notice how Mumbai appears twice. Sometimes we want unique values only. This is where DISTINCT becomes useful.

## What DISTINCT Does

DISTINCT removes duplicate values from the result.

It tells SQL: “Show only unique entries.”

## Basic Syntax

```
SELECT DISTINCT column_name  
FROM table_name;
```

Example:

```
SELECT DISTINCT city  
FROM students;
```

Result:

```
Mumbai  
Delhi  
Pune
```

Now duplicate values are removed.

## Real-World Importance of DISTINCT

Imagine an online store wanting to know:

- which cities customers belong to
- which product categories exist
- which payment methods are used

They don't want repeated values again and again. DISTINCT helps summarize data cleanly.

## DISTINCT Works on Entire Selection

This is important. Suppose you write:

```
SELECT DISTINCT city, name  
FROM students;
```

SQL checks the *combination* of:

- city
- name

not each column individually.

So, duplicates are removed only when the entire selected row matches.

## Common Beginner Confusions

- **Thinking DISTINCT Changes the Table**

It does not. Like most **SELECT** operations, **DISTINCT** only changes the output. The original data remains untouched.

- **Overusing DISTINCT**

Beginners sometimes use **DISTINCT** everywhere to “clean” data. But duplicates are not always bad. Sometimes repeated values are meaningful. **DISTINCT** should be used only when uniqueness is actually needed.

## A Small Real-World Story

Imagine a music app with millions of songs. If the company wants to know:

- all unique artists
- all available genres

they would use **DISTINCT**.

Without it, the result would contain repeated artist names thousands of times. **DISTINCT** helps transform noisy data into cleaner insights.

## What You Learned?

At this point, you now know how to:

- rename columns using aliases
- use the **AS** keyword
- improve readability of results
- remove duplicate values using **DISTINCT**

These features may seem small, but they are used constantly in real-world SQL work.

## Closing Thought

SQL is not only about retrieving data. It is also about presenting data clearly. Aliases help make results understandable. **DISTINCT** helps make results cleaner. And together, they move SQL one step closer from “raw database output” to meaningful information people can actually use.

# SQL CRUD OPERATIONS

Up until now, you have learned how data is stored, how databases are structured, and how SQL helps us retrieve information. You explored tables, rows, columns, filtering, sorting, and querying. But all of that was mostly focused on *viewing* data.

Now we enter the part where databases truly start feeling alive. Real-world databases are never static. Information is constantly being added, viewed, modified, and removed. Every modern application you use today survives because data is continuously moving and changing behind the scenes.

A student joins a school portal. A customer places an order. A user edits their profile picture. An old notification gets removed. All these actions are part of one fundamental cycle in database systems called:

## CRUD Operations

CRUD is one of the most important concepts in database management and software development. Almost every system built today revolves around these four actions.

### Understanding CRUD Through Real Life

Imagine running a library.

Every day:

- new books arrive
- readers search for books
- book information gets updated
- damaged books are removed

Without realizing it, the library performs CRUD operations all day long. Or think about your smartphone apps.

When you install Instagram and create an account, your information is added to a database. When you open the app and scroll through posts, data is retrieved from the database. When you edit your bio, information is updated. When you delete a message or a post, data is removed.

CRUD is happening everywhere around us, quietly and constantly. Once you understand CRUD, you begin to realize something interesting: Most digital systems are really just advanced ways of managing data.

## **Why CRUD is So Important**

Databases exist to manage information. But information becomes useless if it cannot change.

Imagine a hospital system where:

- new patients cannot be added
- reports cannot be viewed
- addresses cannot be updated
- old records cannot be removed

The system would become outdated almost immediately. CRUD solves this problem by providing a complete lifecycle for data.

It allows systems to:

- grow with new information
- provide access to stored information
- keep records accurate and updated
- remove unnecessary or outdated data

Without CRUD, databases would simply become giant storage rooms with no practical usability.

## // The Four CRUD Operations

These four operations may look simple on paper, but together they power nearly every application and website in existence.

### **CREATE**

CREATE operations are responsible for adding new information into the system. This is the moment data enters the database for the first time.

Examples include:

- creating a user account
- adding a product to an online store
- inserting marks into a student system
- registering a patient in a hospital database

Without CREATE operations, databases would never grow.

### **Real-World Story**

Imagine joining a new social media platform.

The moment you sign up:

- your username
- email
- password
- profile details

are stored inside the database.

The system creates a new record specifically for you. That is a CREATE operation. Even something as simple as posting a comment online creates new data inside a database.

### **SQL and CREATE**

In SQL, CREATE operations are usually performed using commands like:

- CREATE DATABASE
- CREATE TABLE
- INSERT

Some commands create structure, while others add actual records. You will explore all of these step by step.

## **READ**

READ operations retrieve information from the database. This is the most commonly used database operation in real-world systems because people constantly need to *view* information.

Whenever you:

- search products online
- view notifications
- check account balance
- scroll through posts

the system is performing READ operations.

### **Real-World Story**

Imagine opening a food delivery app. You search for: “Pizza near me” The app immediately shows restaurants, prices, ratings, and delivery times.

Where did all this information come from? The database. The app asked the database for matching records and displayed the result to you. That is READ.

### **SQL and READ**

READ operations mainly use: SELECT. This is the command you have already started learning. But as you move forward, you will discover that READ operations can become extremely powerful. SQL can:

- filter data
- sort information
- combine tables
- calculate statistics
- generate reports

All through queries.

## **UPDATE**

Data in the real world changes constantly. People move to new cities. Prices change. Passwords get reset. Products go out of stock.

If databases could not modify existing information, systems would quickly become inaccurate. This is why UPDATE operations exist. UPDATE allows us to change existing records without deleting and recreating them.

### **Real-World Story**

Imagine a customer changes their phone number. Instead of deleting the customer and creating a new account, the system simply updates the phone number field. Or imagine an online store increasing product prices during a sale season. Again, existing records are updated rather than recreated.



## **Why UPDATE Needs Care**

One wrong UPDATE query can affect thousands of rows instantly. Imagine accidentally updating:

- all employee salaries
- all customer addresses
- all product prices

This is why professionals are always careful while writing UPDATE queries. Most developers first test queries using READ operations before performing updates.

## **DELETE**

DELETE operations remove data from the database.

Sometimes information is no longer needed:

- deleting old notifications
- removing canceled accounts
- clearing temporary records

DELETE helps keep systems organized and relevant.

## **Real-World Story**

Suppose a user permanently deletes their social media account.

The platform may remove:

- profile information
- posts
- messages
- settings

from the database. That is a DELETE operation.

## **The Dangerous Side of DELETE**

DELETE is powerful and sometimes risky. A single incorrect DELETE query can wipe out huge amounts of important data. Because of this, large companies often avoid permanent deletion immediately. Instead, they:

- archive records
- mark them inactive
- move them to backup storage

This allows recovery if mistakes happen.

## CRUD is Everywhere

Once you understand CRUD, you begin seeing it everywhere.

### Netflix

- Create watchlists
- Read recommendations
- Update preferences
- Delete history

### Amazon

- Add products
- View orders
- Update cart quantities
- Remove wishlist items

### Banking Systems

- Create accounts
- Read balances
- Update transactions
- Delete temporary records

CRUD is the heartbeat of modern digital systems.

## SQL Makes CRUD Possible

SQL provides commands specifically designed for CRUD operations.

| CRUD Operation | SQL Command |
|----------------|-------------|
| Create         | INSERT      |
| Read           | SELECT      |
| Update         | UPDATE      |
| Delete         | DELETE      |

Each command serves a different purpose, but together they allow complete control over data.

## The Big Shift Happening Now

Until this point:

- databases felt structural
- tables felt static
- rows felt like examples

Now things become dynamic.

You are moving from: understanding databases to controlling databases

This is where SQL starts feeling practical and powerful.

### **Closing Thought**

CRUD operations may seem simple at first glance.

Just four actions:

- Create
- Read
- Update
- Delete

But together, they form the foundation of almost every application, website, and digital system in existence. Every app you open today is constantly performing CRUD operations behind the scenes. And now, you are learning how to do the same yourself.

# CREATE Operations

Until now, most of your interaction with SQL has been about *reading* data. You learned how to:

- retrieve rows
- filter information
- sort results
- ask questions to databases

But now we move into a completely different stage of SQL. Until this point, you were exploring an already existing world. Now, you are going to **build that world yourself**.

This chapter is about creating structure. Because before data can be queried, filtered, analyzed, or visualized... ..it first needs a place to live.

## What Are CREATE Operations?

CREATE operations are used to build the foundation of a database system.

They allow us to create:

- databases
- tables
- columns
- relationships
- rules and constraints

Think of it like constructing a city. Before people can live there, you need:

- land
- buildings
- roads
- organization

Similarly, before data can exist, we need structure. That structure is created using SQL CREATE operations.

## Why CREATE Operations Matter

Imagine trying to run:

- a hospital
- a bank
- an online shopping platform

without proper organization.

If customer data, product information, transactions, and user details were all mixed randomly, the system would collapse into chaos very quickly.

CREATE operations solve this by defining:

- what kind of data can exist

- where it should be stored
- how it should be organized

Without proper structure:

- queries become confusing
- updates become risky
- analytics becomes unreliable

Good databases are not accidental. They are designed carefully.

## **CREATE is More Than “Making Tables”**

Beginners often think: “CREATE just means making a table.” But in reality, CREATE operations are about: designing information systems

When you create a table, you are making decisions like:

- What data is important?
- What should each column store?
- Which values must be unique?
- How are tables connected?
- What data types should be used?

These decisions affect:

- performance
- readability
- scalability
- future analytics

This is why database design is considered both:

- a technical skill
- and a planning skill

## **Thinking Like a Data Analyst**

This is where an important mindset shift happens. As a data analyst, you are not just storing random information. You are designing systems that will later answer questions.

For example, imagine an e-commerce company. A poorly designed table may make it difficult to answer:

- Which products sell best?
- Which customers are inactive?
- Which city generates highest revenue?

A well-designed structure makes analytics smooth and efficient. This is why analysts and database designers think ahead before creating tables.

## **Questions to Ask Before Creating Tables**

Before creating any table, professionals usually think about questions like:

What information needs to be stored?  
Students? Products? Transactions? Employees?

What columns are required?  
Names? IDs? Dates? Prices?

What type of data will each column hold?

- numbers
- text
- dates
- boolean values

Which field uniquely identifies records?  
Usually this becomes the primary key.

Which information changes frequently?  
This affects design decisions.

Will this data be useful for reports and analytics later?  
Very important for analysts.

### **Understanding Databases vs Tables Again**

This distinction becomes very important during CREATE operations.

**Database:** A database is like a container or workspace.

Example: school\_db

Inside this database, you may have:

- students table
- teachers table
- marks table

**Table:** A table stores actual structured data.

Example: students

Inside the table:

- rows = records
- columns = fields

### **Where We Create Things (MySQL + DBeaver)**

At this stage, students often get confused between:

- MySQL
- DBeaver
- SQL

So, let's make this crystal clear.

## MySQL

MySQL is the actual database system.

It:

- stores data
- manages databases
- executes SQL queries

Think of MySQL as: the engine

## DBeaver

DBeaver is the interface where you:

- write SQL
- run queries
- view results visually

Think of DBeaver as: the control panel

## SQL

SQL is the language you use to communicate with MySQL. So the workflow becomes:



## Creating Your First Database (Conceptually)

When you create a database, you are basically creating a dedicated space for a project or system.

For example:

**School System:** school\_db

**E-commerce System:** ecommerce\_db

**Movie Platform:** movies\_db

Each database contains its own related tables.

## Creating Tables: Designing Structure

Tables are where actual data lives. This is where you define:

- columns
- data types
- keys
- constraints

Example student table:

|    |      |     |       |
|----|------|-----|-------|
| id | name | age | marks |
|----|------|-----|-------|

This looks simple, but every column is a design decision.

### Thinking About Data Types

When creating tables, one major responsibility is choosing correct data types.

For example:

- age → number
- name → text
- joining\_date → date

Choosing poor data types can:

- waste storage
- slow queries
- create inconsistencies

This becomes very important in large systems.

### Real-World Story

Imagine a food delivery company designing its database. At first, they create simple tables quickly without much planning.

Later they realize:

- customer addresses are inconsistent
- city names are duplicated
- delivery timings are hard to analyze
- analytics reports are unreliable

Eventually, they redesign the database structure entirely. This happens often in real companies. Good CREATE operations save future headaches.

### CREATE Operations Build the Foundation

Everything in SQL depends on structure. Bad structure causes:

- messy data
- difficult queries
- poor analytics
- slow performance

Good structure creates:

- clean systems
- efficient queries
- reliable insights
- scalable applications

This is why database creation is such an important phase.



## **What You Will Learn Next**

In the upcoming topics, you will learn how to:

- create databases
- create tables
- define columns
- use data types
- apply constraints
- design relationships

You will move from: using database to building databases

## **Closing Thought**

CREATE operations are the architectural stage of databases. This is where ideas become structure.

Every app, every website, every platform storing information began with someone carefully deciding:

- what data matters
- how it should be organized
- and how future systems will interact with it

And now, you are beginning to learn how to make those decisions yourself.

## // Creating Databases & Tables

Now we finally begin building things ourselves. Until this point, databases may have felt like ready-made systems that already existed somewhere. But in real-world development and analytics, professionals constantly create new databases and tables for:

- projects
- applications
- analytics systems
- dashboards
- company software

This is where SQL starts feeling creative. You are no longer just asking questions to data. You are designing the place where data will live.

### Creating a Database

A database acts like a container or workspace. Think of it like creating a new folder for a project on your computer.

For example:

- a school system may have `school_db`
- an online store may have `ecommerce_db`
- a hospital may have `hospital_db`

Inside these databases, tables and records are stored.

### Basic Syntax for Creating Database

The syntax is very simple: `CREATE DATABASE database_name;`

This can almost be read like English: “Create a database with this name.”

#### Example 1: Creating Student Database

```
CREATE DATABASE school_db;
```

This creates a database named: `school_db`

#### Example 2: E-commerce Database

```
CREATE DATABASE ecommerce_db;
```

#### Example 3: Hospital Database

```
CREATE DATABASE hospital_db;
```

## What Happens Internally?

When MySQL executes this command:

- it creates storage space
- prepares system files
- registers the database inside MySQL

The database is now ready to hold tables and data.

## Using the Database

After creating a database, you must tell MySQL: “I want to work inside this database.” This is done using: `USE school_db;`

Without `USE`, MySQL may not know where to create tables.

## Common Beginner Mistake

Many beginners create a database and immediately try creating tables without selecting it first.

Example mistake: `CREATE TABLE students (...);`

without: `USE school_db;`

This often causes confusion or errors.

## Creating Tables

Now comes the most important part. Databases are containers. Tables are where actual data lives.

A table defines:

- what information will be stored
- what columns exist
- what type of data each column holds

## Basic Table Syntax

General structure:

```
CREATE TABLE table_name (  
    column1 datatype,  
    column2 datatype,  
    column3 datatype  
);
```

This means: “Create a table with these columns and data types.”

**Example: Simple Students Table**

```
CREATE TABLE students (  
  id INT,  
  name VARCHAR(50),  
  age INT,  
  joining_date DATE  
);
```

**Understanding This Query**

Let's break it slowly.

**id INT:**

- id → column name
- INT → integer datatype

This column stores whole numbers like: 1, 2, 100, 5000

Usually, IDs use integers.

**name VARCHAR(50):** VARCHAR stores text.

The number (50) means: maximum 50 characters allowed.

Examples: John, Sarah, Michael

**age INT:** Again, using integer because age is numeric.

**joining\_date DATE:** This stores dates.

Example: 2026-05-20

Format: YYYY-MM-DD

**Real-World Example: Designing an E-Commerce Table**

Imagine creating a products table for an online store. You may design:

```
CREATE TABLE products (  
  product_id INT,  
  product_name VARCHAR(100),  
  category VARCHAR(50),  
  price INT,  
  launch_date DATE  
);
```

Notice how each datatype matches the type of information:

- numbers → INT
- text → VARCHAR
- dates → DATE

This is exactly how real databases are designed.

## More CREATE TABLE Examples

### Library System

```
CREATE TABLE books (  
    book_id INT,  
    title VARCHAR(150),  
    author VARCHAR(100),  
    publish_date DATE  
);
```

### Movies Table:

```
CREATE TABLE movies (  
    movie_id INT,  
    movie_name VARCHAR(100),  
    release_date DATE,  
    rating INT  
);
```

### Employees Table:

```
CREATE TABLE employees (  
    employee_id INT,  
    employee_name VARCHAR(100),  
    department VARCHAR(50),  
    joining_date DATE  
);
```

### A Small Important Detail

SQL ignores spaces and line formatting.

This: `CREATE TABLE students (id INT, name VARCHAR(50));`

and this:

```
CREATE TABLE students (  
    id INT,  
    name VARCHAR(50)  
);
```

mean the same thing.

But formatting properly makes queries:

- cleaner
- easier to read
- more professional

## Common Beginner Mistakes

### Forgetting commas between columns

Wrong:

```
CREATE TABLE students (  
  id INT  
  name VARCHAR(50)  
);
```

Correct:

```
CREATE TABLE students (  
  id INT,  
  name VARCHAR(50)  
);
```

### Forgetting semicolon

SQL queries usually end with:

```
;
```

### Choosing wrong datatype

Example:

```
age VARCHAR(100)  
Age should ideally be:  
age INT
```

## Why CREATE Statements Matter So Much

Everything in SQL depends on structure.

Good structure makes:

- querying easier
- analytics faster
- systems cleaner

Poor structure creates:

- messy data
- duplicate information
- difficult reports

Professional databases are carefully planned before large amounts of data are inserted.

## // Understanding Data Types

When creating tables, SQL needs to know: *what kind of data each column will store*. This is done using **data types**.

Different data types are designed for different kinds of information:

- numbers
- text
- dates
- time
- true/false values
- large content
- decimal values

Choosing correct datatypes helps databases remain:

- fast
- organized
- memory efficient

### Numeric Data Types

Numeric datatypes are used for storing numbers.

**INT:** Stores whole numbers.

Examples:

- age
- quantity
- IDs
- stock count

Example:

```
CREATE TABLE students (  
    student_id INT,  
    age INT  
);
```

**BIGINT:** Used for very large whole numbers. Useful when normal INT range is not enough.

Real-world examples:

- large transaction IDs
- social media post IDs
- huge analytics systems

Example:

```
CREATE TABLE youtube_views (  
    total_views BIGINT  
);
```

**DECIMAL:** Used for precise decimal numbers.

Very important for:

- money
- banking
- financial systems

Because floating-point inaccuracies can cause serious problems.

Example:

```
CREATE TABLE products (  
  price DECIMAL(10,2)  
);
```

Here:

10 = total digits

2 = digits after decimal

Examples:

499.99

1200.50

### Why Not INT for Prices?

Using: price INT would remove decimal values.

That means: 499.99 → 499 which is bad for financial systems.

**FLOAT / DOUBLE:** Used for approximate decimal values.

Common in:

- scientific calculations
- measurements
- analytics

But not ideal for exact money calculations.

**Example**

```
CREATE TABLE weather (  
  temperature FLOAT  
);
```

## Text Data Types

Used for storing characters and text.

**VARCHAR:** Stores variable-length text. Most commonly used text datatype.

Examples:

- names
- emails
- city names

Example: `name VARCHAR(100)`

Maximum: 100 characters



**CHAR:** Stores fixed-length text. If length is fixed, CHAR can be faster.

Examples:

- country codes
- gender values
- short status codes

Example: `country_code CHAR(3)`

Examples:

IND  
USA  
UAE

### Difference Between VARCHAR and CHAR:

| VARCHAR          | CHAR                         |
|------------------|------------------------------|
| Variable length  | Fixed length                 |
| Saves space      | Faster for fixed-size values |
| Common for names | Common for codes             |

**TEXT:** Used for storing large text content.

Examples:

- article content
- comments
- blog descriptions

Example

```
CREATE TABLE blogs (
  article TEXT
);
```

## Date & Time Data Types

Used for storing time-related information. Very important in analytics and business systems.

**DATE:** Stores only date.

Format: YYYY-MM-DD

Example: `joining_date DATE`

**TIME:** Stores time only.

Example: `meeting_time TIME`

Example value: 14:30:00

**DATETIME:** Stores both date and time together. Very commonly used.

Example: `created_at DATETIME`

Example: 2026-05-20 14:30:00

### **Real-World Usage**

DATETIME is heavily used for:

- login timestamps
- order creation
- payment records
- social media posts

Almost every modern application tracks timestamps.

**BOOLEAN:** Used for true/false values.

Examples:

- `is_active`
- `is_paid`
- `email_verified`

Example: `is_active BOOLEAN`

Possible values:

`TRUE`  
`FALSE`

In MySQL, BOOLEAN is internally treated like:

1 = TRUE  
0 = FALSE

**BLOB:** BLOB stands for: Binary Large Object

Used for storing binary files like:

- images
- videos
- documents

Example: `profile_photo BLOB`

Though in modern systems, files are often stored separately and only file paths are stored in databases.

## Full Real-World Example Table

```
CREATE TABLE employees (  
    employee_id INT PRIMARY KEY,  
    employee_name VARCHAR(100) NOT NULL,  
    age INT,  
    salary DECIMAL(10,2),  
    joining_date DATE,  
    login_time DATETIME,  
    country_code CHAR(3),  
    is_active BOOLEAN,  
    bio TEXT  
);
```

This table uses many professional datatypes together.

## How Professionals Choose Datatypes

Professionals usually think about:

- accuracy
- storage efficiency
- future scalability
- query performance

For example:

- prices → DECIMAL
- IDs → INT
- timestamps → DATETIME
- descriptions → TEXT

Good datatype selection improves system quality significantly.

## Common Beginner Mistakes

### Using VARCHAR for Everything

Very common beginner habit.

Example:

```
age VARCHAR(50)  
salary VARCHAR(50)
```

This creates:

- poor validation
- slower queries
- inconsistent data

### Using FLOAT for Money

Can create precision errors.

Prefer: DECIMAL

## Using Tiny VARCHAR Limits

Example: `name VARCHAR(5)`

Now: Alexander cannot fit.

## What You Learned

At this point, you now know many important SQL datatypes:

- how to create databases
- how to select databases using `USE`
- how to create tables
- how datatypes work
- when to use:
  - `INT`
  - `VARCHAR`
  - `DATE`

| Type     | Purpose                    |
|----------|----------------------------|
| INT      | Whole numbers              |
| BIGINT   | Large numbers              |
| DECIMAL  | Precise decimal values     |
| FLOAT    | Approximate decimal values |
| VARCHAR  | Variable text              |
| CHAR     | Fixed text                 |
| TEXT     | Large text                 |
| DATE     | Dates                      |
| TIME     | Time                       |
| DATETIME | Date + time                |
| BOOLEAN  | True/False                 |
| BLOB     | Binary files               |

These are the building blocks used while designing professional databases.

## Closing Thought:

Creating tables may feel simple right now. But this is the architectural phase of databases. Every major application in the world started with someone deciding: what columns should exist, what type of data should be stored, and how information should be structured. And now, you are learning to make those same decisions yourself.

## // Constraints While Creating Tables

Until now, you have learned how to create databases, define tables, and choose data types. But there is still one big problem.

What stops someone from inserting:

- an empty student name?
- duplicate email IDs?
- invalid or inconsistent data?

Without rules, databases can become messy very quickly. Imagine a school database where:

- two students share the same roll number
- names are left blank
- important fields contain random values

The system would slowly become unreliable. This is why databases use something called:

### Constraints

Constraints are rules applied to table columns that control what kind of data is allowed inside the database. They help maintain:

- accuracy
- consistency
- reliability

Think of constraints like security guards standing at the entrance of a database table. They check incoming data and reject anything that breaks the rules.

### Why Constraints Matter So Much?

In small practice tables, mistakes may seem harmless. But imagine a banking system where:

- account numbers are duplicated
- transaction dates are missing
- customer IDs are repeated

Even tiny inconsistencies can create serious real-world problems.

Constraints exist because databases are trusted systems. Companies rely on them to store important information correctly. Good databases are not only about storing data. They are about storing **valid data**.

## Constraints Are Added During Table Creation

Constraints are usually written while creating tables.

General idea:

```
CREATE TABLE table_name (
    column_name datatype CONSTRAINT
);
```

You attach rules directly to columns.

## PRIMARY KEY

The PRIMARY KEY constraint is one of the most important concepts in relational databases. A primary key uniquely identifies every row in a table. Think about students in a school.

Many students may have:

- same names
- same age
- same city

But each student has a unique roll number. That unique identity is similar to a primary key.

### What PRIMARY KEY Does?

A primary key:

- must be unique
- cannot contain NULL values
- identifies each row uniquely

This ensures that every record can be identified properly.

### Example: Students Table

```
CREATE TABLE students (
    student_id INT PRIMARY KEY,
    name VARCHAR(50),
    age INT
);
```

Here: student\_id, becomes the unique identity for every student.

### Real-World Examples of PRIMARY KEY

| Table     | Primary Key |
|-----------|-------------|
| Students  | student_id  |
| Employees | employee_id |
| Products  | product_id  |
| Orders    | order_id    |

Almost every professional table contains some form of primary key.

## What Happens If Duplicate Values Are Inserted?

Suppose this already exists:

| student_id | name |
|------------|------|
| 1          | John |

Now if someone tries: `INSERT INTO students VALUES (1, 'Sarah', 20);`

MySQL will reject it because: `student_id = 1`, already exists. This prevents duplicate identities.

## NOT NULL

Some information is too important to be left empty.

Imagine:

- a student record without a name
- an employee record without salary
- an order without customer details

Such data becomes incomplete and unreliable. This is where NOT NULL helps.

## What NOT NULL Does?

NOT NULL ensures that a column must always contain a value. Empty values are not allowed.

**Example:**

```
CREATE TABLE employees (
  employee_id INT PRIMARY KEY,
  employee_name VARCHAR(100) NOT NULL,
  salary INT
);
```

Here: `employee_name`, cannot be left empty.

## Example Error

This would fail: `INSERT INTO employees VALUES (1, NULL, 50000);`

Because: `employee_name`, is required.

## Real-World Importance

NOT NULL is commonly used for:

- names
- emails
- IDs
- important dates
- required fields

It helps ensure essential information always exists.

## UNIQUE

Now imagine a website where two users accidentally register with the same email address. That would create confusion:

- password resets break
- notifications go wrong
- accounts overlap

Some fields should contain only unique values. This is where the UNIQUE constraint helps.

### What UNIQUE Does?

UNIQUE prevents duplicate values inside a column.

Unlike PRIMARY KEY:

- UNIQUE allows NULL values (usually one)
- but all non-null values must be different

### Example:

```
CREATE TABLE users (
  user_id INT PRIMARY KEY,
  email VARCHAR(100) UNIQUE,
  username VARCHAR(50)
);
```

Now:

- duplicate emails are not allowed
- each email must be different

**Duplicate Example** Suppose this exists:

|                |
|----------------|
| <b>email</b>   |
| john@gmail.com |

Trying: `INSERT INTO users VALUES (2, 'john@gmail.com', 'john2');`  
will fail because the email already exists.

### Real-World Usage of UNIQUE

UNIQUE is commonly used for:

- email addresses
- usernames
- phone numbers
- employee codes

Anywhere duplicate values could create problems.



## DEFAULT

Sometimes you want MySQL to automatically insert a value if the user does not provide one. Imagine a new account being created. If no status is specified, the system may automatically set: active. This is where DEFAULT becomes useful.

### What DEFAULT Does?

DEFAULT assigns a predefined value automatically when no value is given.

#### Example:

```
CREATE TABLE accounts (  
    account_id INT PRIMARY KEY,  
    username VARCHAR(50),  
    status VARCHAR(20) DEFAULT 'active'  
);
```

Now if someone inserts:

```
INSERT INTO accounts (account_id, username)  
VALUES (1, 'john');
```

The table automatically stores:

| account_id | username | status |
|------------|----------|--------|
| 1          | john     | active |

### Real-World Usage of DEFAULT

DEFAULT is commonly used for:

- account status
- registration dates
- country names
- default settings
- stock availability

It reduces repetitive manual input.

## Combining Constraints Together

Real-world tables usually combine multiple constraints together.

Example:

```
CREATE TABLE customers (
  customer_id INT PRIMARY KEY,
  customer_name VARCHAR(100) NOT NULL,
  email VARCHAR(100) UNIQUE,
  country VARCHAR(50) DEFAULT 'India'
);
```

Here:

- `customer_id` → uniquely identifies customers
- `customer_name` → required
- `email` → cannot repeat
- `country` → defaults to India if not specified

This is much closer to how professional databases are designed.

## Constraints Make Databases Reliable

Without constraints:

- duplicate data appears
- important fields become empty
- systems become inconsistent

Constraints protect databases from bad data. In many ways, they act like quality-control systems for information.

## Common Beginner Mistakes

### Forgetting PRIMARY KEY:

Tables without primary keys often become difficult to manage later.

### Overusing NOT NULL:

Not every column must be mandatory.

Some fields can logically remain optional.

### Confusing PRIMARY KEY and UNIQUE:

| PRIMARY KEY          | UNIQUE                |
|----------------------|-----------------------|
| Only one per table   | Multiple allowed      |
| Cannot contain NULL  | Can allow NULL        |
| Main identity column | Additional uniqueness |

### **What You Learned?**

At this point, you now know how to use:

- PRIMARY KEY
- NOT NULL
- UNIQUE
- DEFAULT

while creating tables.

These constraints help create:

- cleaner databases
- reliable systems
- consistent data structures

### **Closing Thought:**

Creating tables is not only about columns and datatypes. It is also about defining rules. Because good databases are not just systems that *store* data. They are systems that protect the quality of data.

## // INSERT Statement - Adding Data to a Table

Up to this point in the course, you have designed databases, created tables, chosen appropriate data types, and applied constraints to ensure that only valid data can be stored. Your database is now like a beautifully constructed building with well-planned rooms and strong walls.

But there is one problem. The building is completely empty. A database without data is like:

- a library with no books,
- a school without students,
- or an online shopping website with no products.

The structure may be perfect, but it has nothing useful to offer. This is where the **INSERT** statement comes into the picture. The INSERT statement is used to place new records into a table. It is the command that transforms an empty database into a valuable source of information.

### Data is Born Through INSERT

Imagine you have just launched a brand-new online shopping website. You have already created a **Products** table with columns such as:

- Product ID
- Product Name
- Price
- Category
- Stock

When a new smartphone arrives at your warehouse, how does it appear on the website? Someone doesn't edit the database manually. Instead, the application executes an INSERT statement that adds a completely new row into the Products table.

The same thing happens everywhere around us. When:

- you register on Instagram,
- order food from Zomato,
- book a train ticket,
- create a Gmail account,
- or enroll in a college,

new information is inserted into a database. Every new account, every transaction, every booking and every purchase begins with an INSERT operation.

## What Does INSERT Actually Do?

The INSERT statement tells SQL: "Create a new row inside this table and store the following information." Unlike the CREATE statement, which builds the structure of a table, INSERT works **inside** that structure by adding actual records.

Think of a notebook. Creating the notebook is similar to using CREATE TABLE. Writing information on each page is similar to using INSERT. Every INSERT command adds another page filled with information.

## Basic Syntax

The general syntax of the INSERT statement is:

```
INSERT INTO table_name VALUES
(value1, value2, value3, ...);
```

Let's understand each part.

- **INSERT INTO** tells SQL that we want to add a new record.
- **table\_name** is the table where the data will be stored.
- **VALUES** contains the information that should be inserted.

SQL reads the values from left to right and places them into the corresponding columns.

## Example

Suppose we have the following table.

| student_id | student_name | age | city |
|------------|--------------|-----|------|
|------------|--------------|-----|------|

To insert a new student:

```
INSERT INTO students VALUES (101, 'Rahul Sharma', 18, 'Mumbai');
```

A new row is now added to the table.

| student_id | student_name | age | city   |
|------------|--------------|-----|--------|
| 101        | Rahul Sharma | 18  | Mumbai |

If another student joins,

```
INSERT INTO students VALUES (102, 'Priya Patel', 19, 'Pune');
```

the table continues to grow. Every INSERT statement creates one more record.

## A Database Grows One Record at a Time

Think about your school's student management system. At the beginning of a new academic year, hundreds of students take admission. The database does not recreate the table every year.

Instead, it simply inserts new student records one after another. Over time, the database grows larger while the table structure remains the same.

This is one of the biggest strengths of relational databases. The structure stays stable while the data continuously changes.

### **Column Order Matters**

One important rule when using INSERT is that SQL assumes the values are written in the same order as the columns were created. Suppose the table was created as: student\_id, student\_name, age, city Then SQL expects values in exactly the same order.

Correct: INSERT INTO students VALUES (101,'Rahul',18,'Mumbai');

Incorrect: INSERT INTO students VALUES ('Mumbai',18,'Rahul',101);

Although four values are provided, they no longer match the correct columns. This can produce errors or incorrect data.

### **Common Beginner Mistakes**

Learning INSERT is usually straightforward, but beginners often make a few common mistakes.

#### **Forgetting Quotes Around Text**

Wrong: INSERT INTO students VALUES (101,Rahul,18,Mumbai);

Correct: INSERT INTO students VALUES (101,'Rahul',18,'Mumbai');

Strings and text values should always be enclosed in single quotes.

### **Mismatching Data Types**

Suppose age is an integer. Writing 'Twenty' instead of 20 may generate an error because SQL expects a number.

### **Violating Constraints**

Suppose the table contains: student\_id INT PRIMARY KEY and a record already exists with: student\_id = 101, Trying to insert another student with the same ID will fail. The PRIMARY KEY constraint prevents duplicate records.

Similarly,

- NOT NULL prevents empty values.
- UNIQUE prevents duplicate values.
- DEFAULT automatically inserts predefined values when necessary.

Constraints continue protecting the database even during INSERT operations.

## Real-World Fact

Large companies process millions of INSERT operations every single day. Consider a busy online payment platform. Every successful payment creates a new transaction record.

During shopping festivals such as Black Friday or Diwali sales, companies may insert thousands of new records every second. Modern database systems are designed to handle enormous numbers of INSERT operations efficiently while ensuring that every transaction is stored safely.

## INSERT vs UPDATE

Students often confuse these two commands. The difference is simple. **INSERT** creates a completely new row. **UPDATE** changes information inside an existing row.

For example,

A new employee joins the company. → Use **INSERT**

The same employee receives a salary increase. → Use **UPDATE**

One adds information. The other modifies information.

## Thinking Like a Data Analyst

As a data analyst, you will rarely type hundreds of INSERT statements manually.

Most of the time, data arrives automatically from:

- CSV files,
- Excel spreadsheets,
- business applications,
- online forms,
- APIs,
- payment systems,
- IoT devices.

However, understanding INSERT is still essential because every imported dataset, every application, and every automated process ultimately uses INSERT operations behind the scenes.

When you import a CSV file into MySQL, thousands of INSERT operations are effectively performed automatically.

## What You Learned

After completing this topic, you now understand:

- why INSERT is used
- how data enters a database
- the basic INSERT syntax
- how SQL matches values with columns

- common beginner mistakes
- how constraints affect inserted data
- the difference between INSERT and UPDATE

You are now ready to learn different ways of inserting data, including inserting specific columns, multiple rows at once, and importing large datasets efficiently.

### **Closing Thought**

Creating a table is like building an empty house. The INSERT statement is what fills that house with life. Every customer account, every online order, every movie on Netflix, every bank transaction, and every message sent on social media exists because, at some point, an INSERT statement quietly added a new record to a database.

Without INSERT, databases would remain nothing more than beautifully designed but completely empty containers.



## // UPDATE Statement - Modifying Existing Data

Imagine that you have successfully built a database for a company. Employees have been added, customers have registered, products have been listed, and orders are being placed every day.

Everything seems perfect. But after a few weeks, something changes. An employee receives a promotion. A customer moves to another city. The price of a laptop increases. A student changes their phone number. A patient is transferred to a different doctor.

Does the company delete the old record and create a completely new one every time something changes? Of course not. Instead, it simply modifies the existing information.

This is exactly what the **UPDATE** statement is designed to do.

### **Data Never Stays the Same**

One of the biggest differences between the real world and a database is that information is constantly changing. Think about your own life.

You may:

- change your mobile number,
- move to a new address,
- receive a salary increment,
- complete a new degree,
- or change your email address.

Your identity remains the same, but some of your information changes over time. Databases work in exactly the same way.

Instead of creating a completely new record every time something changes, SQL allows us to update only the required information while keeping the rest of the record unchanged.

### **What Does UPDATE Actually Do?**

The UPDATE statement modifies data that already exists inside a table. It tells SQL: "Find the record that matches this condition and change the specified value."

Unlike INSERT, which creates new rows, UPDATE works on rows that already exist. Think of a student's report card. The student's name, roll number and admission date remain the same.

Only the marks are updated after each examination. The report card is not recreated every semester. Only the necessary information changes.

## Basic Syntax

The general syntax of the UPDATE statement is:

```
UPDATE table_name SET column_name = new_value WHERE condition;
```

Let us understand each part.

- **UPDATE** tells SQL which table needs modification.
- **SET** specifies which column should be changed.
- **WHERE** identifies the row or rows that should be updated.

Without these three parts, SQL would not know what information needs to change.

## Example

Suppose we have the following table.

| student_id | student_name | city   |
|------------|--------------|--------|
| 101        | Rahul Sharma | Mumbai |
| 102        | Priya Patel  | Pune   |

Rahul moves from Mumbai to Delhi. Instead of deleting his record and creating another one, we simply update his city.

```
UPDATE students SET city = 'Delhi' WHERE student_id = 101;
```

After executing the query:

| student_id | student_name | city  |
|------------|--------------|-------|
| 101        | Rahul Sharma | Delhi |
| 102        | Priya Patel  | Pune  |

Only one value has changed. Everything else remains exactly the same.

## Why the WHERE Clause is So Important

The WHERE clause acts like an address. It tells SQL exactly which record should be modified.

Imagine telling a courier company: "Deliver this package." Without an address, the courier wouldn't know where to go. Similarly, UPDATE without a WHERE clause doesn't know which specific row you intended to modify.

Instead, SQL assumes: "Update every row." This is one of the most common and dangerous beginner mistakes.

## UPDATE Without WHERE

Consider the following table.

| employee_name salary |       |
|----------------------|-------|
| Rahul                | 50000 |
| Priya                | 60000 |
| Aman                 | 55000 |

Now suppose someone writes:

```
UPDATE employees SET salary = 70000;
```

Notice something. There is **no WHERE clause**. SQL updates every employee.

The result becomes:

| employee_name salary |       |
|----------------------|-------|
| Rahul                | 70000 |
| Priya                | 70000 |
| Aman                 | 70000 |

Every salary has changed. Perhaps this wasn't the intention at all. This is why experienced database developers always double-check their WHERE conditions before executing an UPDATE query.

### Real-World Example

Imagine an airline company. A passenger changes their mobile number. Instead of deleting the booking and creating a new one, the airline simply updates one field.

```
UPDATE passengers SET phone = '9876543210' WHERE passenger_id = 105;
```

The passenger's:

- booking history,
- passport details,
- payment information,
- travel records

remain unchanged. Only the phone number is updated. This saves time and preserves the integrity of the data.

### Updating Multiple Columns

Sometimes more than one piece of information changes at the same time. Suppose an employee gets promoted. Their:

- designation,
- department,
- salary

all change together. SQL allows multiple columns to be updated in a single query.

Example:

```
UPDATE employees SET designation = 'Senior Developer', department = 'Technology', salary = 85000 WHERE employee_id = 101;
```

This updates three different columns with one statement.

### **Updating Using Calculations**

One of SQL's powerful features is that new values don't always have to be typed manually. They can also be calculated. Suppose a company announces a 10% salary increase. Instead of updating every employee individually, SQL can perform the calculation automatically.

```
UPDATE employees SET salary = salary * 1.10 WHERE department = 'IT';
```

Notice that the new salary is calculated using the existing salary. This type of update is extremely common in business applications.

### **A Day in the Life of a Data Analyst**

Imagine you are working for an online shopping company. During a festive sale, management decides:

- Increase all laptop prices by 8%.
- Reduce the stock of sold products.
- Mark discontinued items as unavailable.
- Correct the spelling of a product name.

All of these tasks involve UPDATE statements. Data analysts and database administrators perform such operations regularly to keep information accurate and up to date.

### **Common Beginner Mistakes**

Learning UPDATE is simple, but beginners often make a few common mistakes.

#### **Forgetting the WHERE Clause**

This is the most dangerous mistake. It updates every row in the table. Always verify your WHERE condition before executing the query.

#### **Updating the Wrong Column**

Suppose you accidentally write:

```
UPDATE students SET age = 'Mumbai';
```

Instead of updating the city, you have attempted to store text inside a numeric column. Choosing the correct column is just as important as writing the correct value.

## Using Incorrect Data Types

If salary is numeric, salary = 'High' is invalid.

The datatype of the new value should match the datatype of the column.

## Forgetting Quotes Around Text

Incorrect: UPDATE students SET city = Mumbai;

Correct: UPDATE students SET city = 'Mumbai';

Text values should always be enclosed in single quotes.

## Interesting Fact

Large organizations perform millions of UPDATE operations every day. Think about Google Maps. Every traffic update, road closure, or change in estimated travel time involves updating existing information inside massive databases.

Similarly, banking systems continuously update:

- account balances,
- transaction status,
- loan information,
- and payment records.

Without UPDATE operations, databases would quickly become outdated and unreliable.

## INSERT vs UPDATE

Students often confuse these two commands. The difference is very simple.

### INSERT

Creates a new row

Adds new information

Used when data enters the database

### UPDATE

Modifies an existing row

Changes existing information

Used when data changes later

Think of it like this: A new employee joins the company. → **INSERT**

The same employee gets promoted six months later. → **UPDATE**

One creates. The other modifies.

## Thinking Like a Database Professional

Professional developers rarely update data without first checking it. A common habit is to first run a SELECT query using the same WHERE condition.

For example:

```
SELECT * FROM employees WHERE employee_id = 101;
```

Once they verify the correct record has been selected, they execute:

```
UPDATE employees SET salary = 75000 WHERE employee_id = 101;
```

This simple habit prevents accidental changes and is followed in many software companies around the world.

## **What You Learned**

After completing this topic, you now understand:

- what the UPDATE statement does
- why existing data needs to change
- the syntax of UPDATE
- the importance of the WHERE clause
- how to update one or multiple columns
- how to perform calculated updates
- common mistakes made by beginners
- the difference between INSERT and UPDATE

You are now ready to perform real-world modifications on existing databases with confidence.

## **Closing Thought**

If the INSERT statement gives life to a database, the UPDATE statement helps that life evolve. People move, prices change, products improve, and businesses grow. A database that cannot adapt to change quickly becomes outdated.

The UPDATE statement ensures that information remains current, accurate, and useful, making it one of the most frequently used SQL commands in modern software systems.

## // DELETE Statement - Removing Data from a Table

Imagine that your database has been running successfully for several years. Thousands of customers have registered. Millions of orders have been placed. Hundreds of employees have joined the company.

New products are added every day. But over time, another challenge appears. Some products are discontinued. Some customers permanently delete their accounts.

Old temporary records are no longer needed. Test data inserted during software development must be removed. Duplicate records accidentally created by users need to be cleaned up.

Should all this unnecessary information remain inside the database forever? Of course not. Just as information enters a database through the **INSERT** statement and changes through the **UPDATE** statement, unwanted information is removed using the **DELETE** statement.

### Why Do We Delete Data?

Not every piece of information needs to stay in a database forever. Imagine your classroom. At the beginning of the academic year, the attendance register contains only new students. As the year progresses, students transfer to different schools, graduate, or leave the institution. If their records were never removed or archived, the register would eventually become confusing and difficult to manage.

Databases face the same challenge. Old, incorrect, or unnecessary records can:

- occupy storage,
- slow down searches,
- create confusion during analysis,
- and reduce the quality of reports.

DELETE helps keep databases clean and meaningful.

### What Does DELETE Actually Do?

The DELETE statement removes one or more existing rows from a table. Unlike UPDATE, which changes the values inside a row, DELETE removes the entire row itself. Think of a notebook. If UPDATE is like correcting a sentence with an eraser, DELETE is like tearing an entire page out of the notebook. The page no longer exists. Similarly, the record no longer exists inside the database.

### Basic Syntax

The general syntax of the DELETE statement is:

```
DELETE FROM table_name WHERE condition;
```

Let us understand the parts.

- **DELETE FROM** tells SQL that records should be removed from the specified table.

- **WHERE** identifies exactly which records should be deleted.

The WHERE clause is extremely important because it controls which rows are affected.

### Example

Suppose we have the following table.

| <b>student_id</b> | <b>student_name</b> | <b>city</b> |
|-------------------|---------------------|-------------|
| 101               | Rahul Sharma        | Mumbai      |
| 102               | Priya Patel         | Pune        |
| 103               | Aman Verma          | Delhi       |

Aman transfers to another school, and his record needs to be removed.

```
DELETE FROM students WHERE student_id = 103;
```

After executing the query:

| <b>student_id</b> | <b>student_name</b> | <b>city</b> |
|-------------------|---------------------|-------------|
| 101               | Rahul Sharma        | Mumbai      |
| 102               | Priya Patel         | Pune        |

The entire row belonging to Aman has been removed.

### The Importance of the WHERE Clause

The WHERE clause acts as a filter. It tells SQL exactly which records should be removed. Imagine a teacher saying, "Remove the student with Roll Number 25." Only one student leaves the classroom.

Now imagine saying, "Everyone leave." Every student walks out. SQL behaves the same way. Without a WHERE clause, DELETE removes every record from the table.

### DELETE Without WHERE

Suppose we have:

| <b>product_id</b> | <b>product_name</b> |
|-------------------|---------------------|
| 1                 | Laptop              |
| 2                 | Keyboard            |
| 3                 | Mouse               |

Now someone accidentally writes:

```
DELETE FROM products;
```



Notice that there is no WHERE clause. The result? Every record is removed. The table still exists, but it becomes completely empty. This is one of the most dangerous mistakes beginners make.

### **Real-World Example**

Imagine an online shopping website. A seller permanently discontinues a product. The company decides to remove it from its catalog.

```
DELETE FROM products WHERE product_id = 501;
```

The product disappears from the database. Customers can no longer purchase it because its record no longer exists.

### **Deleting Multiple Records**

DELETE is not limited to removing one row. It can remove many rows that satisfy a condition. Suppose a company wants to remove all cancelled orders.

```
DELETE FROM orders WHERE order_status = 'Cancelled';
```

Every cancelled order is removed together. This saves time and reduces repetitive work.

### **A Story from the Real World**

Imagine you work for an online event booking company. Every day, thousands of temporary bookings are created. Some customers complete the payment. Others abandon the booking halfway. After thirty days, management decides to remove all unpaid bookings to keep the database clean.

Instead of deleting records one by one, SQL removes all matching records in a single statement. This is exactly how DELETE is used in many real businesses.

### **DELETE vs TRUNCATE**

Students often confuse DELETE with TRUNCATE. Although both remove data, they behave differently.

| <b>DELETE</b>           | <b>TRUNCATE</b>              |
|-------------------------|------------------------------|
| Removes selected rows   | Removes every row            |
| Uses WHERE clause       | Cannot use WHERE             |
| Slower for large tables | Faster                       |
| Table structure remains | Table structure also remains |

Think of DELETE as removing selected books from a bookshelf. TRUNCATE is like emptying the entire bookshelf at once.

You will learn TRUNCATE in more detail later.

### **Common Beginner Mistakes**

Learning DELETE is simple, but beginners should be careful.

### **Forgetting the WHERE Clause**

This is by far the most common mistake. Instead of deleting one record, every record disappears. Always double-check the WHERE condition before executing the query.

### **Deleting the Wrong Record**

Suppose you write:

```
DELETE FROM students WHERE student_id = 102;
```

when you actually meant student 103. The wrong record is permanently removed. This is why professionals first verify the record using SELECT.

### **Ignoring Relationships**

In relational databases, tables are often connected using foreign keys. Suppose a customer has several orders. Deleting the customer may fail because those orders still reference that customer. This is one reason relational databases are called "relational."

Tables protect each other from accidental data loss.

### **Interesting Fact**

Large companies rarely perform permanent DELETE operations immediately. Instead, they often use a technique called **Soft Delete**.

Rather than removing a row completely, they add a column such as: `is_deleted` or `status` and simply mark the record as inactive. This allows administrators to recover information if it was deleted accidentally.

Many social media platforms, online shopping websites, and banking systems use this approach instead of permanently deleting important records.

### **Thinking Like a Database Professional**

Professional developers almost never execute DELETE directly. Instead, they follow a simple three-step process. First, check which records will be affected.

```
SELECT * FROM students WHERE student_id = 103;
```

If the correct record appears, then execute:

```
DELETE FROM students WHERE student_id = 103;
```

Finally, verify the deletion.

```
SELECT * FROM students WHERE student_id = 103;
```

This careful workflow prevents accidental data loss and is considered a best practice in software development.

## **What You Learned**

After completing this topic, you now understand:

- what the DELETE statement does
- why databases remove unnecessary records
- the syntax of DELETE
- the importance of the WHERE clause
- how to delete one or multiple rows
- the difference between DELETE and TRUNCATE
- common beginner mistakes
- how professionals safely delete data

You are now able to manage the complete life cycle of information inside a database.

## **Closing Thought**

A well-designed database is not simply one that stores a lot of information. It is one that stores the **right** information. As new data enters through INSERT and existing data changes through UPDATE, old and unnecessary information must sometimes make way for the new.

The DELETE statement keeps databases clean, organized, and relevant, ensuring that the information stored today continues to remain accurate and valuable tomorrow.

# SQL Sublanguages (Command Categories)

By now, you have learned several SQL commands.

You have created databases and tables using the **CREATE** statement, inserted new records using **INSERT**, viewed information using **SELECT**, modified records using **UPDATE**, and removed unwanted data using **DELETE**.

Although each command performs a different task, they are not random. SQL organizes these commands into different groups according to the type of work they perform. These groups are known as **SQL Sublanguages** or **SQL Command Categories**.

Think of SQL as a large toolbox.

A carpenter's toolbox contains many different tools. A hammer is used for driving nails, a screwdriver tightens screws, a measuring tape checks dimensions, and a saw cuts wood. Although every tool belongs to the same toolbox, each one has a different purpose.

SQL works in exactly the same way. It is one language, but within that language are several groups of commands, each designed for a specific type of task. Some commands build the structure of a database, some add or modify information, some control user permissions, while others ensure that transactions are completed safely.

Understanding these command groups makes SQL much easier to learn because instead of memorizing dozens of individual commands, you begin to understand the purpose behind them.

## Why Are SQL Commands Divided into Categories?

Imagine managing a large hospital database. On some days, the database administrator creates new tables for patient records. On other days, receptionists add new patients, doctors update medical reports, analysts generate reports, and administrators create backup copies.

Although everyone is working with the same database, they are performing completely different kinds of tasks. It would be confusing if every SQL command belonged to one giant list with no organization.

For this reason, SQL divides its commands into logical categories. Each category focuses on a particular type of database operation. This organization makes SQL easier to understand, easier to teach, and easier to use in real-world applications.

## // The Five Main SQL Sublanguages

Modern SQL is generally divided into five major categories. Each category has its own responsibility.

### 1. DDL (Data Definition Language)

DDL is responsible for **defining and changing the structure of a database**. Instead of working with the data itself, DDL works with the objects that store the data.

Using DDL, we can:

- create databases
- create tables
- modify existing tables
- rename tables
- remove tables
- remove databases

In simple words,

**DDL builds the container in which data will be stored.**

Think of constructing a new apartment building. Before people can move in, engineers must first build the rooms, walls, floors, and doors. DDL performs this construction work for databases.

Some common DDL commands are:

CREATE

ALTER

DROP

TRUNCATE

RENAME

You have already learned the **CREATE** statement, and in the upcoming chapters you will explore the remaining DDL commands in detail.

### 2. DML (Data Manipulation Language)

Once the database structure has been created, we need to work with the information stored inside it. This is the responsibility of DML. DML allows users to insert new records, update existing information, delete unnecessary data, and retrieve information from tables.

These are the commands used every day by developers, data analysts, and business applications. Think of a library. DDL builds the shelves. DML fills those shelves with books, rearranges them, replaces damaged books, and removes outdated ones.

The most commonly used DML commands are:

INSERT

SELECT

UPDATE

DELETE

These are the commands you have spent the last few chapters learning.

### **3. DCL (Data Control Language)**

Not everyone should have permission to view or modify every database. Imagine a university database. Students should be able to view their own marks. Teachers should be able to update marks. The principal should have access to every department. Visitors should not have access at all.

Managing these permissions is the responsibility of **Data Control Language (DCL)**. It controls who can access the database and what actions they are allowed to perform.

The two most common DCL commands are:

GRANT

REVOKE

Although beginners rarely use these commands, they are extremely important in large organizations where database security is essential.

### **4. TCL (Transaction Control Language)**

Suppose you transfer ₹5,000 from your bank account to your friend's account. The banking system performs two operations.

- Money is deducted from your account.
- Money is added to your friend's account.

Imagine if electricity failed after the first operation. Your money would disappear without reaching the other account. To prevent situations like this, databases use **transactions**. A transaction is a group of operations that should either complete entirely or not happen at all. Transaction Control Language manages these operations.

The most common TCL commands are:

COMMIT

ROLLBACK

SAVEPOINT

These commands make databases reliable and are one of the reasons modern banking systems can safely process millions of transactions every day.

## 5. DQL (Data Query Language)

Sometimes, we only want to read information without making any changes. For example:

- Find all employees in the IT department.
- Display customers from Mumbai.
- Show products costing more than ₹10,000.

This is the purpose of **Data Query Language (DQL)**. DQL focuses only on retrieving information from a database.

The primary command of DQL is:

**SELECT**

Many textbooks include SELECT under DML because it works with data. However, modern database literature often treats it as a separate category because it only retrieves information and never modifies it.

Both classifications are accepted in the SQL community. Throughout this book, we will treat **SELECT** as part of DQL to clearly distinguish reading data from changing data.

### A Simple Way to Remember

Imagine building and managing a school.

- **DDL** builds the classrooms.
- **DML** fills the classrooms with students and updates their records.
- **DQL** reads student information and generates reports.
- **DCL** decides who is allowed to enter the classrooms.
- **TCL** ensures important school operations complete safely without losing information.

Together, these five categories make SQL a complete language for managing databases.

### SQL Command Summary

| Sublanguage | Purpose                               | Common Commands                       |
|-------------|---------------------------------------|---------------------------------------|
| <b>DDL</b>  | Define and modify database structure  | CREATE, ALTER, DROP, TRUNCATE, RENAME |
| <b>DML</b>  | Insert, update and delete data        | INSERT, UPDATE, DELETE                |
| <b>DQL</b>  | Retrieve data from the database       | SELECT                                |
| <b>DCL</b>  | Control user permissions and security | GRANT, REVOKE                         |
| <b>TCL</b>  | Manage database transactions          | COMMIT, ROLLBACK, SAVEPOINT           |

## **Interesting Fact**

Although SQL has existed since the 1970s, these command categories are still used in almost every modern relational database system, including **MySQL**, **PostgreSQL**, **Oracle Database**, **Microsoft SQL Server**, and **SQLite**. Whether you become a software developer, database administrator, backend engineer, or data analyst, you will use commands from these categories throughout your career.

## **What You Learned**

After completing this chapter, you now understand:

- why SQL commands are divided into categories
- the purpose of SQL sublanguages
- the difference between DDL, DML, DQL, DCL, and TCL
- which commands belong to each category
- how these categories work together to manage a database



## Data Definition Language (DDL)

When we first began learning SQL, we spent a great deal of time understanding what data is, why databases exist, and how information is organized into tables. We also learned that before a database can store even a single piece of information, it must first have a proper structure.

Think about constructing a new house. Before furniture can be arranged, before people can move in, and before electricity or water can be used, someone must first build the house itself. The walls need to be erected, rooms must be planned, doors and windows must be installed, and the overall structure must be completed.

A database follows the same principle. Before we can insert customer details, employee records, movie information, or banking transactions, we first need to create the database and design the tables that will hold this information.

This responsibility belongs to **Data Definition Language**, commonly known as **DDL**. DDL is one of the most important sublanguages of SQL because it deals with the **structure** of a database rather than the data stored inside it. Instead of adding or modifying records, DDL creates, changes, renames, or removes the objects that make up a database.

In simple words, DDL answers the question: "**How should the database be built?**"

### Why Do We Need DDL?

Imagine you have been hired by a newly established hospital to develop its database system. The hospital tells you that they need to store information about:

- Patients
- Doctors
- Appointments
- Medicines
- Bills

Before the receptionist can register the first patient, you need to decide:

- What tables should exist?
- What columns should each table contain?
- Which column should be the Primary Key?
- What data types should each column have?
- Which values should be mandatory?

These decisions have nothing to do with the actual patient records. Instead, they define the **blueprint** of the database. This blueprint is created using DDL commands. Without DDL, there would be no tables, no columns, and nowhere to store data.

## What Does "Definition" Mean?

The word **Definition** means describing or specifying how something should be built. In SQL, Data Definition Language defines:

- the database itself,
- tables,
- columns,
- data types,
- constraints,
- and other database objects.

You can think of DDL as the architecture of a building. Architects don't decide who will live in the building. They decide:

- how many floors it will have,
- where the rooms will be,
- how large each room should be,
- and how everything connects together.

Similarly, DDL designs the structure of a database before any information is stored.

## DDL Commands

Data Definition Language consists of several important SQL commands. Each command has a different responsibility.

| Command         | Purpose                                                      |
|-----------------|--------------------------------------------------------------|
| <b>CREATE</b>   | Creates databases and tables                                 |
| <b>ALTER</b>    | Changes the structure of existing tables                     |
| <b>RENAME</b>   | Changes the name of database objects                         |
| <b>TRUNCATE</b> | Removes all records from a table while keeping its structure |
| <b>DROP</b>     | Permanently removes a database or table                      |

Together, these commands allow developers to build and maintain database structures throughout the life of an application.

## Revisiting CREATE

The first DDL command you learned was **CREATE**. CREATE is usually the very first SQL command executed in a new project because every database begins with an empty structure.

Using CREATE, we can build:

- new databases,
- new tables,
- and other database objects.

For example, creating a table:

```
CREATE TABLE students(  
    student_id INT PRIMARY KEY,  
    student_name VARCHAR(100),  
    age INT,  
    city VARCHAR(50)  
);
```

This statement creates the table structure. Notice that although the table now exists, it still contains no records. It is simply an empty container waiting to store information. Only after using commands such as **INSERT** does the table begin storing actual data.

## Databases Continue to Grow

Many beginners think that once a table is created, its design never changes. In reality, database structures evolve continuously. Imagine a college management system.

During the first year, the **students** table stores:

- Roll Number
- Name
- Age
- City

A few months later, the administration decides to store students' email addresses. Should the entire table be deleted and recreated? That would also delete all existing student records. Instead, developers simply modify the table by adding a new column.

Similarly, companies often need to:

- increase the size of a column,
- rename a column,
- remove an unnecessary field,
- or even rename an entire table.

DDL provides commands for performing all these tasks without rebuilding the database from scratch.

## DDL in the Real World

Almost every software application begins with DDL. When a new version of Instagram introduces a new profile field, database developers modify existing tables.

When an online shopping website adds a "Product Rating" feature, new columns are added to product tables. When a bank launches a new service, new tables are created to store customer information. Even though end users never see these changes, DDL commands quietly reshape the database behind the scenes. Database structures continue evolving throughout the lifetime of an application, often for many years.

## DDL vs DML

Students often confuse **DDL** with **DML** because both are used with databases. The difference is quite simple. DDL works with the **structure** of the database. DML works with the **data** stored inside that structure.

Think of a bookshelf. DDL builds the bookshelf. DML places books on the shelves, rearranges them, updates their information, or removes old books. Without the bookshelf, there is nowhere to keep the books. Without the books, the bookshelf serves little purpose.

Both are equally important and work together to create a complete database system.

## Interesting Fact

Large software companies rarely modify production databases manually. Instead, they use **database migration tools** that generate DDL commands automatically.

Whenever Facebook, Amazon, Netflix, or Google introduces a new feature, thousands of servers around the world execute carefully planned DDL scripts to update their database structures without interrupting millions of users. This process allows databases to evolve while the application continues running.

## Closing Thought

A database is never truly "finished."

Just as cities expand with new roads, buildings, and neighborhoods, databases continue to evolve with new tables, columns, and relationships. Data Definition Language provides the tools that make this evolution possible. It gives developers the ability to shape, improve, and redesign the foundation of a database without starting over, ensuring that the database grows alongside the organization it serves.

## // ALTER Statement

After creating a table, many beginners assume that its design remains fixed forever. In reality, databases are constantly changing. Businesses grow, new requirements emerge, and applications evolve over time. As a result, the structure of a database often needs to be modified without affecting the existing data.

Suppose you designed a **students** table for a college. Initially, the administration only wanted to store the student's roll number, name, age, and city. A few months later, the college introduces an online learning portal and decides to store each student's email address. Should you delete the entire table and create it again?

Certainly not. Doing so would also erase every student's record already stored in the database. Instead, SQL provides a command that allows us to change the structure of an existing table while preserving its data. This command is called **ALTER**.

The ALTER statement is one of the most frequently used DDL commands because it allows developers to adapt databases as requirements change. Rather than rebuilding everything from the beginning, we simply modify the existing structure.

### What Does ALTER Mean?

The word **alter** means **to change or modify something without replacing it completely**. In SQL, ALTER changes the definition of an existing database object. Unlike CREATE, which builds a new table, ALTER works on a table that already exists.

Using ALTER, we can:

- add new columns,
- remove unnecessary columns,
- modify data types,
- rename columns,
- add or remove constraints.

Because businesses rarely remain the same, ALTER becomes an essential command throughout the life of a database.

### General Syntax

```
ALTER TABLE table_name  
operation;
```

The operation specifies the type of change to be performed, such as adding a column, modifying a data type, or dropping a column.

## Common ALTER Operations

| Operation       | Purpose                                                                      |
|-----------------|------------------------------------------------------------------------------|
| ADD COLUMN      | Add a new column                                                             |
| MODIFY COLUMN   | Change a column's data type or size                                          |
| RENAME COLUMN   | Change a column name                                                         |
| DROP COLUMN     | Remove a column                                                              |
| ADD CONSTRAINT  | Add a new constraint                                                         |
| DROP CONSTRAINT | Remove an existing constraint <i>(syntax varies across database systems)</i> |

### Adding a New Column

One of the most common database changes is adding additional information. Imagine an employee database where only the employee's name and department were stored initially. Later, the HR department decides to record each employee's email address.

Instead of recreating the table, we simply add a new column.

```
ALTER TABLE employees
ADD email VARCHAR(100);
```

After executing this query, every existing record remains intact. The new column appears for all rows, with NULL values until data is entered.

### Modifying a Column

Business requirements often change. Suppose a product name column was originally created as: VARCHAR(50). As the company expands, longer product names become common. Instead of creating a new table, we increase the column size.

```
ALTER TABLE products
MODIFY product_name VARCHAR(150);
```

This changes the structure without affecting the existing records.

### Renaming a Column

Sometimes the data is correct, but the column name is not descriptive enough. For example: phone may later be renamed to mobile\_number to improve readability.

```
ALTER TABLE customers
RENAME COLUMN phone TO mobile_number;
```

Good column names make databases easier to understand and maintain.

## Removing a Column

Occasionally, information becomes unnecessary. Suppose a company stops recording fax numbers. The column can simply be removed.

```
ALTER TABLE customers  
DROP COLUMN fax_number;
```

Once removed, the column and its stored data are permanently deleted. For this reason, developers always verify backups before dropping columns in production databases.

## Real World Example

Imagine a food delivery application. Version 1 stores:

- Customer Name
- Phone Number
- Address

A year later, the company launches email notifications. Developers simply execute:

```
ALTER TABLE customers  
ADD email VARCHAR(100);
```

No customer accounts are lost. The application continues running while the database evolves. This ability to adapt without rebuilding the entire system is exactly why ALTER is such an important command.

## Common Mistakes

Beginners often make mistakes while using ALTER. Some common ones include:

- Trying to alter a table that does not exist.
- Changing a column's data type to one that is incompatible with the existing data.
- Removing a column without checking whether it is used elsewhere in the application.
- Forgetting to back up important data before making structural changes.

Professional database administrators carefully test ALTER commands before executing them on production systems.

## Best Practices

- Plan the new structure before modifying a table.
- Always create a database backup before making major structural changes.
- Use meaningful column names.
- Increase column sizes carefully to avoid unnecessary storage usage.
- Test ALTER statements on a development database before applying them to production.

## // RENAME Statement

When developing a database, choosing meaningful names for tables and columns is extremely important. Good names make a database easier to understand, maintain, and share with other developers.

However, database design is rarely perfect on the first attempt. As projects grow, developers often realize that some names are unclear, inconsistent, or no longer match the purpose of the data they store. Instead of deleting the table and creating a new one, SQL provides a simple way to change the names of database objects without affecting the stored data.

This is done using the **RENAME** statement. The RENAME command allows us to change the name of a table or one of its columns while keeping the data and structure intact. Only the name changes; everything else remains exactly the same.

Imagine changing the nameplate outside a classroom. The students, furniture, and lessons remain unchanged. Only the label on the door is different. The RENAME statement works in much the same way.

### Why Do We Need RENAME?

Suppose you are developing a Hospital Management System. Initially, you create a table named: patient. As the hospital expands, new tables are added such as:

- patient\_history
- patient\_reports
- patient\_billing

Now the original table name **patient** becomes confusing because it stores only the personal details of patients. A better name would be: patients or patient\_details. Instead of recreating the table and moving all the records, we simply rename it. This saves time, preserves data, and avoids unnecessary work.

### General Syntax:

Renaming a Table

```
RENAME TABLE old_table_name  
TO new_table_name;
```

Some database systems also support:

```
ALTER TABLE old_table_name  
RENAME TO new_table_name;
```

Renaming a Column

```
ALTER TABLE table_name  
RENAME COLUMN old_column_name  
TO new_column_name;
```



## Renaming a Table

Suppose we have a table named **student**. Later, we decide that **students** is a better name.

```
RENAME TABLE student TO students;
```

The table now appears with its new name. Notice that:

- All records remain unchanged.
- Primary Keys remain unchanged.
- Foreign Keys continue to work (depending on the database system).
- Only the table name changes.

## Renaming a Column

Sometimes the table name is correct, but a column name is not descriptive enough. Consider this table:

**id name ph**

Although **ph** stores phone numbers, the name is not very clear. A better name would be **phone\_number**.

```
ALTER TABLE customers
RENAME COLUMN ph
TO phone_number;
```

The data remains exactly the same. Only the column heading changes.

## Real-World Example

Imagine an online shopping company. Initially, the developers store customer names in a column called: **cust\_name**

A few years later, hundreds of developers begin working on the project. The abbreviation **cust\_name** becomes confusing for new developers. To improve readability, they rename it to: **customer\_name**

No customer information is lost. Every record remains exactly where it was. Only the column becomes easier to understand. This improves code quality and makes future development easier.

### Table Before Renaming

| Roll No | Name  | Age |
|---------|-------|-----|
| 101     | Rahul | 20  |
| 102     | Priya | 19  |

Table Name: student

**After Renaming**

Table Name: students

| Roll No | Name  | Age |
|---------|-------|-----|
| 101     | Rahul | 20  |
| 102     | Priya | 19  |

The contents remain completely unchanged.

**Common Mistakes**

Beginners often make these mistakes while using RENAME:

- Renaming a table that does not exist.
- Forgetting to update SQL queries that still use the old table name.
- Renaming columns to reserved SQL keywords.
- Choosing names that are too short or difficult to understand.

**Best Practices**

- Use meaningful and descriptive names.
- Follow a consistent naming convention.
- Avoid abbreviations unless they are universally understood.
- Rename objects only when necessary.
- Update application code after renaming database objects.

**Interesting Fact**

Large software companies frequently rename database objects as applications evolve. During major redesigns, developers often rename tables and columns to make the database easier for future teams to understand. Good naming conventions can save thousands of hours of maintenance over the lifetime of a project.

**CREATE vs ALTER vs RENAME**

| Command       | Purpose                                     |
|---------------|---------------------------------------------|
| <b>CREATE</b> | Creates a new database object               |
| <b>ALTER</b>  | Changes the structure of an existing object |
| <b>RENAME</b> | Changes only the name of an existing object |

Notice that RENAME does **not** change the data or structure. It simply gives an existing object a new identity.

## // TRUNCATE Statement

As databases grow, there are situations where we need to remove large amounts of data. Sometimes we want to clear old records, reset a table for testing, or prepare the database for a fresh set of information.

A common mistake beginners make is using the **DELETE** statement to remove every row from a table. Although this works, it is not always the best or fastest solution. SQL provides another command specifically designed for this purpose. This command is called **TRUNCATE**.

The **TRUNCATE** statement removes **all records** from a table while keeping the table itself, its columns, data types, indexes, and constraints intact. In other words, it empties the table without destroying its structure.

Imagine a classroom at the end of the academic year. Every student leaves the room, but the classroom, desks, chairs, and whiteboard remain exactly where they are. The room is simply empty and ready for the next batch of students.

That is exactly how **TRUNCATE** works.

### Why Do We Need TRUNCATE?

Imagine you are developing a **Library Management System**. During software testing, you insert thousands of sample book records.

Once testing is complete, the client asks for a clean database before the software goes live. One option is to delete every row individually. Another option is to remove the entire table. Neither is ideal. Deleting thousands of rows can take time, while dropping the table would also remove its structure.

Instead, we simply empty the table using **TRUNCATE**. The table remains ready for new data without needing to be recreated.

### What Does TRUNCATE Mean?

The word **truncate** means **to shorten by cutting off a part**. In SQL, it means removing every record from a table while leaving the table definition untouched.

Unlike **DROP**, **TRUNCATE** does not remove the table itself. Unlike **DELETE**, **TRUNCATE** removes every row at once instead of processing rows individually. Because of this, **TRUNCATE** is usually much faster than **DELETE** when all records need to be removed.

### General Syntax

```
TRUNCATE TABLE table_name;
```

This command removes all rows from the specified table while preserving its structure.

**Example:** Suppose we have the following table.

**Students Table**

| Roll No | Name  | Age |
|---------|-------|-----|
| 101     | Rahul | 20  |
| 102     | Priya | 19  |
| 103     | Aman  | 21  |
| 104     | Sneha | 20  |

Now execute: **TRUNCATE TABLE students;**

After execution, the table becomes:

| Roll No | Name | Age |
|---------|------|-----|
|---------|------|-----|

The table still exists. Its columns still exist. Its constraints still exist. Only the records have been removed.

### What Happens After TRUNCATE?

After truncating a table:

- The table remains in the database.
- All rows are removed.
- Column names remain unchanged.
- Data types remain unchanged.
- Constraints remain unchanged.
- The table is immediately ready for inserting new records.

Think of it as emptying a notebook without throwing the notebook away.

### Real-World Example

Imagine an online examination system. Every year, thousands of temporary records are stored while students practice mock tests. Once the examination season is over, these temporary practice records are no longer required.

Instead of deleting every row individually, the administrator simply executes:

**TRUNCATE TABLE mock\_test\_results;**

Within a few moments, the table becomes empty and is ready for the next academic year.

## TRUNCATE vs DELETE

Students often confuse TRUNCATE with DELETE because both remove data from a table. However, they serve different purposes.

| Feature                         | DELETE           | TRUNCATE         |
|---------------------------------|------------------|------------------|
| Removes selected rows           | ✓ Yes            | ✗ No             |
| Removes all rows                | ✓ Yes            | ✓ Yes            |
| Uses WHERE clause               | ✓ Yes            | ✗ No             |
| Removes rows one by one         | ✓ Yes            | ✗ No             |
| Faster for deleting all records | ✗ Usually slower | ✓ Usually faster |
| Keeps table structure           | ✓ Yes            | ✓ Yes            |

If you want to remove only specific records, use **DELETE**. If you want to remove every record from the table, **TRUNCATE** is usually the better choice.

## Common Mistakes

Beginners often make these mistakes while using TRUNCATE:

- Expecting to use a WHERE clause with TRUNCATE.
- Thinking TRUNCATE deletes the table itself.
- Confusing TRUNCATE with DROP.
- Forgetting that every row in the table will be removed.

Always verify that the table contains no important data before executing TRUNCATE.

## Best Practices

- Use TRUNCATE only when you want to remove every record.
- Create a backup before truncating important tables.
- Avoid using TRUNCATE on production databases without proper approval.
- Use DELETE instead if only certain rows need to be removed.
- Test the command on a development database before using it in a live environment.

## Interesting Fact

Many companies use TRUNCATE while importing fresh datasets. Data warehouses and reporting systems often clear temporary tables using TRUNCATE before loading new data each day. Because it is optimized for removing all rows, it helps large data-processing systems complete their work much faster.

## ALTER vs TRUNCATE

| Command         | Purpose                                         |
|-----------------|-------------------------------------------------|
| <b>ALTER</b>    | Changes the structure of a table                |
| <b>TRUNCATE</b> | Removes all records while keeping the structure |

## // DROP Statement

Every database begins with creation. Tables are created, columns are added, and records start filling the database. Over time, however, some database objects become unnecessary. Old projects are discontinued, temporary tables are no longer required, duplicate databases are removed, or an application is redesigned from scratch.

In such situations, simply deleting the data is not enough. Sometimes the entire database object itself must be removed. This is where the **DROP** statement comes into use. The DROP statement permanently removes a database object from the system. Unlike TRUNCATE, which only empties a table, DROP completely deletes the object along with everything stored inside it.

Imagine demolishing an old building. The rooms, walls, doors, furniture, and everything inside disappear completely. If someone wants the building again, it must be constructed from the beginning. That is exactly how the DROP statement works.

### Why Do We Need DROP?

Suppose you are developing an **Online Shopping System**. During development, you create several temporary tables to test different features. After testing is complete, these tables are no longer useful.

Keeping unnecessary tables inside the database only makes the system larger and more confusing. Instead of leaving unused tables behind, developers permanently remove them using the **DROP** statement. Similarly, if an entire database is no longer required, it can also be removed using DROP.

### What Does DROP Mean?

The word **drop** simply means **to remove or discard something completely**. In SQL, DROP permanently deletes a database object from the database system. Depending on the command, DROP can remove:

- Databases
- Tables
- Views
- Indexes
- Stored Procedures
- Other database objects

Once dropped, the object no longer exists. To use it again, it must be created from the beginning.

## General Syntax

Dropping a Table: **DROP TABLE** table\_name;

Dropping a Database: **DROP DATABASE** database\_name;

### Dropping a Table

Suppose a table named **temporary\_students** was created only for testing. After testing is complete, it is no longer required.

```
DROP TABLE temporary_students;
```

After executing this statement:

- The table disappears.
- All rows disappear.
- All columns disappear.
- All constraints disappear.
- The table cannot be queried anymore.

Trying to execute: **SELECT \* FROM temporary\_students;** will result in an error because the table no longer exists.

### Dropping a Database

Suppose a database named **practice\_db** was created while learning SQL. Once practice is complete, the entire database can be removed.

```
DROP DATABASE practice_db;
```

This command deletes:

- Every table
- Every record
- Every relationship
- Every view
- Every index

Everything inside the database is permanently removed.

### Real-World Example

Imagine a software company building a new payroll system. During development, the developers create several databases for testing. payroll\_test, payroll\_demo, payroll\_backup After the final system is ready, only the production database is needed. The temporary databases are removed using:

```
DROP DATABASE payroll_test;
```

```
DROP DATABASE payroll_demo;
```

This keeps the database server clean and organized.

## DROP vs TRUNCATE

Many beginners think DROP and TRUNCATE do the same thing because both remove data. However, they are completely different.

| Feature                           | TRUNCATE | DROP                       |
|-----------------------------------|----------|----------------------------|
| Removes records                   | ✓ Yes    | ✓ Yes                      |
| Removes table structure           | ✗ No     | ✓ Yes                      |
| Table exists after execution      | ✓ Yes    | ✗ No                       |
| Can insert data again immediately | ✓ Yes    | ✗ No (must recreate table) |
| Removes columns and constraints   | ✗ No     | ✓ Yes                      |

Think of it this way:

- **TRUNCATE** empties a notebook.
- **DROP** throws the notebook into the recycling bin.

## CREATE vs DROP

CREATE and DROP are opposite operations.

| CREATE                   | DROP                      |
|--------------------------|---------------------------|
| Builds a database object | Removes a database object |
| Creates tables           | Deletes tables            |
| Creates databases        | Deletes databases         |
| Starts a project         | Ends a project            |

Every database begins with CREATE and eventually, when no longer required, may end with DROP.

## Common Mistakes

Students often make these mistakes while learning DROP:

- Dropping the wrong table.
- Dropping a production database instead of a test database.
- Forgetting to create a backup before executing DROP.
- Assuming DROP can be undone.

Since DROP permanently removes database objects, it should always be used with great care.

## Best Practices

- Double-check the table or database name before executing DROP.
- Always create a backup of important data.
- Use DROP only when the object is no longer required.
- Never experiment with DROP on production databases.
- Perform testing on development or practice databases first.



## Interesting Fact

Large organizations almost never execute DROP commands directly on production systems. Before removing a table or database, they typically archive important data, review dependencies, and obtain approval from multiple team members. Many companies also automate these changes using database migration tools to reduce the risk of accidental data loss.

## DDL Commands at a Glance

| Command         | Purpose                                               |
|-----------------|-------------------------------------------------------|
| <b>CREATE</b>   | Creates a new database or table                       |
| <b>ALTER</b>    | Modifies the structure of an existing table           |
| <b>RENAME</b>   | Changes the name of a table or column                 |
| <b>TRUNCATE</b> | Removes all records while keeping the table structure |
| <b>DROP</b>     | Permanently removes a database object                 |

Together, these five commands form the foundation of **Data Definition Language (DDL)** and are responsible for building, modifying, maintaining, and removing database structures.

## Data Query Language (DQL)

So far in this book, we have learned how to build databases using Data Definition Language (DDL), manage records using Data Manipulation Language (DML), and retrieve information using the basic features of Data Query Language (DQL).

By now, you already know how to write simple SELECT statements to display data from a table. You have learned how to choose specific columns, filter records using the WHERE clause, sort data with ORDER BY, and limit the number of rows returned.

These skills are essential because they form the foundation of every SQL query. However, retrieving data is only the beginning.

Imagine a supermarket with millions of sales records. Displaying every transaction tells us very little. Managers are not interested in reading millions of rows one by one. Instead, they want answers to questions such as:

- Which products generate the highest sales?
- Which customers spend the most money?
- How many movies belong to each category?
- Which city has the largest number of customers?
- What is the average payment made by customers?
- Which employee handled the most transactions?

These questions require more than simply selecting rows. They require **data analysis**. This is where Data Query Language becomes truly powerful.

### What is Data Analysis?

Data analysis is the process of examining, organizing, summarizing, and interpreting data to discover meaningful information. Instead of looking at individual records, analysts search for patterns, trends, relationships, and insights that help organizations make better decisions.

Imagine a teacher looking at the marks of 500 students. Reading every student's marks individually would take a long time and reveal very little. Instead, the teacher might ask questions such as:

- What is the average score?
- How many students scored above 90?
- Which class performed the best?
- Which subject has the lowest average marks?

These questions summarize the data rather than simply displaying it. SQL allows us to answer similar questions for businesses, hospitals, schools, banks, online shopping websites, and countless other applications.

## **Why is Data Analysis Important?**

Modern organizations collect enormous amounts of information every second. Banks record financial transactions. Hospitals maintain patient histories. Airlines store booking details. E-commerce platforms track customer purchases. Streaming services monitor what people watch. Social media platforms record likes, comments, and shares.

Raw data by itself has little value. Its real value comes from transforming it into useful information that supports better decisions. This transformation is called **data analysis**. With SQL, we can quickly identify trends, compare performance, summarize large datasets, and answer business questions that would otherwise require hours of manual work.

## **Data Analysis in Everyday Life**

You don't have to work for a large company to understand data analysis. Think about the applications you use every day. When a music app recommends your favourite songs, it has analysed your listening history. When an online shopping website recommends products, it has analysed your previous purchases. When a food delivery app displays the most popular restaurants in your area, it has analysed thousands of customer orders. Even weather forecasting, traffic navigation, sports statistics, and health monitoring rely heavily on data analysis.

Behind many of these features, SQL queries play an important role in retrieving and summarizing information from relational databases.

## **How Does DQL Help in Data Analysis?**

Data Query Language provides a rich collection of commands and clauses that allow us to transform raw data into meaningful information. Some commands help us search for specific records. Others summarize large amounts of information.

Some combine data from multiple tables, while others perform calculations or compare records. As you continue through this chapter, you will gradually learn how SQL can answer increasingly complex business questions with just a few well-written queries.

## // Advanced Filtering Techniques

In the previous chapters, we learned how to retrieve records from a table using the SELECT statement. We also explored the WHERE clause, which allows us to filter records based on specific conditions.

For example, if a table contains information about thousands of employees, we can use a simple condition to display only employees working in the Sales department or only students who scored more than 80 marks. This works well for simple situations.

However, real-world databases often contain **millions or even billions of records**. In such cases, simple comparisons such as = or > are no longer enough. Businesses need more flexible ways to search for information.

Imagine an online shopping website with over 50 million products. Customers may remember only a part of a product's name. They might search for products starting with "Sam", ending with "Phone", or containing the word "Leather".

Similarly, a hospital may need to find patients admitted between two dates, a school may want students from specific cities, or a bank may need to identify customers whose phone numbers are missing. These situations require **advanced filtering techniques**. Advanced filtering allows us to search data more intelligently, making SQL one of the most powerful tools for analysing large datasets.

### Why is Filtering Important?

Imagine opening a spreadsheet containing **two million customer records**. Finding a single customer manually would take an enormous amount of time. Now imagine typing a simple SQL query and receiving the exact result within a fraction of a second. That is the power of filtering.

Filtering reduces unnecessary information and displays only the records that satisfy a particular condition. Instead of reading every record, SQL performs the search automatically. This not only saves time but also helps organizations make faster and more accurate decisions. Almost every application you use today relies on filtering.

When you search products on Amazon, search movies on Netflix, search contacts on your phone, or search emails in Gmail, advanced filtering techniques are working behind the scenes.

## Types of Advanced Filtering

SQL provides several powerful filtering techniques that allow us to search data in different ways. The most commonly used techniques include:

| Technique                | Purpose                  |
|--------------------------|--------------------------|
| <b>LIKE</b>              | Search using patterns    |
| <b>Wildcards (% , _)</b> | Match unknown characters |
| <b>BETWEEN</b>           | Search within a range    |
| <b>IN</b>                | Match multiple values    |
| <b>IS NULL</b>           | Find missing values      |
| <b>IS NOT NULL</b>       | Find available values    |

Each of these techniques solves a different type of problem, and together they form the foundation of advanced SQL searching.

### Pattern Matching with LIKE

Sometimes we know only a part of the information we are searching for. Suppose you are searching for a movie titled "**Academy Dinosaur**." Unfortunately, you only remember that the title starts with the word **Academy**. Searching with the equality operator (=) is impossible because you do not know the complete title.

This is where the **LIKE** operator becomes useful. The LIKE operator allows SQL to search for patterns instead of exact values. Rather than asking, "Is this value exactly equal?" LIKE asks, "Does this value follow this pattern?"

This makes LIKE one of the most commonly used operators in search engines, e-commerce websites, banking applications, and customer management systems.

#### General Syntax:

```
SELECT column_name
FROM table_name
WHERE column_name LIKE 'pattern';
```

## Wildcards

A pattern becomes powerful when combined with **wildcards**. Wildcards are special symbols that represent unknown characters. SQL mainly provides two wildcard characters.

| Wildcard | Meaning                                  |
|----------|------------------------------------------|
| %        | Represents zero, one, or many characters |
| _        | Represents exactly one character         |

Wildcards transform LIKE into an extremely flexible searching tool.

### Percent (%) Wildcard

The **percent (%)** wildcard represents **any number of characters**, including zero characters. Suppose a bookstore stores thousands of book titles.

To display every title beginning with the letter **A**:

```
SELECT title
FROM books
WHERE title LIKE 'A%';
```

Possible results:

- Artificial Intelligence
- Algorithms
- Astronomy Today
- Adventure World

The percent sign tells SQL: "The title must begin with A, but anything can come after it."

To search for titles ending with **ing**:

```
SELECT title
FROM books
WHERE title LIKE '%ing';
```

Results might include:

- Programming
- Learning
- Cooking
- Reading

To search for titles containing the word **Data** anywhere:

```
SELECT title
FROM books
WHERE title LIKE '%Data%';
```

Possible results:

```
Data Science
Big Data Analytics
Introduction to Data Mining
```

### **Underscore (\_) Wildcard**

Unlike %, the underscore represents **exactly one character**. Suppose we want names where the second letter is **A**.

```
SELECT first_name
FROM customer
WHERE first_name LIKE '_A%';
```

Possible results:

```
James
David
Karen
```

The first character may be anything, the second must be **A**, and any number of characters may follow.

Suppose we want movie titles containing exactly five letters.

```
SELECT title
FROM film
WHERE title LIKE '_____';
```

Each underscore represents one character. Only titles with exactly five letters are returned.

### **Real-World Applications of LIKE**

Pattern matching is used almost everywhere.

- Google suggests search results based on partially typed keywords.
- E-commerce websites find products using incomplete product names.
- Banking applications search customers using partial account names.
- Hospitals search patient names without requiring the complete spelling.
- Social media platforms locate usernames using partial matches.

Although these applications use more advanced technologies in addition to SQL, pattern matching with LIKE remains one of the fundamental techniques used in database searching.

## Searching Within a Range using BETWEEN

Many searches involve ranges rather than exact values. A school may need students whose marks are between **70 and 90**. A hotel may need bookings between two dates. An online store may search products priced between **₹500 and ₹2000**.

Instead of writing multiple comparison operators, SQL provides the **BETWEEN** operator.

### General Syntax:

```
SELECT column_name  
FROM table_name  
WHERE column_name BETWEEN value1 AND value2;
```

### Example:

Display students scoring between 70 and 90 marks.

```
SELECT *  
FROM students  
WHERE marks BETWEEN 70 AND 90;
```

This query returns students whose marks are **70, 71, 72... up to 90**. An important fact about BETWEEN is that **both boundary values are included**. This means the query includes students who scored exactly **70** and exactly **90**.

BETWEEN can also be used with dates.

```
SELECT *  
FROM orders  
WHERE order_date  
BETWEEN '2025-01-01'  
AND '2025-12-31';
```

This displays orders placed during the year 2025.

## Matching Multiple Values with IN

Sometimes we need to search for several possible values. Suppose a company wants employees from:

- Mumbai
- Delhi
- Pune

Without IN, the query becomes lengthy. WHERE city='Mumbai' OR city='Delhi' OR city='Pune'

The IN operator makes the query cleaner and easier to read.

### General Syntax:

```
SELECT column_name  
FROM table_name  
WHERE column_name IN(value1,value2,value3);
```



**Example:**

```
SELECT *
FROM employees
WHERE city IN
('Mumbai','Delhi','Pune');
```

Only employees from these three cities are displayed.

The IN operator is particularly useful when filtering against a predefined list of values, improving both readability and maintainability of SQL queries.

**Understanding NULL Values**

One of the most misunderstood concepts in SQL is **NULL**. Many beginners think NULL means:

- zero,
- blank,
- or an empty string.

This is incorrect.

A NULL value simply means: The value is unknown, missing, or has not been entered.

For example,

| Name  | Email           |
|-------|-----------------|
| Rahul | rahul@gmail.com |
| Priya | NULL            |

This does not mean Priya has an empty email address. It means the email address is currently unknown or unavailable.

**Finding Missing Information using IS NULL:**

Suppose a company wants to find customers whose email addresses have not yet been recorded.

```
SELECT *
FROM customers
WHERE email IS NULL;
```

This query returns only those customers whose email values are missing.

**Finding Available Information using IS NOT NULL:**

To display customers whose email addresses are available:

```
SELECT *
FROM customers
WHERE email IS NOT NULL;
```

This excludes every record with missing values.

## Why NULL is Important

Handling NULL values correctly is extremely important in data analysis. Imagine a hospital analyzing patient blood groups.

If some blood groups are missing and treated as ordinary values, the analysis may produce incorrect statistics. Similarly, businesses often examine NULL values to identify incomplete forms, missing customer details, or records that require attention.

Proper handling of NULL values improves the accuracy and reliability of reports.

## Interesting Facts

- Pattern matching using LIKE is one of the oldest and most widely supported SQL features, making it available in almost every relational database system.
- Large e-commerce websites often combine SQL filtering with specialized search technologies to deliver faster and smarter search results.
- The BETWEEN operator is **inclusive**, meaning both the starting and ending values are part of the result set.
- The IN operator can greatly simplify queries that would otherwise require many OR conditions.
- NULL represents the absence of a value, not the value zero or an empty string. Treating these as the same can lead to incorrect reports and analyses.

## // Aggregate Functions

Until now, our SQL queries have focused on retrieving individual records from a table. We have learned how to search for data, filter rows, sort results, and display only the information we need. While these techniques are useful, businesses are rarely interested in reading thousands or millions of individual records one by one.

Imagine you work for an online shopping company. The company's database contains over ten million orders. Will the manager read every single order to understand the company's performance? Of course not.

Instead, the manager asks questions such as:

- How many orders were placed today?
- What is the total revenue this month?
- What is the average order value?
- Which product has the highest price?
- What is the cheapest product available?

Notice something interesting. These questions do not ask for individual rows. Instead, they ask for a **summary of the data**. This process of converting many records into meaningful information is called **aggregation**. SQL provides a special set of functions for performing this task. These are known as **Aggregate Functions**.

Aggregate functions examine multiple rows and produce a **single summarized result**, allowing analysts and decision-makers to understand large datasets quickly and efficiently.

### What Does "Aggregate" Mean?

The word **aggregate** means **to collect, combine, or summarize many individual items into one meaningful result**. Imagine a teacher checking the marks of 100 students.

Instead of reading every student's marks individually, the teacher calculates:

- the total marks,
- the average marks,
- the highest score,
- the lowest score,
- and the number of students who appeared for the exam.

These values summarize the performance of the entire class.

Similarly, SQL aggregate functions summarize thousands or even millions of records into useful statistics. Instead of displaying every row, they answer important business questions in a single query.

## Why Are Aggregate Functions Important?

Modern organizations generate enormous amounts of data every day. Banks record millions of financial transactions. Hospitals maintain patient records. Airlines store ticket bookings. E-commerce websites process thousands of purchases every minute.

Reading every individual record would be impossible. Aggregate functions allow organizations to quickly understand what is happening by producing summaries instead of raw data. These summaries help managers monitor performance, analysts identify trends, and executives make informed business decisions.

Without aggregate functions, meaningful data analysis would become slow, difficult, and time-consuming.

## Types of Aggregate Functions

SQL provides several built-in aggregate functions. Each function performs a different type of calculation.

| Function       | Purpose                      |
|----------------|------------------------------|
| <b>COUNT()</b> | Counts the number of records |
| <b>SUM()</b>   | Calculates the total value   |
| <b>AVG()</b>   | Finds the average value      |
| <b>MIN()</b>   | Finds the smallest value     |
| <b>MAX()</b>   | Finds the largest value      |

Although each function performs a different calculation, they all share one common goal: They summarize multiple rows into a single result.

### COUNT()

The **COUNT()** function counts how many records satisfy a given condition. Instead of displaying every row, COUNT simply tells us **how many rows exist**. Imagine a school principal asking, "How many students are enrolled this year?" There is no need to display every student's record. The principal only needs the total number. This is exactly what COUNT() does.

General Syntax

```
SELECT COUNT(column_name)
FROM table_name;
```

To count every row in a table:

```
SELECT COUNT(*)
FROM students;
```

Example:

Suppose a students table contains 500 records.

```
SELECT COUNT(*)  
FROM students;
```

Output: 500

COUNT() is commonly used to determine:

- Total customers
- Total employees
- Total products
- Total orders
- Total movies

### **COUNT(column\_name)**

If a column contains NULL values:

```
SELECT COUNT(email)  
FROM customers;
```

Only rows where **email is not NULL** are counted. This is an important difference between: COUNT(\*) and COUNT(column\_name)

### **SUM()**

The **SUM()** function calculates the total of all numeric values in a column. Imagine the finance department wants to know the company's total sales for the month. Instead of adding thousands of invoices manually, SQL calculates the result instantly.

General Syntax

```
SELECT SUM(column_name)  
FROM table_name;
```

Example

```
SELECT SUM(amount)  
FROM payment;
```

Output: 67416.51

SUM() is commonly used for:

- Total revenue
- Total salary
- Total sales
- Total expenses
- Total quantity sold

It only works with **numeric data types**.

## AVG()

The **AVG()** function calculates the arithmetic mean of numeric values. Average values help organizations understand typical performance rather than focusing on individual records. Imagine a university wants to know the average marks scored by students. Instead of manually calculating hundreds of marks, AVG() produces the answer immediately.

General Syntax

```
SELECT AVG(column_name)
FROM table_name;
```

Example

```
SELECT AVG(salary)
FROM employees;
```

Output: 42500.75

Businesses frequently use AVG() to calculate:

- Average salary
- Average product price
- Average customer spending
- Average movie length
- Average examination score

## MIN()

The **MIN()** function returns the smallest value from a column. Imagine an airline wants to know the lowest ticket price available for a particular route. Instead of sorting thousands of records, MIN() immediately identifies the smallest value.

General Syntax:

```
SELECT MIN(column_name)
FROM table_name;
```

Example:

```
SELECT MIN(price)
FROM products;
```

Output: 99

MIN() is often used to find:

- Lowest salary
- Cheapest product
- Earliest date
- Youngest age
- Minimum temperature

## MAX()

The **MAX()** function returns the largest value in a column. Imagine a movie streaming company wanting to identify its longest movie. Rather than checking every movie manually, MAX() instantly provides the answer.

General Syntax:

```
SELECT MAX(column_name)
FROM table_name;
```

Example

```
SELECT MAX(length)
FROM film;
```

Output: 185

MAX() is frequently used for:

- Highest salary
- Most expensive product
- Largest payment
- Highest marks
- Longest movie

## How Aggregate Functions Handle NULL Values

A common question among beginners is: What happens if some records contain NULL values? Most aggregate functions ignore NULL values.

Suppose a table contains:

| Marks |
|-------|
| 80    |
| 90    |
| NULL  |
| 70    |

Then:

```
SELECT AVG(marks)
FROM students;
```

The average is calculated using:  $80 + 90 + 70$  The NULL value is ignored because it represents an unknown value, not zero.

Similarly:

- SUM() ignores NULL values.
- AVG() ignores NULL values.
- MIN() ignores NULL values.
- MAX() ignores NULL values.
- COUNT(column) counts only non-NULL values.
- COUNT(\*) counts every row.

Understanding this behaviour is essential for producing accurate reports.

## Real-World Applications

Aggregate functions are used in almost every industry.

An online shopping company calculates daily sales using SUM(). A hospital counts the number of admitted patients using COUNT(). A school determines the average examination score using AVG(). A bank identifies the highest transaction of the day using MAX(). A weather department records the minimum temperature using MIN().

Although the industries are different, the purpose remains the same: summarize large amounts of information into meaningful insights.

## Interesting Facts

- Aggregate functions are among the most frequently used SQL features in business reporting and dashboards.
- Many charts and graphs displayed in tools such as Power BI, Tableau, and Excel are built using aggregate calculations performed by SQL.
- Database engines are highly optimized for aggregate functions, allowing them to summarize millions of records within seconds.
- Nearly every business report begins with one or more aggregate functions before moving on to grouping and visualization.

## Aggregate Function Summary

| Function | Returns           | Example Use     |
|----------|-------------------|-----------------|
| COUNT()  | Number of records | Total customers |
| SUM()    | Total value       | Total sales     |
| AVG()    | Average value     | Average salary  |
| MIN()    | Smallest value    | Lowest price    |
| MAX()    | Largest value     | Highest marks   |



## // GROUP BY and HAVING

In the previous chapter, we learned about **Aggregate Functions** such as COUNT(), SUM(), AVG(), MIN(), and MAX(). These functions helped us summarize large amounts of data into a single result. For example, suppose a company wants to know its total sales.

```
SELECT SUM(amount)
FROM payment;
```

The result might be: 67416.51

This tells us the company's total revenue. While this information is useful, businesses often need much more detailed insights. Imagine you are the manager of a supermarket chain operating across several cities.

Knowing the company's total revenue is helpful, but it doesn't answer questions like:

- Which city generated the highest sales?
- Which store served the most customers?
- Which department sold the most products?
- Which movie rating has the largest number of films?
- Which employee handled the highest number of transactions?

These questions require the data to be **divided into groups** before calculations are performed. This is where the **GROUP BY** clause becomes one of the most powerful features of SQL.

### What is GROUP BY?

The GROUP BY clause divides rows that share a common value into separate groups. Instead of performing one calculation on the entire table, SQL performs the same calculation **independently for each group**. Imagine a teacher who has the marks of 300 students from six different classes.

Calculating the average marks of all 300 students gives one overall average. However, if the teacher wants to compare the performance of each class, the students must first be grouped according to their class.

Once grouped, the average can be calculated separately for every class. SQL works in exactly the same way. It first creates groups, then performs calculations on each group individually.

## Why Do We Need GROUP BY?

Suppose an online movie rental company stores information about thousands of films. If we execute:

```
SELECT COUNT(*)
FROM film;
```

We receive the total number of movies. But what if the manager asks: "How many movies belong to each rating?"

Now a single number is no longer enough. The movies must first be grouped according to their rating. Only then can SQL count the number of movies in each category. This ability to summarize data by category is the primary purpose of the GROUP BY clause.

General Syntax:

```
SELECT column_name,
aggregate_function(column_name)
FROM table_name
GROUP BY column_name;
```

The column listed in the GROUP BY clause becomes the basis for forming groups. The aggregate function is then applied separately to each group.

### Example 1: Counting Students in Each Class

Suppose a school maintains the following table.

| Student | Class |
|---------|-------|
| Rahul   | A     |
| Priya   | A     |
| Aman    | B     |
| Sneha   | B     |
| Karan   | B     |

The query:

```
SELECT class,
COUNT(*)
FROM students
GROUP BY class;
```

produces:

| Class | Students |
|-------|----------|
| A     | 2        |
| B     | 3        |

Instead of returning one total count, SQL creates separate groups and counts the students in each group.

**Example 2: Average Salary by Department**

Suppose an employee table contains several departments.

```
SELECT department,
AVG(salary)
FROM employees
GROUP BY department;
```

Possible output:

| Department | Average Salary |
|------------|----------------|
| HR         | 42000          |
| Sales      | 39000          |
| IT         | 61000          |
| Finance    | 54000          |

Notice how the average salary is calculated independently for every department.

**GROUP BY with Different Aggregate Functions**

Almost every aggregate function can be combined with GROUP BY.

COUNT() Counts records in each group.

SUM() Calculates the total value for each group.

AVG() Calculates the average value for each group.

MIN() Finds the smallest value in each group.

MAX() Finds the largest value in each group.

This combination makes SQL extremely useful for generating business reports.

**Real-World Applications of GROUP BY**

Every day, businesses generate reports using GROUP BY. A bank groups transactions by branch to calculate total deposits. A hospital groups patients by department to determine patient counts. An airline groups flights by destination to analyze passenger traffic. An online shopping platform groups orders by product category to calculate sales. A university groups students by course to determine average examination scores.

Without GROUP BY, creating these reports would require enormous manual effort.

**Understanding HAVING**

Once the data has been grouped, another question often arises. Suppose a company has sales data for every branch. The manager is interested only in branches that generated more than ₹10,00,000 in sales. Can we use the WHERE clause?

No. The WHERE clause filters **rows before grouping**. However, the manager wants to filter **groups after aggregation**. This is why SQL provides the **HAVING** clause.

## What is HAVING?

The HAVING clause filters the groups created by GROUP BY. Think of it as the WHERE clause for grouped results. The sequence is:

1. SQL reads the table.
2. SQL creates groups.
3. SQL performs aggregate calculations.
4. SQL applies the HAVING condition.

General Syntax:

```
SELECT column_name,
       aggregate_function(column_name)
FROM table_name
GROUP BY column_name
HAVING condition;
```

Example:

Suppose a movie database contains films with different ratings. The company wants to display only ratings that contain more than 200 movies.

```
SELECT rating,
       COUNT(*)
FROM film
GROUP BY rating
HAVING COUNT(*) > 200;
```

Possible output:

| Rating | Movies |
|--------|--------|
| PG     | 209    |
| PG-13  | 223    |

Ratings with fewer than 200 movies are excluded.

## WHERE vs HAVING

Students often confuse these two clauses. The difference is simple.

| WHERE                          | HAVING                            |
|--------------------------------|-----------------------------------|
| Filters individual rows        | Filters groups                    |
| Applied before GROUP BY        | Applied after GROUP BY            |
| Cannot use aggregate functions | Commonly uses aggregate functions |

Imagine a school. The principal first selects students from Class A. That is similar to WHERE.

After calculating the average marks of every class, the principal keeps only classes whose average exceeds 80. That is similar to HAVING.

### Using WHERE and HAVING Together

Both clauses can appear in the same query. Suppose a company wants to calculate average salaries **only for permanent employees**, and then display departments whose average salary exceeds ₹50,000.

```
SELECT department,
AVG(salary)
FROM employees
WHERE employment_type='Permanent'
GROUP BY department
HAVING AVG(salary)>50000;
```

Notice the order. First, WHERE removes temporary employees. Then GROUP BY creates departments. Finally, HAVING filters the grouped results.

### Real-World Example

Imagine a food delivery company. The company stores millions of orders. The manager wants to know:

- Total orders placed in each city.
- But display only cities having more than 5,000 orders.

SQL first groups all orders according to city. Then it counts the orders in every city. Finally, HAVING removes cities that do not satisfy the required condition. This approach helps managers focus only on the most important information.

## Common Mistakes

Beginners frequently make these mistakes:

- Using WHERE COUNT(\*) > 10 instead of HAVING COUNT(\*) > 10.
- Forgetting to include grouped columns in the GROUP BY clause.
- Selecting non-aggregated columns that are not grouped.
- Confusing row filtering with group filtering.

Understanding the difference between WHERE and HAVING is one of the most important skills in SQL.

## Interesting Facts

- Most business intelligence reports rely heavily on GROUP BY to summarize large datasets.
- Sales dashboards, financial summaries, customer reports, and inventory analyses often begin with grouped SQL queries.
- Even modern visualization tools such as **Power BI**, **Tableau**, and **Looker** frequently generate GROUP BY queries behind the scenes when creating charts and dashboards.
- Database systems are optimized to perform grouping efficiently, allowing millions of records to be summarized in seconds.

## GROUP BY and HAVING at a Glance

| Clause   | Purpose                                         |
|----------|-------------------------------------------------|
| GROUP BY | Divides rows into groups based on common values |
| HAVING   | Filters the grouped results after aggregation   |

Together, these clauses transform raw records into meaningful summaries, making them essential tools for reporting and data analysis.

## // JOINS - Combining Data from Multiple Tables

Until now, every SQL query we have written has retrieved information from a **single table**. We searched students from the **students** table, employees from the **employees** table, movies from the **film** table, and customers from the **customer** table. This approach works well for small databases. However, real-world databases rarely store all information in one giant table. Imagine an online shopping website. Customer information is stored in one table. Orders are stored in another. Products are stored in a third. Payments are stored in a fourth. Shipping details are stored in yet another table.

Why not keep everything in one table? Because it would create enormous amounts of duplicate data, waste storage space, slow down queries, and make the database extremely difficult to maintain. Instead, relational databases divide information into multiple related tables. This design is called **normalization**, which we studied earlier. But dividing information creates a new challenge. How do we retrieve information spread across multiple tables? Suppose a manager asks, "Show the customer's name, the product they purchased, the payment amount, and the shipping city." None of these details exist in a single table.

SQL solves this problem using **JOINS**. A JOIN combines rows from two or more tables based on a common column, allowing us to retrieve related information as if it were stored together. Without JOINS, relational databases would lose much of their power.

### Why Do We Need JOINS?

Imagine a university database. Instead of storing everything in one table, the information is organized like this.

Students

| Student ID | Name  |
|------------|-------|
| 101        | Rahul |
| 102        | Priya |
| 103        | Aman  |

Courses

| Course ID | Course |
|-----------|--------|
| C01       | Python |
| C02       | SQL    |

Enrollments

| Student ID | Course ID |
|------------|-----------|
| 101        | C01       |
| 101        | C02       |
| 102        | C02       |

Now suppose someone asks,

### Which course is Rahul studying?

The answer cannot be found in a single table.

SQL must combine information from multiple tables.

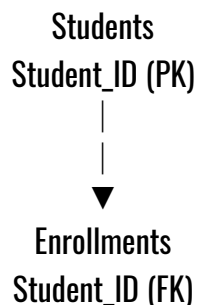
That combination is performed using a JOIN.

### What is a JOIN?

A JOIN is an SQL operation that combines rows from two or more tables using a related column. The related column is usually:

- Primary Key
- Foreign Key

For example,



The matching values act like a bridge between the tables. Whenever SQL finds matching values, it combines the related rows into a single result.

### Real-World Analogy

Imagine a library. One register stores: Book ID, Book Name. Another register stores: Book ID, Student Name, Issue Date

If the librarian wants to know "Which student borrowed which book?" the two registers must be compared using **Book ID**. JOIN performs exactly this comparison automatically.

### Types of SQL JOINS

SQL provides several kinds of JOINS. Each serves a different purpose.

| JOIN       | Purpose                                  |
|------------|------------------------------------------|
| INNER JOIN | Returns only matching records            |
| LEFT JOIN  | Returns all records from the left table  |
| RIGHT JOIN | Returns all records from the right table |
| CROSS JOIN | Returns every possible combination       |
| SELF JOIN  | Joins a table with itself                |

Each type solves a different business problem.



## INNER JOIN

The INNER JOIN returns only those rows where matching values exist in both tables. If a record exists in one table but not in the other, it is ignored.

Visual Representation

| Customers | Orders     |
|-----------|------------|
| 101 Rahul | 101 Laptop |
| 102 Priya | 103 Phone  |
| 103 Aman  | 104 Mouse  |

### INNER JOIN



101 Rahul Laptop  
103 Aman Phone

Notice that:

- Rahul appears because Customer ID 101 exists in both tables.
- Aman appears because Customer ID 103 exists in both tables.
- Priya has no order.
- Customer 104 has no customer record.

Both are excluded. Only matching records remain.

Syntax:

```
SELECT *
FROM customer
INNER JOIN payment
ON customer.customer_id =
payment.customer_id;
```

Real-World Uses:

INNER JOIN is commonly used to:

- Display customer purchases
- Show employee departments
- Display movie languages
- Display student courses
- Generate invoices

## LEFT JOIN

Sometimes we want every record from one table, even if no matching record exists. This is where LEFT JOIN becomes useful.

LEFT JOIN returns:

- every record from the left table
- matching records from the right table

If no match exists, SQL fills the missing columns with NULL.

Diagram

| Customers | Orders |
|-----------|--------|
| 101 Rahul | Laptop |
| 102 Priya |        |
| 103 Aman  | Phone  |
| LEFT JOIN |        |
| 101 Rahul | Laptop |
| 102 Priya | NULL   |
| 103 Aman  | Phone  |

Priya appears even though she has never placed an order.

Syntax:

```
SELECT *
FROM customer
LEFT JOIN payment
ON customer.customer_id =
payment.customer_id;
```

Real Uses:

LEFT JOIN is used for:

- Customers who never ordered
- Employees without projects
- Students without attendance
- Products never sold

Finding missing information is one of the biggest uses of LEFT JOIN.

## RIGHT JOIN

RIGHT JOIN is simply the opposite of LEFT JOIN.

It returns:

- every record from the right table
- matching rows from the left table

Diagram

| Customers  | Orders |
|------------|--------|
| 101 Rahul  | Laptop |
| 102 Priya  | Mouse  |
| RIGHT JOIN |        |
| Rahul      | Laptop |
| NULL       | Mouse  |

The order still appears even though there is no customer.

Syntax:

```
SELECT *  
FROM customer  
RIGHT JOIN payment  
ON customer.customer_id =  
payment.customer_id;
```

Fact

Many SQL developers rarely use RIGHT JOIN because the same result can usually be achieved by swapping the table positions and using a LEFT JOIN. For this reason, LEFT JOIN is generally more common in real-world projects.

## **CROSS JOIN**

Unlike other joins, CROSS JOIN does not require matching columns. Instead, it produces every possible combination.

Example:

Colors: Red, Blue

Sizes: Small, Large

Result: Red Small, Red Large, Blue Small, Blue Large

Every color combines with every size.

Syntax:

```
SELECT *  
FROM colors  
CROSS JOIN sizes;
```

Real Uses

- Product variations
- Seating arrangements
- Timetable generation
- Test data generation
- Mathematical combinations

## SELF JOIN

A SELF JOIN joins a table with itself. This may sound strange at first, but it is extremely useful. Imagine an employee table.

| Employee | Manager ID |
|----------|------------|
| Rahul    | 1          |
| Priya    | 1          |
| Aman     | 2          |

The manager is also an employee. Therefore, the employee table must be joined with itself.

Diagram



Syntax:

```

SELECT A.employee_name, B.employee_name
FROM employees A
JOIN employees B
ON A.manager_id = B.employee_id;
  
```

Notice the aliases. A and B represent two logical copies of the same table.

Real Uses

- Employee Manager hierarchy
- Family trees
- Organizational charts
- Category Parent Category relationships

**Choosing the Correct JOIN:**

| Situation                           | JOIN       |
|-------------------------------------|------------|
| Only matching records               | INNER JOIN |
| Keep all records from left table    | LEFT JOIN  |
| Keep all records from right table   | RIGHT JOIN |
| Generate every possible combination | CROSS JOIN |
| Compare rows within the same table  | SELF JOIN  |

**Performance Tips**

As databases grow larger, JOIN operations become more expensive because SQL must compare rows across multiple tables. Database systems use indexes, query optimizers, and efficient algorithms to make these comparisons as fast as possible. Proper indexing of primary keys and foreign keys can dramatically improve JOIN performance, especially when working with millions of records.

**Common Mistakes**

Students often make these mistakes while learning JOINS:

- Forgetting the ON condition, which can produce incorrect results or an unintended Cartesian product.
- Joining tables using unrelated columns instead of matching keys.
- Confusing LEFT JOIN with RIGHT JOIN.
- Assuming INNER JOIN returns all rows, when it only returns matching records.
- Forgetting to use table aliases when performing a SELF JOIN.

Understanding the relationships between tables is just as important as understanding the JOIN syntax itself.

**Interesting Facts**

- JOINS are one of the defining features of **relational databases**, which is why they are called *relational*.
- Large organizations such as banks, airlines, hospitals, and e-commerce platforms execute millions of JOIN queries every day to generate reports and support applications.
- Most dashboards built with **Power BI**, **Tableau**, and **Looker** rely on JOINS to combine information from multiple tables before creating charts and visualizations.
- In modern database systems, query optimizers can automatically choose highly efficient algorithms, such as nested loop joins, hash joins, or merge joins, depending on the data and indexes.

**JOIN Summary**

| <b>JOIN Type</b>  | <b>Returns</b>                                                |
|-------------------|---------------------------------------------------------------|
| <b>INNER JOIN</b> | Only matching rows from both tables                           |
| <b>LEFT JOIN</b>  | All rows from the left table and matching rows from the right |
| <b>RIGHT JOIN</b> | All rows from the right table and matching rows from the left |
| <b>CROSS JOIN</b> | Every possible combination of rows                            |
| <b>SELF JOIN</b>  | A table joined with itself                                    |

## // Subqueries (Nested Queries)

As we have progressed through SQL, our queries have become increasingly powerful. We started by retrieving data using the SELECT statement, filtered records using various conditions, summarized data with aggregate functions, grouped records using GROUP BY, and combined multiple tables using different types of JOINS.

While these techniques solve many database problems, there are situations where one query depends on the result of another query. Imagine a company manager asking,

- Which employee earns the highest salary?
- Which customers have made payments greater than the average payment?
- Which movies are longer than the average movie length?
- Which students scored above the class average?

Notice something interesting. Before answering these questions, SQL must first calculate another value. For example, To find employees earning the highest salary, SQL must first determine what the highest salary actually is. Similarly, before finding movies longer than the average length, SQL must first calculate the average movie duration. These are situations where one query depends on another query.

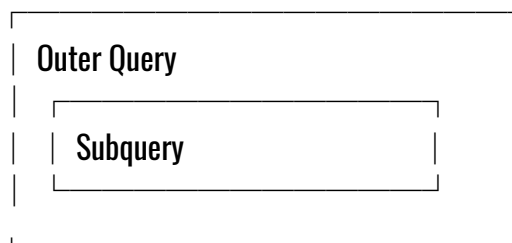
SQL solves such problems using Subqueries, also known as Nested Queries. Subqueries allow one SQL statement to be placed inside another SQL statement, making it possible to solve complex problems in a simple and organized way.

### What is a Subquery?

A Subquery is a query written inside another SQL query. Because one query is placed within another, it is also called a Nested Query. The inner query executes first and produces a result. The outer query then uses that result to complete its task. In simple words, A Subquery is a query inside another query whose result is used by the outer query.

### Why is it Called a Nested Query?

The word nested means *placed inside something else*. Think about Russian Matryoshka dolls. Each doll contains another smaller doll inside it.

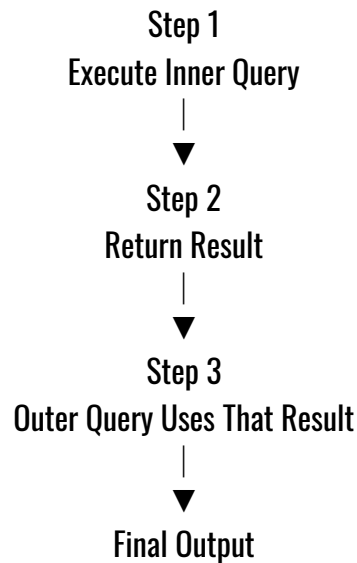


Similarly, in SQL, the main query contains another query inside it. The inner query works first. Its output becomes the input for the outer query. This layered structure is why it is called a Nested Query.



## How Does a Subquery Work?

The execution of a subquery follows a simple sequence.



The user writes only one SQL statement, but internally SQL executes the inner query before executing the outer query.

### General Syntax:

```
SELECT column_name
FROM table_name
WHERE column_name operator (
    SELECT column_name
    FROM another_table
);
```

The query enclosed within parentheses is the **Subquery**. The query outside the parentheses is the **Outer Query**.

### Simple Example

Suppose an employee table stores salary information. The manager wants to find employees earning the highest salary. Instead of manually finding the highest salary first, SQL can perform both tasks automatically.

```
SELECT employee_name
FROM employees
WHERE salary = (
    SELECT MAX(salary)
    FROM employees
);
```

## How SQL Executes This Query

First, SQL executes: `SELECT MAX(salary) FROM employees;`

Suppose the result is: 95000

Now SQL replaces the subquery with this value.

The outer query becomes: `SELECT employee_name FROM employees WHERE salary = 95000;`

Finally, the employee earning ₹95,000 is displayed.

## Why Do We Need Subqueries?

Subqueries make SQL more intelligent. Without them, users would often need to execute two or three separate queries manually. For example,

First query:

```
SELECT AVG(marks)
FROM students;
```

Suppose the answer is: 76

Second query:

```
SELECT *
FROM students
WHERE marks > 76;
```

This requires two separate steps. A subquery combines both operations into one SQL statement.

```
SELECT *
FROM students
WHERE marks > (
    SELECT AVG(marks)
    FROM students
);
```

The database performs both calculations automatically. This makes queries shorter, easier to maintain, and less prone to errors.

## Types of Subqueries

SQL supports several kinds of subqueries depending on the type of result they return.

| Type                     | Returns                                       |
|--------------------------|-----------------------------------------------|
| Scalar Subquery          | One value                                     |
| Multiple Row Subquery    | Multiple rows                                 |
| Multiple Column Subquery | Multiple columns                              |
| Correlated Subquery      | Executes once for each row of the outer query |

Each type is useful in different situations.

## Scalar Subquery

A **Scalar Subquery** returns only **one value**. It is the simplest and most commonly used type of subquery.

### Example:

Find movies longer than the average movie length.

```
SELECT title
FROM film
WHERE length > (
    SELECT AVG(length)
    FROM film
);
```

The inner query returns one value. Suppose: 115 The outer query then displays every movie longer than 115 minutes.

## Multiple Row Subquery

Sometimes the inner query returns several rows instead of one. In such cases, operators such as IN, ANY, or ALL are commonly used.

### Example:

Display customers living in cities where stores exist.

```
SELECT customer_name
FROM customer
WHERE city_id IN (
    SELECT city_id
    FROM store
);
```

The inner query may return many city IDs. The outer query compares each customer against all of them.

## Multiple Column Subquery

Sometimes the inner query returns more than one column. These are called **Multiple Column Subqueries**.

### Example:

Suppose we want employees having the same department and salary as a particular employee.

```
SELECT employee_name
FROM employees
WHERE (department, salary) = (
    SELECT department, salary
    FROM employees
    WHERE employee_id=5
);
```

The subquery returns two values together. The outer query compares both columns simultaneously.

## Correlated Subquery

A **Correlated Subquery** is more advanced. Unlike ordinary subqueries, it cannot execute independently. Instead, it executes once for **every row** processed by the outer query.

Think of it as a conversation between the outer query and the inner query. The outer query sends one row at a time. The inner query processes that row and returns a result. This process repeats until every row has been examined.

### Example:

Find employees earning more than the average salary in their own department.

```
SELECT employee_name, department, salary
FROM employees e1
WHERE salary > (
    SELECT AVG(salary)
    FROM employees e2
    WHERE e1.department = e2.department
);
```

Here, the inner query depends on the current department from the outer query. Because of this dependency, SQL must execute the subquery repeatedly.

Correlated subqueries are extremely powerful but are usually slower than ordinary subqueries because they run many times.

## Subquery vs JOIN

Students often wonder whether they should use a JOIN or a Subquery. The answer depends on the problem.

| JOIN                            | Subquery                             |
|---------------------------------|--------------------------------------|
| Combines related tables         | Uses the result of another query     |
| Retrieves related information   | Solves dependent calculations        |
| Often faster for combining data | Often simpler for logical conditions |
| Used for relationships          | Used for intermediate results        |

Both techniques are important, and experienced SQL developers often use them together.

## Real-World Applications

Subqueries are used in many practical situations. A bank finds customers whose account balance is above the average balance. A hospital identifies patients whose treatment cost exceeds the average treatment cost. An online shopping website lists products priced higher than the average price. A university finds students who scored above the class average. A movie streaming service recommends movies with ratings higher than the average rating.

These tasks would require multiple manual steps without subqueries.

## Common Mistakes

Beginners often encounter these errors:

- Forgetting to enclose the subquery within parentheses.
- Using = when the subquery returns multiple rows instead of using IN.
- Assuming the outer query executes first. In most cases, the inner query executes first.
- Confusing JOINS with subqueries. JOINS combine tables, while subqueries use the result of another query.
- Writing correlated subqueries when a simpler ordinary subquery or JOIN would be sufficient.

Understanding the type of result returned by a subquery helps avoid many of these mistakes.

## Interesting Facts

- A subquery can often replace multiple separate SQL statements with a single, readable query.
- Many SQL optimizers automatically transform certain subqueries into efficient JOIN operations behind the scenes.
- Correlated subqueries are powerful but can be slower on very large datasets because they may execute once for every row processed by the outer query.
- Complex reports in business intelligence systems often combine aggregate functions, GROUP BY, HAVING, JOINS, and subqueries within the same SQL statement.

**Summary of Subqueries**

| Type                     | Description                | Common Operators            |
|--------------------------|----------------------------|-----------------------------|
| Scalar Subquery          | Returns one value          | =, <, >                     |
| Multiple Row Subquery    | Returns many rows          | IN, ANY, ALL                |
| Multiple Column Subquery | Returns multiple columns   | Tuple comparison            |
| Correlated Subquery      | Depends on the outer query | Executes for each outer row |

# SQL Practice

## Customers and Products Analysis Using SQL

### Story:

Congratulations! You have recently been hired as a **Junior Data Analyst** at **RetailKart Analytics Pvt. Ltd.**, a fast-growing retail analytics company that helps businesses understand their customers and improve sales through data.

RetailKart works with several online shopping companies that sell thousands of products every day. Every purchase made by a customer generates valuable information such as customer details, products purchased, payment amounts, quantities, order dates, and delivery locations.

Over the years, the company has collected a large amount of business data. Although this data contains valuable insights, it is currently just a collection of records stored inside a relational database. The management team wants to transform this raw data into meaningful information that can answer important business questions.

As the newest member of the Data Analytics team, your responsibility is to explore the database, understand its structure, and generate reports that will help managers make better business decisions. Throughout this practice, you will use SQL to analyse customer behaviour, product performance, sales trends, and business operations using real-world datasets.

### Background:

RetailKart has received transaction data from one of its partner e-commerce companies. The data has been exported from the company's operational database and shared with the analytics team for further analysis. Instead of manually creating hundreds of records, the company has already provided the datasets in CSV format.

Your first task is to import these datasets into MySQL and understand the information available before beginning any analysis. Only after understanding the business data can meaningful reports be generated. Remember, **A good data analyst always understands the data before writing queries.**

**Objectives:**

During this project, you will gradually solve a series of business problems using SQL. Instead of learning isolated SQL commands, you will apply everything learned in this book to answer real business questions.

Throughout the project, you will:

- Import real-world datasets into MySQL.
- Create a relational database using the imported tables.
- Explore the structure of the database.
- Understand relationships between different tables.
- Identify important business entities such as customers, products, orders, and payments.
- Retrieve meaningful information using SQL queries.
- Filter and sort business data.
- Analyze customer purchasing behavior.
- Calculate sales statistics using aggregate functions.
- Generate business reports using GROUP BY and HAVING.
- Combine multiple tables using different types of JOINS.
- Solve complex analytical problems using subqueries.
- Present business insights that help management make informed decisions.

By completing these tasks, you will experience a simplified version of the workflow followed by professional data analysts.

**Roadmap:**

This mini project is divided into four practical programs. Each program builds upon the previous one, gradually increasing in complexity.

**Program 47: Database Setup and Data Exploration**

In this program, you will begin by preparing the working environment for analysis.

You will create a new MySQL database, import the provided datasets, examine the tables, understand their relationships, and explore the available columns and records. Instead of immediately writing analytical queries, your goal will be to become familiar with the business data and understand what information is available for future analysis.

By the end of this program, the database will be fully prepared and ready for reporting.



## Program 48: Customer and Product Exploration

After understanding the database structure, you will begin answering basic business questions. Using SQL filtering techniques such as WHERE, LIKE, BETWEEN, IN, IS NULL, ORDER BY, and LIMIT, you will search for customers, products, orders, and payments based on different business conditions.

This stage focuses on retrieving useful information from the database.

## Program 49: Business Performance Analysis

Now that the data can be retrieved efficiently, the management team wants summarized business reports. Using aggregate functions, GROUP BY, and HAVING, you will calculate total sales, average product prices, customer counts, revenue by category, city-wise sales, payment summaries, and many other key performance indicators (KPIs).

These reports will help managers understand overall business performance.

## Program 50: Advanced Customer and Sales Analytics

In the final stage of the project, management requests more advanced insights. Using JOINS and Subqueries, you will combine information from multiple tables to answer real business questions such as identifying the highest spending customers, products that have never been sold, best-selling categories, city-wise revenue, customer purchase history, and many other analytical reports.

This final program brings together everything learned throughout the SQL course.

## Expected Learning Outcomes

After completing this project, you will be able to:

- Work confidently with real-world datasets.
- Import CSV files into MySQL databases.
- Understand the structure of relational databases.
- Analyze customer and product information using SQL.
- Generate professional business reports.
- Solve practical data analysis problems.
- Apply SQL concepts in real industry scenarios.
- Think like a Data Analyst rather than simply writing SQL queries.

## // Program 47

### Database Setup and Data Exploration

Before any organization can analyse its business data, the information must first be collected, organized, and stored in a database. This preparation stage is one of the most important parts of every data analytics project. Even the most advanced SQL queries cannot produce meaningful results if the underlying data has not been properly organized.

Imagine joining a company as a Data Analyst on your very first day. Your manager does not immediately ask you to create reports or dashboards. Instead, the first task is to understand the company's database.

Questions such as the following naturally arise:

- What data is available?
- How many tables are present?
- What does each table represent?
- Which tables are related to each other?
- What information does every column contain?
- Are there missing values?
- What type of business can be understood from this data?

Professional data analysts always begin by exploring the data before performing any analysis. This process is known as **Data Exploration** or **Exploratory Data Analysis (EDA)**. Although EDA often includes visualization and statistical techniques, the first step in SQL-based projects is understanding the database structure.

### Why Import Existing Datasets?

Earlier in this book, we created small tables manually using the `CREATE TABLE` and `INSERT INTO` statements. That approach was useful for learning SQL syntax, but real organizations rarely enter thousands of records manually.

Instead, businesses collect data automatically through websites, mobile applications, billing software, payment gateways, inventory systems, and customer management software. These systems usually export their information into formats such as **CSV (Comma-Separated Values)**.

Rather than recreating these records ourselves, we will import an existing dataset into MySQL. This allows us to work with realistic business data and experience a workflow similar to that followed by professional data analysts.

Using a real dataset also introduces you to practical challenges such as understanding unfamiliar tables, interpreting column names, and discovering relationships between different entities.

## About the Dataset

For this project, we will work with a real-world e-commerce dataset instead of creating small sample tables manually. Using an actual business dataset allows us to experience the same workflow followed by professional data analysts in the industry.

The dataset chosen for this project is the **Brazilian E-Commerce Public Dataset by Olist**, one of the most popular datasets used for learning SQL, data analysis, and business intelligence. It contains real transactional data collected from an online marketplace, including information about customers, products, orders, sellers, payments, and reviews. The data is organized into multiple related CSV files, making it an excellent example of a relational database.

The complete dataset can be downloaded from Kaggle using the following link:

Dataset Name: Brazilian E-Commerce Public Dataset by Olist

Author: Olist

Download Link: <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>

After opening the dataset page, click the **Download** button to download the ZIP file. Extract the ZIP archive to obtain all the CSV files.

## Dataset Files Required for This Project

The original dataset contains several CSV files. However, for this mini project, we will focus on the following files:

| CSV File                                | Purpose                                                                           |
|-----------------------------------------|-----------------------------------------------------------------------------------|
| <b>olist_customers_dataset.csv</b>      | Stores customer information such as customer ID, city, and state.                 |
| <b>olist_products_dataset.csv</b>       | Contains product details including product category, dimensions, and weight.      |
| <b>olist_orders_dataset.csv</b>         | Stores order information such as order status, purchase date, and delivery dates. |
| <b>olist_order_items_dataset.csv</b>    | Contains information about the products included in each order.                   |
| <b>olist_order_payments_dataset.csv</b> | Stores payment details including payment type, installments, and payment value.   |

These five datasets provide everything required to perform customer analysis, product analysis, sales reporting, payment analysis, and advanced SQL queries using JOINS and Subqueries.

## **Creating the Database**

The first practical task is to create a dedicated database for the project. Keeping project data in a separate database helps organize information and prevents conflicts with other databases already present on the system. After creating the database, it will be selected as the active database using the USE statement. Once selected, every imported table will become part of this project database.

## **Importing CSV Files**

Modern database systems provide several methods for importing external data. MySQL Workbench includes an Import Wizard that automatically creates tables from CSV files and loads their records into the database.

During the import process, MySQL reads every row from the CSV file and converts it into records inside a relational table. This saves considerable time compared to manually inserting thousands of rows using SQL statements.

## **Understanding the Database Structure**

Once all files have been imported, the next task is understanding the database. A data analyst should never begin writing SQL queries immediately.

Instead, they should first answer questions such as:

- How many tables exist?
- What is stored inside each table?
- Which columns uniquely identify each record?
- Which columns connect one table to another?
- Which tables will be useful for customer analysis?
- Which tables will be useful for sales analysis?

Understanding the structure of the database makes later analysis much easier.

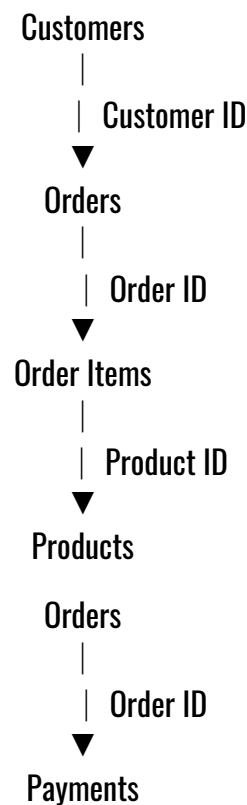
## **Exploring Tables**

SQL provides several commands for inspecting database objects. Some commands display available tables, while others reveal the structure of individual tables. Viewing the first few rows of each table also helps analysts understand the type of information stored in the dataset. Rather than memorizing column names, analysts become familiar with the business itself.

## Database Relationships

One of the strengths of relational databases is that information is distributed across multiple related tables. For example, A customer places an order. An order contains one or more products. Every order has one or more payments. Instead of storing all this information in one massive table, each entity is stored separately.

Their relationships can be represented as follows:



These relationships will become extremely important when learning SQL JOINS in the next programs.

### What Will We Explore?

During this program, you will learn how to:

- Create a new database.
- Import CSV datasets.
- Verify imported tables.
- Explore the structure of every table.
- Display sample records.
- Count available records.
- Understand relationships between tables.
- Identify primary business entities.

By the end of this program, the database will be completely prepared for analysis.

## // Program 48

### Customer & Product Exploration using SQL

The Retail Analytics database has now been successfully created, and all the required datasets have been imported into MySQL. During Program 1, you explored the database structure, examined the available tables, and became familiar with the information stored in each dataset.

Now it is time to begin your first analytical assignment.

On your second day at **RetailKart Analytics Pvt. Ltd.**, your manager assigns you a simple but important task. Before generating advanced business reports, the analytics team needs a clear understanding of the customers, products, and orders stored in the database.

Rather than calculating sales or predicting future trends, the objective of this stage is to explore the available data and answer everyday business questions. Managers frequently request information such as finding customers from a particular city, identifying products within a specific price range, locating recent orders, or viewing transactions made using a particular payment method.

Although these requests may appear simple, they form the foundation of every data analysis project. Before meaningful insights can be generated, analysts must first learn how to retrieve the right information efficiently.

In this program, you will use SQL filtering techniques to search, sort, and retrieve records from the database based on different business conditions.

#### Business Scenario

The management team has scheduled its monthly business review meeting. Before discussing sales performance, they require several preliminary reports that describe the current state of customers, products, and orders. As the Junior Data Analyst, your responsibility is to prepare these reports by writing SQL queries.

Each report answers a specific business question. Together, these reports will help management better understand their customer base and product catalog before moving on to deeper business analysis.

## What Will You Use?

Throughout this program, you will apply the SQL concepts learned earlier in this book to retrieve meaningful information from a real-world dataset.

You will practice using:

- SELECT
- WHERE
- ORDER BY
- LIMIT
- DISTINCT
- LIKE
- Wildcards (% and \_)
- BETWEEN
- IN
- IS NULL
- IS NOT NULL
- Multiple Conditions using AND, OR, and NOT

These SQL statements are among the most frequently used commands by data analysts because they help locate exactly the information required for business decision-making.

## Business Questions to Solve

Instead of writing SQL queries without context, imagine that each query is requested by a department within the company.

During this program, you will answer questions such as:

- Which customers belong to a particular city or state?
- Which products belong to a specific category?
- Which orders were placed within a particular time period?
- Which payment methods are most commonly used?
- Which products fall within a given price range?
- Which customers have missing information?
- Which orders are currently delivered, shipped, or cancelled?
- Which products start with a particular keyword?
- Which customers joined from a particular location?
- What are the first few records available in each business table?

Although these questions are simple, they represent the type of information that businesses request on a daily basis.

## Why Is Data Exploration Important?

Before performing sales forecasting, customer segmentation, or business intelligence reporting, analysts spend considerable time exploring their datasets. This stage helps answer questions such as:

- Is the data complete?
- Are there missing values?
- Which categories contain the most products?
- Which cities have the largest customer base?
- Which payment methods are available?
- Are there duplicate records?
- Does the imported dataset appear accurate?

Finding answers to these questions ensures that future analyses are built on reliable and well-understood data.

## SQL Techniques Used

Throughout this program, you will use various SQL clauses to retrieve information efficiently. The WHERE clause will filter records based on business conditions. LIKE and wildcards will help search for partial text matches. BETWEEN will retrieve records within a specified range. IN will simplify filtering using multiple values. IS NULL and IS NOT NULL will help identify incomplete records. ORDER BY will arrange the results in ascending or descending order. LIMIT will display only a selected number of rows for easier inspection.

By combining these techniques, you will learn how to write flexible and efficient SQL queries for real-world business problems.

## Manager's Assignment

Your manager has assigned you the following task: *"We have recently imported our customer, product, order, and payment data into the Retail Analytics database. Before the management meeting, I need a series of reports that help us understand our customers, products, and transactions. Use SQL to retrieve the requested information accurately and present it in an organized manner. These reports will serve as the foundation for our upcoming sales analysis."*

It is now your responsibility to write SQL queries that answer each business question.

## Program Overview

In this program, you will complete approximately **40 business queries** covering customer exploration, product exploration, order analysis, payment analysis, filtering, sorting, searching, and basic reporting. The knowledge gained here will prepare you for **Program 3**, where you will begin analysing business performance using aggregate functions, GROUP BY, and HAVING to generate meaningful management reports.



## // Program 49

### **Business Performance Analysis using SQL**

After successfully exploring the Retail Analytics database in Program 2, you have gained a good understanding of the available customers, products, orders, and payment information. The management team is satisfied that the imported data is complete and reliable.

Now begins the most important phase of the project. Simply viewing customer records or product details is no longer enough. The company now wants to understand how the business is performing.

As a Data Analyst, your role is changing. Instead of retrieving individual records, you must now summarize thousands of transactions into meaningful reports that help managers understand the overall health of the business. This process is called **Business Performance Analysis**.

Modern companies generate enormous amounts of data every day. Looking at individual transactions tells us very little about the company's performance. Business leaders are interested in answers to questions such as:

- How much revenue has the company generated?
- Which payment method is used the most?
- Which product categories are the most popular?
- Which states have the largest number of customers?
- Which order status appears most frequently?
- Which products generate the highest sales?
- Which customers place the most orders?

These questions cannot be answered by simply displaying records one by one. Instead, SQL must summarize thousands of rows into meaningful statistics. This is exactly what Aggregate Functions, GROUP BY, and HAVING are designed to do.

### **Business Scenario**

The monthly management meeting is approaching. The CEO, Sales Manager, Marketing Manager, Finance Department, and Operations Team all require reports before making important business decisions.

As the Junior Data Analyst, you have been asked to prepare these reports using SQL. Each department has submitted a list of business questions. Your responsibility is to write SQL queries that answer each question accurately. The reports you generate will help management understand customer behaviour, product performance, sales trends, and payment patterns.

## Why Business Analysis Matters

Raw data rarely helps managers make decisions. Imagine a table containing 100,000 customer orders. Reading every order individually would take several days. Instead, managers need concise reports.

For example:

- Total Revenue
- Average Order Value
- Number of Customers
- Orders per State
- Payments by Payment Method
- Most Popular Product Categories

These summaries allow businesses to identify trends, compare performance, monitor growth, and make informed decisions. This is why data analysts spend a significant amount of their time creating aggregated reports rather than simply retrieving records.

## SQL Concepts Used

Throughout this program, you will apply several SQL analytical functions and clauses.

Aggregate Functions

- COUNT()
- SUM()
- AVG()
- MIN()
- MAX()

Grouping

- GROUP BY

Filtering Groups

- HAVING

Together, these SQL features transform raw business data into useful information.

## Business Reports You Will Generate

During this program, you will answer questions such as:

- How many customers are present in the database?
- How many products are available?
- How many orders have been placed?
- What is the total payment collected?
- What is the average payment value?

- Which payment method is used most frequently?
- Which states have the largest customer base?
- Which product categories contain the most products?
- Which order statuses occur most often?
- Which payment methods generate the highest revenue?
- Which states have more than 500 customers?
- Which product categories have more than 100 products?
- Which payment methods generate average payments greater than ₹500?
- Which customer states contribute the largest number of orders?

Each query represents a real business report requested by management.

### **Expected Learning Outcomes**

After completing this program, you will be able to:

- Summarize large datasets.
- Generate business reports.
- Calculate KPIs (Key Performance Indicators).
- Analyze customer and payment behavior.
- Create grouped reports using GROUP BY.
- Filter summarized data using HAVING.
- Interpret business information from SQL results.

By the end of this program, you will no longer be retrieving data. You will be analyzing it.

### **Manager's Assignment**

Monthly Business Performance Review: "Excellent work exploring the database. We now have confidence in our data quality. The next step is to prepare management reports for this month's review meeting. I need summaries of customer distribution, product categories, payment statistics, and order performance. These reports will help each department understand how the business is performing and identify opportunities for improvement. Use Aggregate Functions, GROUP BY, and HAVING to generate meaningful business insights."

## // Program 50

### Advanced Customer & Sales Analytics using SQL

You have reached the final stage of the **Customers and Products Analysis Using SQL** project. During the previous programs, you successfully imported a real-world dataset, explored the available information, and generated several business reports using SQL. You learned how to retrieve records, filter data, summarize business performance, and generate management reports using aggregate functions.

The management team was impressed with your work. Now they have assigned you to a much more important project. Instead of answering simple questions about individual tables, you are now expected to analyse the business as a whole.

This means combining information from multiple tables, identifying customer purchasing behavior, understanding payment patterns, and discovering insights that cannot be obtained by looking at a single table alone.

This is where SQL becomes a true business intelligence tool. As a Data Analyst, your responsibility is not simply to retrieve data but to convert raw business records into meaningful insights that help managers make better decisions.

Throughout this program, you will combine customer, order, product, and payment information using **JOINS** and **Subqueries** to solve real-world business problems. Many of these queries resemble the reports generated every day by retail companies, banks, airlines, hospitals, and e-commerce organizations around the world.

#### Business Scenario

The company has successfully completed its monthly sales reporting. However, the management team wants to understand **why** certain trends are occurring. The CEO has organized a strategy meeting involving the Sales Department, Marketing Team, Finance Department, and Operations Team.

Each department has submitted a list of analytical questions that require information from multiple business tables. Some reports require customer information together with orders. Others require payments together with products. Several reports require comparing records against company averages or identifying exceptional customers.

These questions cannot be answered using a single table. Instead, they require SQL JOINS and Subqueries. As the company's Junior Data Analyst, your responsibility is to prepare these reports before tomorrow's meeting.

## Why Advanced Analytics Matters

Imagine that an online shopping company receives one million orders every month. Knowing the total revenue is useful. However, managers also want to know:

- Which customers contribute the highest revenue?
- Which states generate the most sales?
- Which products are frequently purchased together?
- Which customers haven't placed any orders?
- Which orders generated the largest payments?
- Which payment method is preferred for expensive purchases?
- Which customers spend more than the average customer?

These questions help businesses understand customer behaviour, improve marketing strategies, optimize inventory, and increase profitability. Without advanced SQL techniques, answering these questions would require significant manual effort.

## SQL Concepts Used

This program combines almost every concept learned throughout the course.

JOIN Operations:

- INNER JOIN
- LEFT JOIN
- RIGHT JOIN
- CROSS JOIN
- SELF JOIN

Advanced Queries:

- Scalar Subqueries
- Nested Queries
- IN Subqueries
- Aggregate Subqueries

By combining these techniques, SQL becomes capable of solving complex business problems.

## Business Questions You Will Solve

During this program, you will prepare advanced analytical reports for the management team. Some of the questions include:

- Which customers have placed the highest number of orders?
- Which customers have never placed an order?
- Which products have never been ordered?
- Which payment method generates the highest revenue?
- Which customers spend more than the average customer?
- Which product categories contribute the largest number of sales?

- Which states generate the highest revenue?
- Which orders contain multiple products?
- Which customers use installment payments most frequently?
- Which products are purchased most often?

These reports will help management understand customer behavior, identify business opportunities, and improve future decision-making.

### **Expected Learning Outcomes**

After completing this program, you will be able to:

- Combine information from multiple tables using JOINS.
- Generate advanced business reports.
- Solve real-world analytical problems.
- Write SQL queries using nested and subqueries.
- Identify customer purchasing patterns.
- Discover meaningful business insights.
- Apply SQL in professional Data Analyst workflows.

### **Final Assignment**

CEO's Business Intelligence Report: *"The monthly reports prepared by the analytics team have given us a good understanding of our overall business performance. However, before making strategic decisions for the next quarter, I need deeper insights into customer purchasing behavior, payment trends, and product performance. Combine information from multiple tables and prepare advanced reports that will help us understand our business more effectively. Use SQL JOINS and Subqueries wherever appropriate."*

# THE END!

If you've reached this page, take a moment to appreciate how far you've come. When you opened this book, SQL may have seemed like just another programming language filled with unfamiliar commands and confusing syntax. Today, you have built databases, created tables, inserted records, explored real-world datasets, analyzed business information, and generated professional reports using SQL.

More importantly, you have learned to think differently. Instead of simply writing queries, you have learned how to ask questions, explore data, discover patterns, and transform raw information into meaningful insights. That is the true purpose of SQL.

Throughout this journey, you worked with a real e-commerce dataset, solved practical business problems, and experienced the type of work performed by data analysts, database developers, and business intelligence professionals every day.

You have not simply learned SQL commands. You have taken your first step into the world of **Data Analytics**.

## What Should You Learn Next?

SQL is one of the most valuable skills in technology, but it is only one part of the data journey. Now that you understand relational databases and can confidently write SQL queries, you are ready to explore more advanced topics.

A natural learning path would be:

### Data Analysis

- Microsoft Excel
- Google Sheets
- Python for Data Analysis
- NumPy
- Pandas

### Data Visualization

- Power BI
- Tableau
- Matplotlib
- Seaborn

### Database Administration

- Database Design
- Indexing
- Views
- Stored Procedures

- Triggers
- Transactions
- Database Security

#### Modern Databases

- PostgreSQL
- SQL Server
- Oracle Database
- SQLite
- MongoDB
- Cloud Databases

#### Data Science & AI

- Statistics
- Machine Learning
- Artificial Intelligence
- Deep Learning

Each new skill builds upon the SQL knowledge you have developed in this book.

#### **Data.**

Every business, hospital, school, bank, government organization, and scientific institution depends on data to make decisions. SQL has remained one of the world's most important technologies for more than four decades because it solves a timeless problem: helping people understand and manage information.

Learning SQL is not just about writing database queries. It is about learning how information is organized, how businesses operate, and how decisions are supported by data. The skills you have developed here will continue to be valuable regardless of which programming language, framework, or career path you choose in the future.

#### **Keep Practicing**

The best way to master SQL is by solving real problems. Create your own databases. Download public datasets. Ask business questions. Experiment with different queries. Try writing a query before searching for the answer. Every query you write strengthens your understanding.

Remember: **Reading SQL teaches you the language. Writing SQL makes it yours.**



## Thank You!

*“Learning never truly ends. Every database you explore, every dataset you analyse, and every query you write will teach you something new. Keep asking questions. Keep exploring data. Keep building. Most importantly, **Never stop learning.**”*